

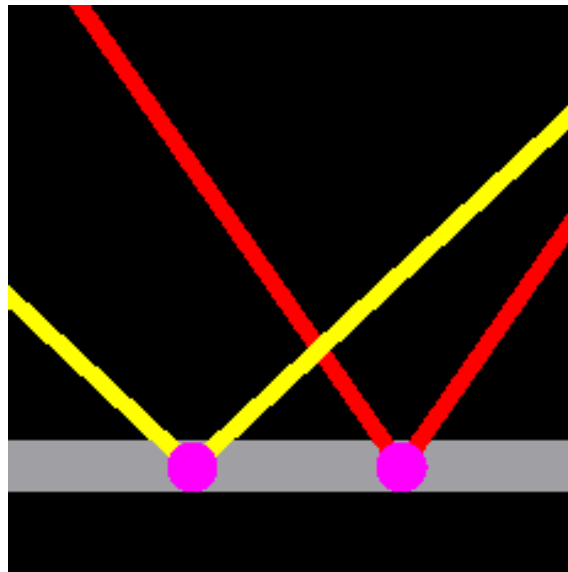
ELEC-H415: Communication channels
Professor: Philippe De Doncker

Project Report

5G Channel Modeling with Ray-Tracing

Lucas Placentino

2026



.....

BRUFACE
BRUSSELS FACULTY
OF ENGINEERING

.....



Contents

1	Introduction	2
2	Implementation & Modeling	2
2.1	Summary of Implemented Features	2
2.2	Application Interface and Usage	3
2.3	Propagation Hypotheses and Parameters	3
2.4	Sensitivity and Bit Rate Mapping	3
2.5	Received Power Computation	4
2.6	Image Method and Ray-Tracing	4
2.6.1	Principle of the method	5
2.6.2	Determination of the reflection point	5
2.6.3	Two-reflection case	5
2.6.4	Three-reflections case	5
3	Validation	6
3.1	Free-Space (Friis) Validation	6
3.2	Single Reflection Validation	6
3.3	Urban Canyon Validation (Dielectric Slab Modeled Walls)	7
4	Results	8
4.1	Environment Geometry	8
4.2	Received Power-Bitrate Coverage Map	8
4.3	Path Loss Law	9
4.3.1	Analysis of the Surrogate Model	10
5	Extensions/Extra Features	11
5.1	BS Coverage Quantification	11
5.2	Multi-BS Simulation	11
5.3	Power Delay Profile (PDP)	12
5.3.1	Derivation of the Tapped Delay Line (TDL) Model	13
6	Conclusion	13
	Appendix	14
A	Validation 2D Scenes	14
A.1	Free Space Validation	14
A.2	Single Reflection Validation	14
A.3	Urban Canyon Validation	14
B	Notice of AI Use	15
C	Code	15

Chapter 1

Introduction

The deployment of 5G small-cell networks operating at 26 GHz introduces severe propagation challenges. At these frequencies, signals experience high free-space path loss, total blockage by structural obstacles, and negligible diffraction. To avoid the computational overhead of performing continuous geometric ray-tracing during higher-level network performance evaluations, this project aims to extract a simplified, empirical surrogate channel model. Using a custom deterministic 2D ray-tracing engine built in C++, we simulate the horizontal-plane propagation of a 20 dBm Base Station (BS) in a simplified urban environment. By calculating the coherent superposition of multipath rays, the simulator evaluates the spatially averaged received power ($\langle P_{RX} \rangle$) of User Equipments (UE), spatial coverage limits, and achievable bitrate. Finally, the deterministic data is compressed via statistical regression to extract the canonical path loss model parameters: the path loss exponent (n) and the log-normal shadowing variance (σ_L).

Chapter 2

Implementation & Modeling

2.1 Summary of Implemented Features

The simulator application built for this project presents the following features:

- Simulate using ray-tracing the propagation of 5G waves,
- Compute path loss and draw validation simulations,
- Show the received total coherent power and bitrate of each UE/cell,
- Change the map resolution (from 1 square meter down to 12.5×12.5cm),
- Preset map of 120×70m,
- Add multiple Base Stations (BS), each with their position and transmitting power,
- Show the connected BS of each UE/cell,
- Simulate and draw all rays only to a single UE/cell,
- Compute and plot a path loss model for the simulation,
- Compute each UE/cell's tap power profile according to the Tapped Delay Line (TDL) model,
- Plot the Power Delay Profile (PDP), showing the multipath components, of a single UE/cell,
- Save simulation and graph windows to an image,
- Possibility to remove all walls.

2.2 Application Interface and Usage

To facilitate network coverage analysis, the simulator features an interactive graphical user interface (see Figure 2.1) that allows users to configure and run propagation simulations without modifying the underlying code. Through a side control panel, users can select the analysis mode (a global *Coverage Heatmap* or a targeted *Rays to Single Cell* diagnostic), adjust the spatial grid resolution down to 12.5×12.5 cm, and dynamically deploy multiple base stations by defining their exact spatial coordinates and transmission power. The interface also provides tools to modify the physical environment, such as a toggle to instantly remove all walls for free-space baseline comparisons.

Upon execution, the engine calculates the propagation paths and projects the results onto an interactive 2D map. In heatmap mode, the coverage quality is displayed using a logarithmic color gradient that represents the received power and achievable bitrate. Alternatively, the targeted mode visually traces the multipath trajectories, color-coding the direct and reflected rays. Users can hover over any map cell to instantly view its exact metrics via a tooltip. Furthermore, the application allows users to extract the temporal dispersion metrics by computing the Power Delay Profile (PDP) for any selected cell, execute predefined validation scenarios, export the final visual rendering to an image, and compute and plot the empirical path law based on the simulation result.

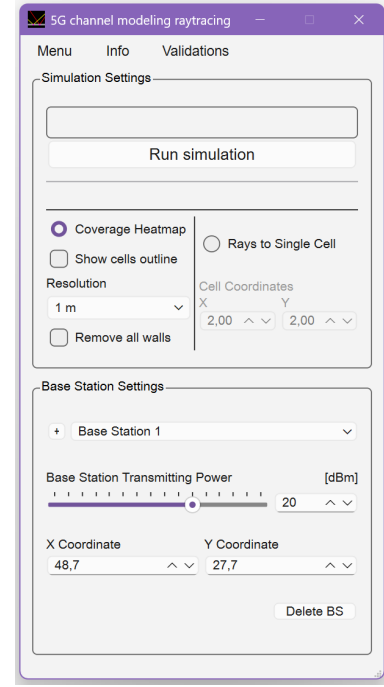


Figure 2.1: Screenshot of the simulator application

2.3 Propagation Hypotheses and Parameters

The simulation operates under far-field assumptions where waves are treated as locally plane. The approach is only two-dimensional; the BS and UE are co-planar, and only horizontal propagation is considered.

The system parameters, coming from the project guide, are defined as follows:

- Operating Frequency: $f = 26$ GHz ($\lambda \approx 1.15$ cm), with a bandwidth $B = 200$ MHz.
- Transmit Power: $P_{TX} = 20$ dBm (can be varied in the application).
- Antennas: Both TX and RX antennas are lossless vertical $\lambda/2$ dipoles (electric field is perpendicularly polarized), linear gain of $G = 1.64$.
- Obstacles: All walls are modeled with a relative permittivity of $\varepsilon_r = 4$.

2.4 Sensitivity and Bit Rate Mapping

The achievable communication bit rate R_b is constrained by the UE sensitivity S . Rather than using a Shannon model, we need to use the provided empirical UE hardware model which is, according to the project guidelines, that S (in dBm) varies linearly with $\log_{10}(R_b)$. Given the boundary conditions $S(-80 \text{ dBm}) \rightarrow 100$ Mbps and $S(-60 \text{ dBm}) \rightarrow 1$ Gbps, we derive the log-linear interpolation:

$$\log_{10}(R_{b,\text{Mbps}}) = 0.05 \cdot S_{\text{dBm}} + 6$$

Therefore, the maximum achievable bit rate at any spatial coordinate is:

$$R_b = 10^{(0.05 \cdot P_{RX} + 6)} \quad [\text{Mbps}]$$

If $P_{RX} < -80$ dBm, the area is classified as a dead zone (out of coverage). If $P_{RX} \geq -60$ dBm, the data rate is capped at 1 Gbps.

2.5 Received Power Computation

To compute the deterministic multipath components, the engine evaluates the environment using the method of images. For a given UE location, the algorithm identifies the direct Line-of-Sight (LOS) path and computes reflection paths up to the third order. Diffraction is explicitly neglected.

Because the transmit and receive antennas are vertical dipoles, the electric field is polarized normal to the horizontal plane of incidence (perpendicular polarization). When a wave intersects a wall, the single-interface reflection coefficient is calculated as:

$$\Gamma_{\perp}(\theta_i) = \frac{\cos \theta_i - \sqrt{\varepsilon_r - \sin^2 \theta_i}}{\cos \theta_i + \sqrt{\varepsilon_r - \sin^2 \theta_i}} \quad (2.1)$$

where θ_i is the angle of incidence.

Because the walls have a finite thickness l , the engine accounts for infinite internal reflections within the material. The total reflection (Γ_m) and transmission (T_m) coefficients are evaluated using the Dielectric Slab Model:

$$\Gamma_m(\theta_i) = \Gamma_{\perp}(\theta_i) \frac{1 - e^{-2jq}}{1 - \Gamma_{\perp}^2(\theta_i) e^{-2jq}} \quad (2.2)$$

$$T_m(\theta_i) = \frac{(1 - \Gamma_{\perp}^2(\theta_i)) e^{-jq}}{1 - \Gamma_{\perp}^2(\theta_i) e^{-2jq}} \quad (2.3)$$

where $q = \beta l \sqrt{\varepsilon_r - \sin^2 \theta_i}$ represents the electrical phase delay of the wave traveling through the thickness of the material, and $\beta = \frac{2\pi}{\lambda}$ is the phase constant.

To compute the received power, the algorithm evaluates the local spatial average power $\langle P_{RX} \rangle$, which explicitly removes small-scale fading (phase interference) to represent macroscopic coverage (path loss and shadowing). Rather than simulating the intermediate electric field magnitudes and complex phase shifts, the model calculates the individual power of each ray using an extended form of the Friis transmission equation.

For the k -th ray traveling a total unfolded distance d_k , the combined complex coefficient C_k resulting from r reflections and t transmissions is:

$$C_k = \left(\prod_{u=1}^r \Gamma_{m,u} \right) \left(\prod_{v=1}^t T_{m,v} \right) \quad (2.4)$$

The expected power contribution P_k of this individual ray, accounting for both the free-space spreading loss and the material interface interactions, is:

$$P_k = P_{TX} G_{TX} G_{RX} \left(\frac{\lambda}{4\pi d_k} \right)^2 |C_k|^2 \quad (2.5)$$

The macroscopic local average received power delivered to the antenna is then obtained through the incoherent sum of the powers from all K arriving multipath components:

$$\langle P_{RX} \rangle = \sum_{k=1}^K P_k \quad [\text{W}] \quad (2.6)$$

This linear average power is then converted to dBm to evaluate the UE sensitivity limits and extract the final surrogate path loss model.

2.6 Image Method and Ray-Tracing

Based on [PH24], to model all the waves (with one or two reflections) that are transmitted from the transmitter to the receiver, the image method was used. This method relies on the concept of image antennas, which are fictitious antennas placed in such a way as to simulate the effects of reflections.

2.6.1 Principle of the method

The principle of the image method is to replace the reflections of waves on surfaces with fictitious sources, called images, located on the other side of these surfaces. Each reflection on a surface can be modeled by an image located symmetrically with respect to this surface.

2.6.2 Determination of the reflection point

Let us consider a transmitting antenna TX , placed at a point on the right side of a wall. The position of the reflection point can be calculated as follows:

$$P_r \equiv \vec{x}_0 + t\vec{u} \text{ with } t = \frac{d_y(r_{ix} - x_0) - d_x(r_{iy} - y_0)}{u_x d_y - u_y d_x}$$

with therefore, $\vec{x}_0$ the origin position of the wall, $\vec{u}$ the direction vector of the wall, r_{ix} and r_{iy} the coordinates of the receiving antenna, and d_x , d_y , u_x , and u_y the components of the direction vectors.

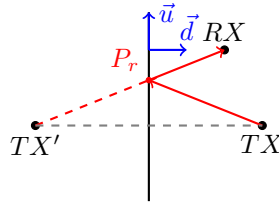


Figure 2.2: Image antenna TX' of the transmitter TX [PH24]

2.6.3 Two-reflection case

For the two-reflection case, it is sufficient to first take into account the image of the first wall on which the reflection occurs (I_1 in the diagram), and then from this image point, we determine its own image point with respect to the second wall on which the reflection occurs (I_2).

We then consider that the reflection point on the second wall (Pr_2) is the intersection between the said wall and the receiver, and for the first reflection point (Pr_1), it is the intersection between the first image and Pr_2 . This is illustrated in Figure 2.3. It is relevant to note that the total distance of the ray (composed of the three green lines) is equal to the distance between the receiver (RX) and the last image point (I_2).

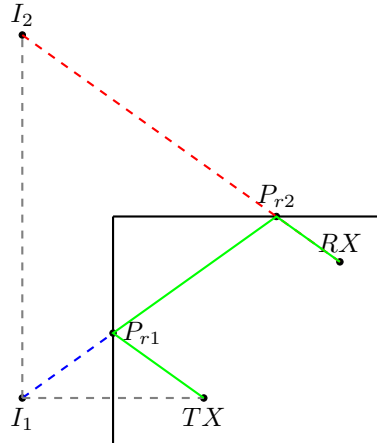


Figure 2.3: Method of images for two reflections between TX and RX [PH24]

2.6.4 Three-reflections case

The same algorithm can be applied to find the next image point (I_3) through the next wall to get the third reflection point (Pr_3), and so on in order to build the three-reflections ray path. A recursive algorithm can be designed in order to compute even more reflections.

Chapter 3

Validation

Before evaluating the simulated urban topology, the core physics engine was validated against theoretical analytical models to ensure accurate path length computation, phase accumulation, reflection scaling, and dielectric slab modeling.

3.1 Free-Space (Friis) Validation

The engine was evaluated in an obstacle-free vacuum. The simulated received power perfectly overlapped the theoretical Friis transmission equation, verifying the $1/d^2$ decay and correct antenna gain scaling (see Figure 3.1).

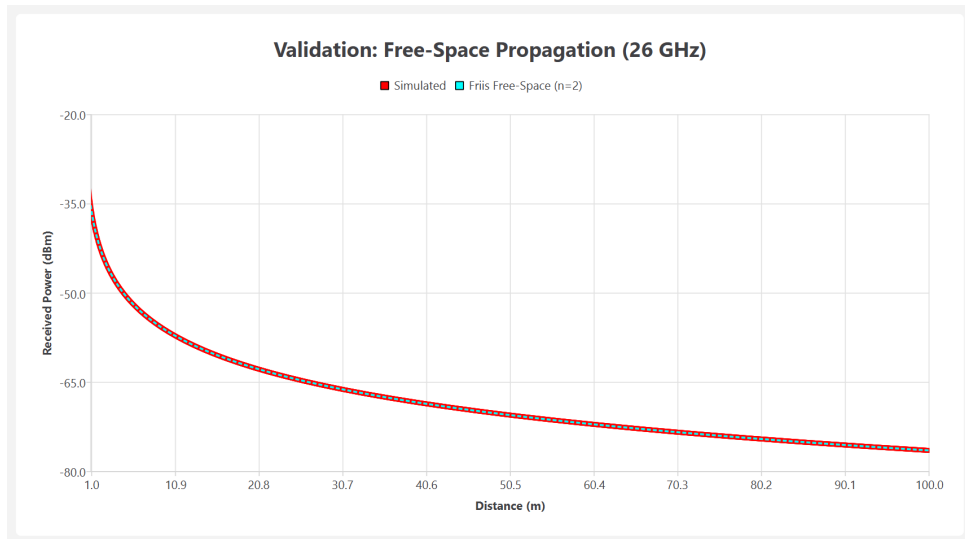


Figure 3.1: Free-space path loss model validation (linear scale x-axis)

3.2 Single Reflection Validation

A single infinitely long wall was introduced parallel to the propagation path (analogue to Over-the-Ground when $d \gg h$, we can just interpret the distance of the BS to the wall as its height $h_{TX} = h_{RX} = w$). The resulting interference between the direct ray and the single reflected ray accurately produced deep fading nodes (destructive interference) at mathematically predicted spatial intervals, with the envelope transitioning correctly to a $1/d^4$ decay rate. The simulation was extended to 10km in order to see this convergence of the empirical result to the Over-the-Ground model at those very large distances. The breakpoint ($d_{BP} = 4\pi w^2/\lambda$) is also computed and shown on the graph (see Figure 3.2).

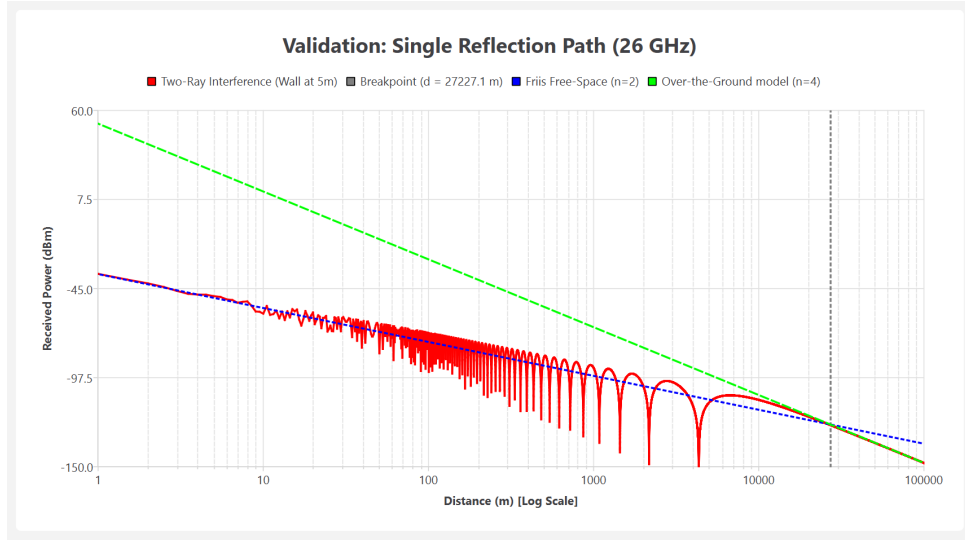


Figure 3.2: Two-ray (single reflection) path loss model validation (logarithmic scale x-axis)

3.3 Urban Canyon Validation (Dielectric Slab Modeled Walls)

The UE was swept down the centerline of a 10m-wide corridor bounded by two parallel dielectric slabs ($\epsilon_r = 4$, thickness = 40 cm). The coherent sum of 1-bounce, 2-bounce, and 3-bounce paths shows the expected path loss result, validating the implementation of the complex dielectric slab reflection coefficient Γ_m (See Figure 3.3).

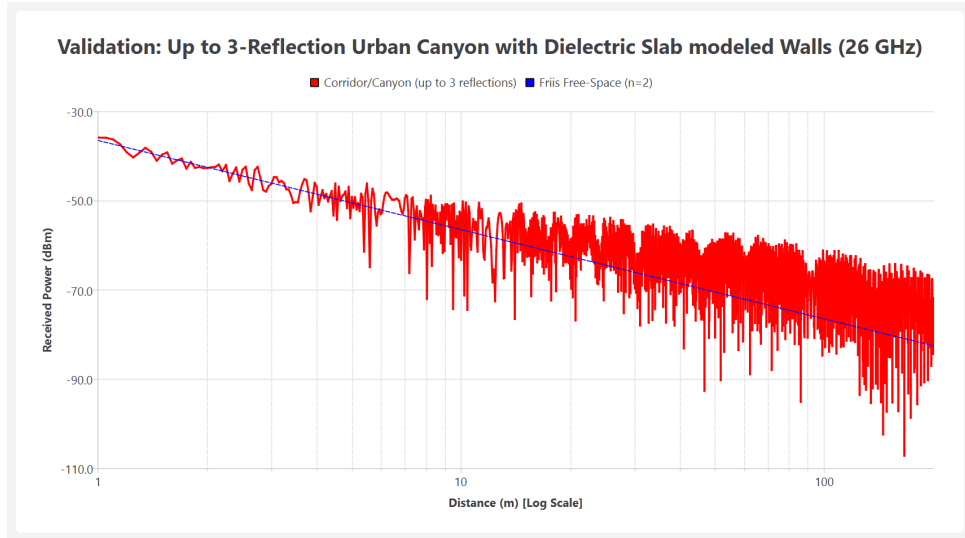


Figure 3.3: Up to 3-reflection urban canyon path loss model using dielectric slab modeled walls (logarithmic scale X-axis)

Chapter 4

Results

4.1 Environment Geometry

The chosen simulation map is a simplified 2D urban layout measuring 120×70 meters. It features a main street intersected by slightly narrower street, including a cut corner and chunk, and a small cul-de-sac alleyway (See Figure 4.1).

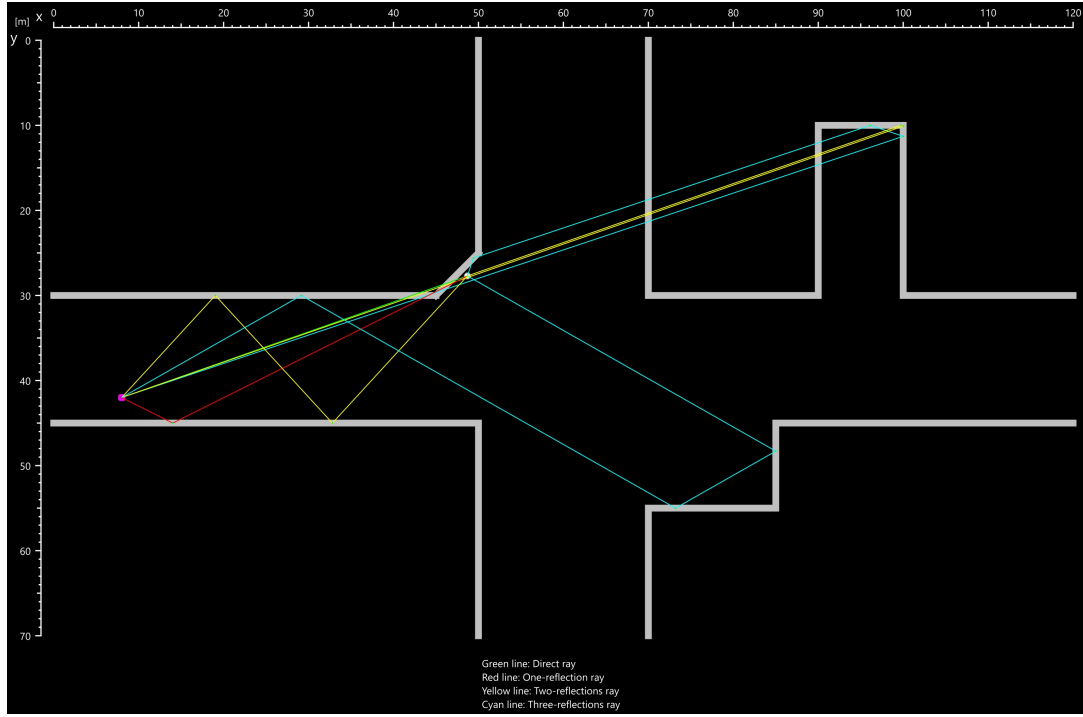


Figure 4.1: Preset simulation map geometry (with single cell simulation ran, showing rays' paths)

To extract the spatial channel parameters, the environment was discretized into a grid of local areas measuring 1×1 meter (or smaller, at the cost of computation time and memory usage). The complex electric field was evaluated at the center of each local area, allowing the extraction of the average spatial power $\langle P_{RX} \rangle$ while maintaining enough resolution to capture localized shadowing transitions. Walls are modeled as idealized 1D segments for reflection geometry, though their defined thickness is utilized internally to compute the dielectric slab transmission losses.

4.2 Received Power-Bitrate Coverage Map

The simulation output (see Figure 4.2) shows the 2D heatmap of $\langle P_{RX} \rangle$ and reveals distinct propagation zones. The two streets, acting mostly as direct LOS waveguides, maintain power levels well above the -60 dBm threshold, guaranteeing high data throughput. Conversely, deep NLOS regions such as the small alleyway rely only on 2nd and 3rd order reflections (as well as direct, transmitted through walls, rays). In some NLOS regions, transmission losses drop the power, sometimes below

−80 dBm resulting in dead zones.

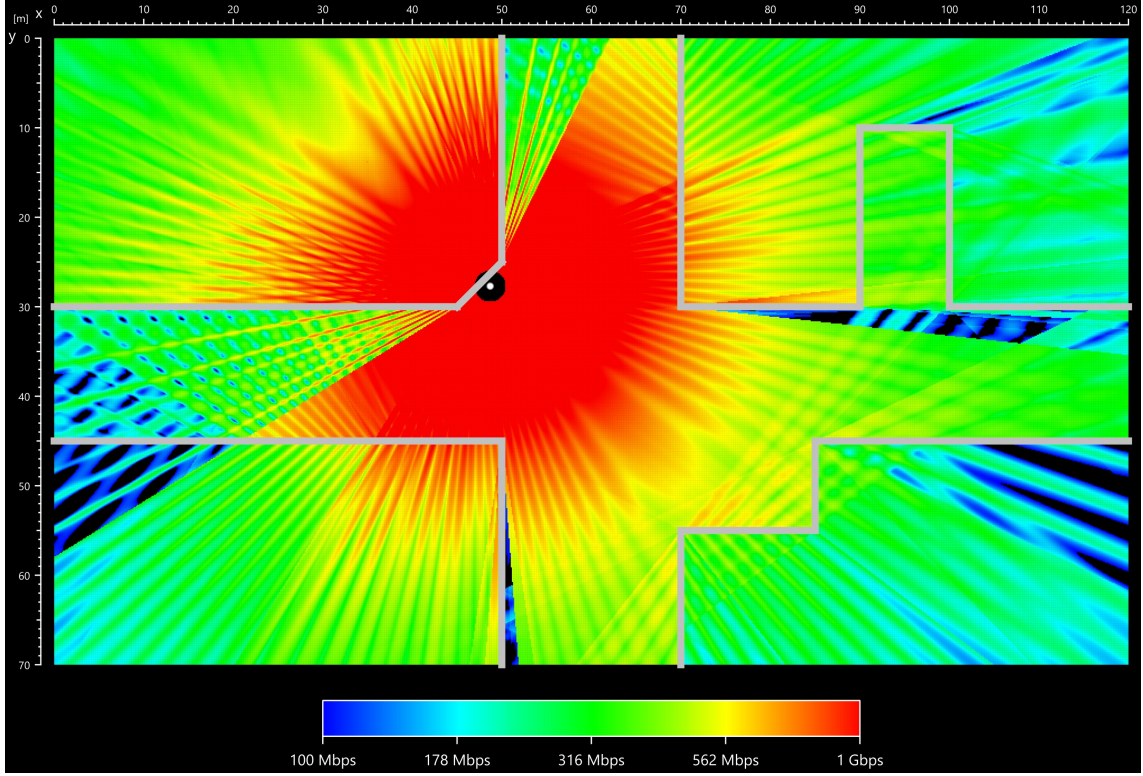


Figure 4.2: Coverage map result for $P_{TX} = 20\text{dBm}$ and $12.5 \times 12.5\text{cm}$ resolution cells

Hovering with the mouse over any cell shows its coordinates, received power (in mW and dBm), bitrate, and which BS it is connected to (See Figure 4.3).

The non-far-field cells around the BS are ignored (received power set to 0) during the simulation when $d < r_{ff}$

$$\text{where } r_{ff} = \text{Max} \left\{ 1.6\lambda ; 5D ; \frac{2D^2}{\lambda} \right\}$$

with D representing the maximum dimension of the antenna. These are thus painted black.

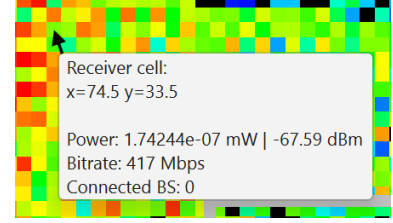


Figure 4.3: Tooltip shown on mouse hover of a cell

4.3 Path Loss Law

One of the goals of the simulation is to condense the deterministic data into a statistical surrogate model. Following the canonical path loss framework, the average received power is modeled as:

$$\langle P_{RX}(d) \rangle [dBm] = \langle P_{RX}(d_0) \rangle - 10n \log_{10} \left(\frac{d}{d_0} \right) + X_\sigma$$

where n is the path loss exponent and X_σ is a zero-mean Gaussian random variable representing shadowing variability, with standard deviation σ_L .

By plotting the path loss of all *valid* cells against the (log of the) distance and applying a least-squares linear regression for a BS with $P_{TX} = 20$ dBm at its default coordinates (48.7 ; 27.7)m, we get the following parameters for our specific urban geometry (see Figure 4.4 for linear x-axis, and Figure 4.5 for logarithmic x-axis):

- Path Loss Exponent (PLE): $n = 2.07$
- Shadowing Standard Deviation: $\sigma_L = 2.9$ dB

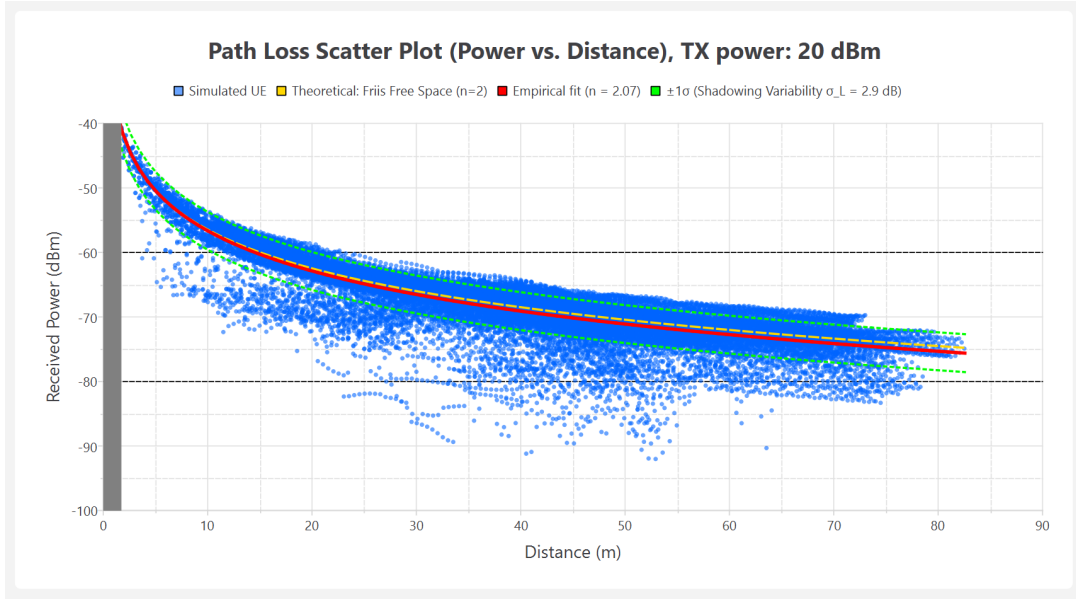


Figure 4.4: Path loss law plot from empirical simulation results (linear scale X-axis)

The horizontal gray dotted lines represent the maximum and minimum UE sensitivity (for the bitrate computation), and the gray area the non-far-field not-simulated/ignored zone.

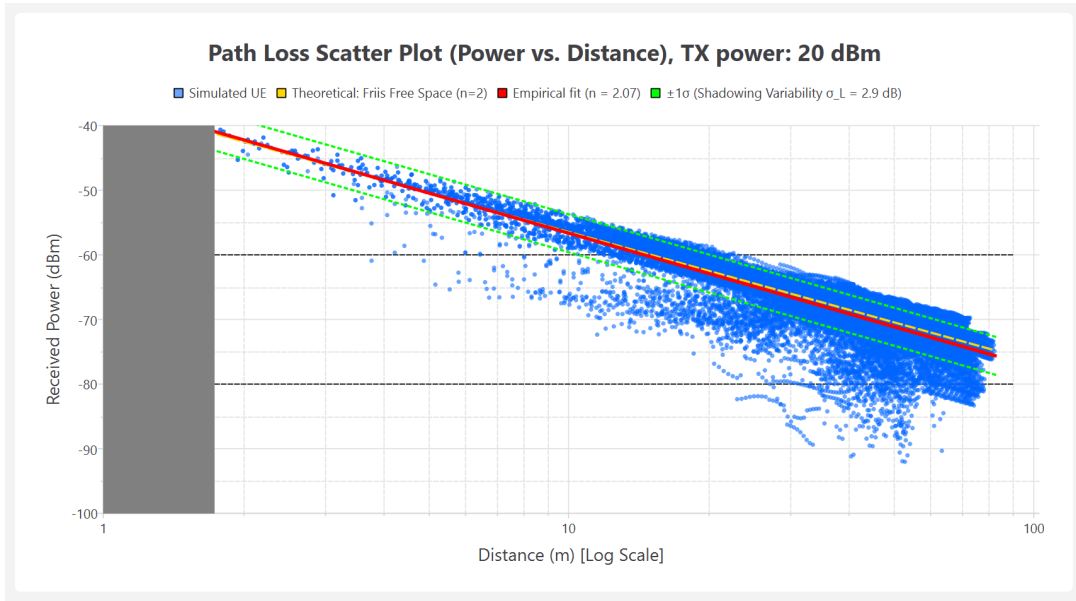


Figure 4.5: Path loss law plot from empirical simulation results (logarithmic scale X-axis)

These two parameters form the final surrogate model, allowing future network simulations to bypass these expensive ray-tracing computations entirely.

4.3.1 Analysis of the Surrogate Model

The extracted PLE $n = 2.09$ is physically consistent with the simulated environment. While heavy urban clutter typically drives n towards 3 or 4, this simulated environment contains strong waveguiding corridors and flat, highly reflective generic walls that minimize diffusion. The high proportion of LOS and waveguide-assisted NLOS cells brings the path loss closer to the free-space (Friis) model (value of $n = 2.0$).

Furthermore, the standard deviation $\sigma_L = 2.9$ dB accurately quantifies the macroscopic shadowing (slow fading) caused by the structural topology. While this value is lower than the typical 4 to 10 dB range observed in empirical outdoor urban measurements, it is a logical outcome of the

simulation's parameters. Because the deterministic 2D model strictly evaluates primary reflections and transmissions only, the path loss data fits the $n = 2.14$ regression model much more tightly. Ultimately, this validates the surrogate model for future computationally efficient simulations.

Chapter 5

Extensions/Extra Features

5.1 BS Coverage Quantification

Hovering with the mouse over a BS shows a tooltip containing its details (coordinates, G_{TX} , P_{TX}) and its coverage area of the map, in square meters and in percentage of total map area. (See Figure 5.1).

To evaluate the efficiency of a base station deployment, the simulator numerically quantifies the total coverage area. The coverage footprint is defined as the spatial integral of all local areas where the received power exceeds the minimum sensitivity threshold ($S_{min} = -80$ dBm). In our discretized grid, this is computed as:

$$A_{coverage} = \sum_{i=1}^N A_{cell} \cdot \mathbf{1}_{\{\langle P_{RX} \rangle_i \geq S_{min}\}} \quad [\text{m}^2]$$

where A_{cell} is the physical area of a single simulation cell (e.g., 1 m^2) and $\mathbf{1}$ is the indicator function.

In multi-BS scenarios, this metric dynamically updates to reflect only the cells logically connected to each specific transmitter.

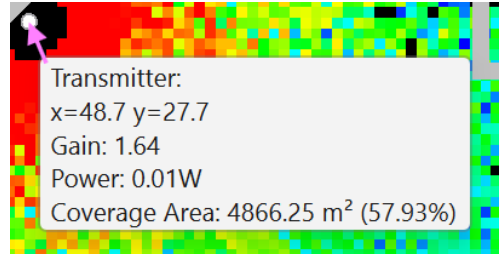


Figure 5.1: Tooltip shown on mouse hover of a BS

5.2 Multi-BS Simulation

In complex urban environments, a single base station is rarely sufficient to provide continuous coverage. Deep NLOS regions suffer from severe penetration and reflection losses, creating coverage dead zones where the received power falls below the minimum sensitivity threshold of -80 dBm. To mitigate this, the simulator was extended to support the deployment of multiple base stations.

When a second BS is introduced into the topology, the ray-tracing engine evaluates the propagation paths from both transmitters to every local area in the simulation grid independently. In a real-world cellular network, a UE evaluates the Reference Signal Received Power (RSRP) from surrounding cells and connects to the one offering the highest link quality. To emulate this, the simulator does not coherently or incoherently sum the powers from different base stations, as they operate as distinct logical links. Instead, it simply selects and saves the maximum among the multiple BS:

$$\text{For } n \text{ BS : } P_{RX} = \max(P_{RX,BS1}, P_{RX,BS2}, \dots, P_{RX,BSn}) \quad [\text{dBm}]$$

By deploying a second BS at a strategic complementary location, the previously identified suboptimal coverage areas benefit better received power, improving the overall bitrate over the simulated map (See Figure 5.2).

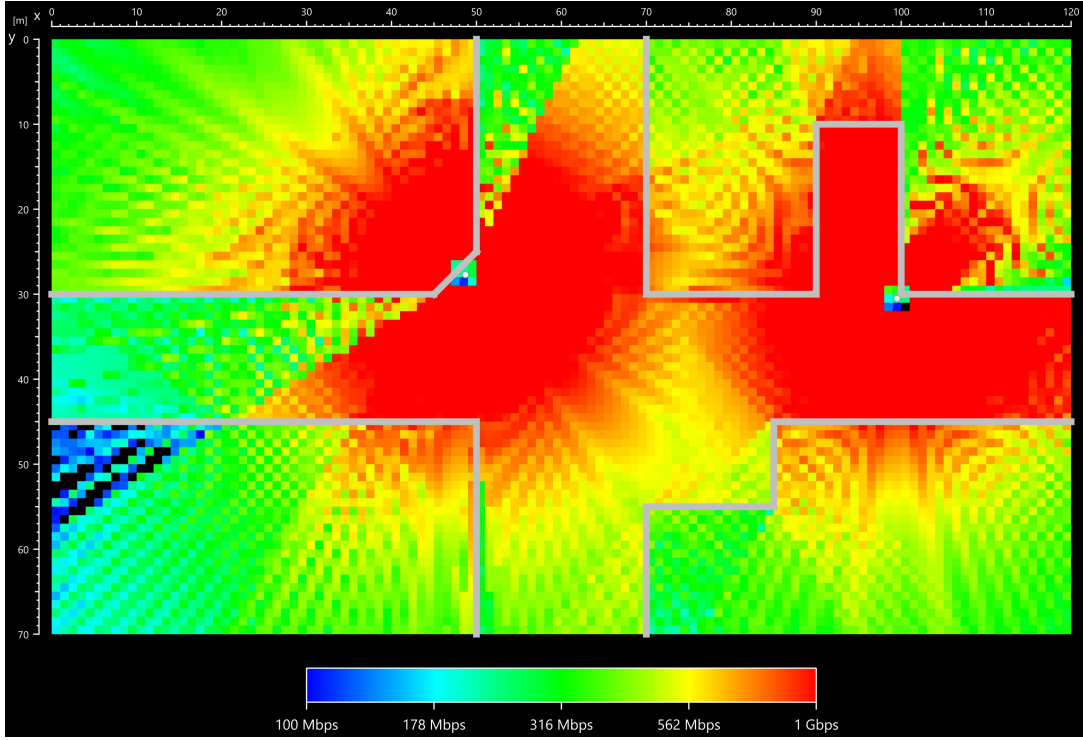


Figure 5.2: Example of simulation containing 2 BS ($1m^2$ resolution)

5.3 Power Delay Profile (PDP)

While the primary objective of the surrogate model is to map the macroscopic path loss ($\langle P_{RX} \rangle$), the deterministic nature of the ray-tracing engine intrinsically captures the wideband properties of the channel. Because the simulator computes and saves the exact physical path length d_k of every k -th multipath component for every UE/cell, the temporal dispersion of the channel can be extracted.

The absolute time of arrival, or propagation delay τ_k , for each ray is strictly a function of its traveled distance and the speed of light: $\tau_k = \frac{d_k}{c}$

By plotting the calculated received power of each individual ray against its delay τ_k , the discrete Power Delay Profile (PDP) is obtained. This was computed for specific LOS and NLOS cells, clearly demonstrating the arrival timeline of the multipath components. The graph on Figure 5.3 exhibits the classic LOS signature: a dominant first peak representing the unobstructed direct path. The subsequent taps arriving at later delay intervals represent the 1st, 2nd, and 3rd order reflections, which are attenuated due to the extra distance traveled and the reflection losses.

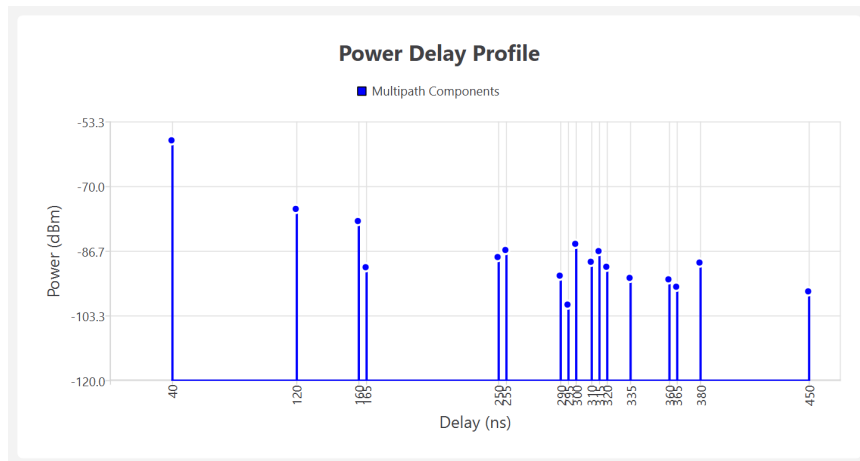


Figure 5.3: Example Power Delay Profile (PDP) for a specific LOS cell

Conversely, the PDP on Figure 5.4 for the NLOS scenario illustrates a different channel behavior. Because the direct path is obstructed by the buildings' geometry, the initial peak is not dominant, being attenuated by transmission losses through the walls. Instead, the dominant peak is a later-arriving multipath component generated by reflections. This results in a flatter overall profile with significantly lower received power, highlighting the severe shadowing effects.

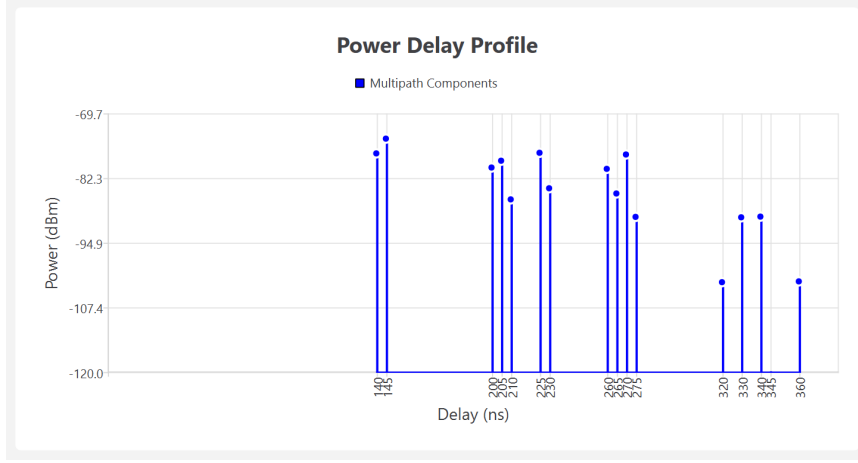


Figure 5.4: Example Power Delay Profile (PDP) for a specific NLOS cell

5.3.1 Derivation of the Tapped Delay Line (TDL) Model

The extraction of the PDP enables the creation of a TDL model for emulating wideband channel impulse responses in digital baseband simulations.

A communication system with a bandwidth of $B = 200$ MHz (as given in the project guide) has a time-domain resolution of $\Delta\tau = 1/B = 5$ ns. The deterministic rays generated by our simulator can be quantized into time bins of width $\Delta\tau$. Rays falling into the same time bin are coherently summed to form a single "tap." Consequently, the ray-tracer output can be perfectly translated into a baseband Finite Impulse Response (FIR) filter:

$$h_{TDL}(\tau) = \sum_{l=0}^L h_l \delta(\tau - l\Delta\tau)$$

where h_l represents the complex coefficient of the l -th tap, calculated as the coherent sum of all multipath components falling within the l -th delay interval, and $l\Delta\tau$ is its quantized delay. This proves that the developed ray-tracing software is a complete tool capable of parameterizing 5G TDL channel models.

Chapter 6

Conclusion

A deterministic 2D ray-tracer was successfully developed and mathematically validated to simulate 26 GHz 5G propagation. By modeling a simplified urban topology, computationally expensive deterministic ray-tracing was transformed into an efficient statistical surrogate model, characterized by a path loss exponent of $n = 2.14$ and a log-normal shadowing standard deviation of $\sigma_L = 5.1$ dB. Furthermore, the engine's capabilities were extended beyond narrowband coverage to evaluate multi-BS simulations, and to capture wideband temporal dispersion via Power Delay Profiles (PDP) for the parameterization of Tapped Delay Line (TDL) models.

In summary, this project successfully translates physical electromagnetic interactions into the essential channel metrics required for 5G network dimensioning and planning using simplified statistical models.

Appendix A

Validation 2D Scenes

A.1 Free Space Validation



Figure A.1: 2D ray-tracing scene of the free space validation

A.2 Single Reflection Validation

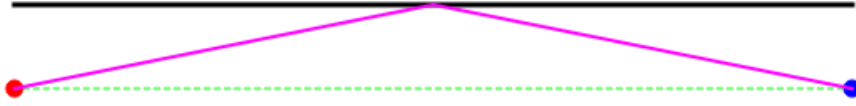


Figure A.2: 2D ray-tracing scene of the single reflection validation

A.3 Urban Canyon Validation

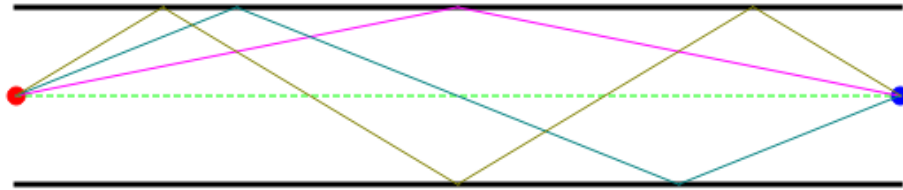


Figure A.3: 2D ray-tracing scene of the urban canyon validation (only half of the rays shown for clarity, fully symmetric)

Appendix B

Notice of AI Use

AI/LLMs were utilized during this project in order to help rephrase and help the wording of some parts of this report.

Appendix C

Code

All the code is also available in the project's Github repository: https://github.com/LucasPlacentino/5G_channel_modeling_raytracing.

C.1 CMakeLists.txt

```
1  cmake_minimum_required(VERSION 3.5)
2
3  project(5G_channel_modeling_raytracing VERSION 0.1 LANGUAGES CXX)
4
5  set(CMAKE_AUTOUIC ON)
6  set(CMAKE_AUTOMOC ON)
7  set(CMAKE_AUTORCC ON)
8
9  set(CMAKE_CXX_STANDARD 17)
10 set(CMAKE_CXX_STANDARD_REQUIRED ON)
11
12 find_package(QT NAMES Qt6 Qt5 REQUIRED COMPONENTS Widgets Charts)
13 find_package(Qt${QT_VERSION_MAJOR} REQUIRED COMPONENTS Widgets Charts)
14
15 set(PROJECT_SOURCES
16     main.cpp
17     mainwindow.cpp
18     mainwindow.h
19     mainwindow.ui
20 )
21
22 if(${QT_VERSION_MAJOR} GREATER_EQUAL 6)
23     qt_add_executable(5G_channel_modeling_raytracing
24         MANUAL_FINALIZATION
25         ${PROJECT_SOURCES}
26         validate_freespace.h validate_freespace.cpp
27         validate_wall.h validate_wall.cpp
28         validate_corridor.h validate_corridor.cpp
29         validate_common_utils.h validate_common_utils.cpp
30         simulation.h simulation.cpp
31         ray.h ray.cpp
```

```

32     obstacle.h obstacle.cpp
33     transmitter.h transmitter.cpp
34     receiver.h receiver.cpp
35     parameters.h
36     raysegment.h raysegment.cpp
37     resources.qrc
38
39
40
41
42
43
44 )
45 # Define target properties for Android with Qt 6 as:
46 #   set_property(TARGET 5G_channel_modeling_raytracing APPEND PROPERTY QT_ANDROID_PACKAGE_SOURCE_DIR
47 #               ${CMAKE_CURRENT_SOURCE_DIR}/android)
48 # For more information, see https://doc.qt.io/qt-6/qt-add-executable.html#target-creation
49 else()
50     if(ANDROID)
51         add_library(5G_channel_modeling_raytracing SHARED
52                     ${PROJECT_SOURCES}
53                     )
54 # Define properties for Android with Qt 5 after find_package() calls as:
55 #   set(ANDROID_PACKAGE_SOURCE_DIR "${CMAKE_CURRENT_SOURCE_DIR}/android")
56     else()
57         add_executable(5G_channel_modeling_raytracing
58                       ${PROJECT_SOURCES}
59                       )
60     endif()
61 endif()
62
63 target_link_libraries(5G_channel_modeling_raytracing PRIVATE
64     Qt${QT_VERSION_MAJOR}::Core
65     Qt${QT_VERSION_MAJOR}::Gui
66     Qt${QT_VERSION_MAJOR}::Widgets
67     Qt${QT_VERSION_MAJOR}::Charts
68 )
69
70 # Qt for iOS sets MACOSX_BUNDLE_GUI_IDENTIFIER automatically since Qt 6.1.
71 # If you are developing for iOS or macOS you should consider setting an
72 # explicit, fixed bundle identifier manually though.
73 if(${QT_VERSION} VERSION_LESS 6.1.0)
74     set(BUNDLE_ID_OPTION MACOSX_BUNDLE_GUI_IDENTIFIER com.example.5G_channel_modeling_raytracing)
75 endif()
76 set_target_properties(5G_channel_modeling_raytracing PROPERTIES
77     ${BUNDLE_ID_OPTION}
78     MACOSX_BUNDLE_BUNDLE_VERSION ${PROJECT_VERSION}
79     MACOSX_BUNDLE_SHORT_VERSION_STRING ${PROJECT_VERSION_MAJOR}.${PROJECT_VERSION_MINOR}
80     MACOSX_BUNDLE TRUE
81     WIN32_EXECUTABLE TRUE
82 )
83
84 include(GNUInstallDirs)
85 install(TARGETS 5G_channel_modeling_raytracing
86     BUNDLE DESTINATION .
87     LIBRARY DESTINATION ${CMAKE_INSTALL_LIBDIR}
88     RUNTIME DESTINATION ${CMAKE_INSTALL_BINDIR}
89 )
90
91 if(QT_VERSION_MAJOR EQUAL 6)
92     qt_finalize_executable(5G_channel_modeling_raytracing)
93 endif()

```

C.2 main.cpp

```
1  #include "mainwindow.h"
2
3  #include <QApplication>
4
5  int main(int argc, char *argv[])
6  {
7      //Q_INIT_RESOURCE(application);
8
9      QApplication app(argc, argv);
10     QApplication::setOrganizationName("Lucas Placentino");
11     QApplication::setApplicationName("5G channel modeling raytracing");
12     QApplication::setApplicationVersion(QT_VERSION_STR);
13
14     MainWindow mainWin;
15     mainWin.show();
16
17     return app.exec();
18 }
19
```

C.3 mainwindow

C.3.1 mainwindow.h

```
1  #ifndef MAINWINDOW_H
2  #define MAINWINDOW_H
3
4  #include <QGraphicsView>
5  #include <QMainWindow>
6  #include <QProgressBar>
7
8  QT_BEGIN_NAMESPACE
9  namespace Ui {
10     class MainWindow;
11 }
12 QT_END_NAMESPACE
13
14 class MainWindow : public QMainWindow
15 {
16     Q_OBJECT
17
18 public:
19     MainWindow(QWidget *parent = nullptr);
20     ~MainWindow();
21
22 private slots:
23     void on_runSimulationButton_clicked();
24     void on_sliderBaseStationPower_valueChanged(int value);
25     void on_actionExit_triggered();
26     void on_actionReset_triggered();
27     void on_actionSee_Github_triggered();
28     void on_actionAbout_triggered();
29     void on_actionSave_image_triggered();
30     void on_spinBoxBaseStationPower_valueChanged(int value);
31     void on_transmitterSelector_activated(int index);
32     void on_addTransmitterButton_clicked();
33     void on_deleteBaseStationPushButton_clicked();
34     void on_baseStationXspinBox_valueChanged(double x);
35     void on_baseStationYspinBox_valueChanged(double y);
36     void on_singleCellRadioButton_clicked(bool checked);

```

```

37     void on_coverageHeatmapRadioButton_clicked(bool checked);
38     void on_liftOnFloorCheckBox_clicked(bool checked);
39     void on_showCellOutlineCheckBox_toggled(bool checked);
40     void on_resolutionComboBox_currentIndexChanged(int index);
41     //void on_actionRun_Validation_Simulation_triggered();
42     void on_actionSave_Validation_image_triggered();
43     void on_cellXspinBox_valueChanged(double x);
44     void on_cellYspinBox_valueChanged(double y);
45
46     void on_actionRun_Free_Space_Validation_triggered();
47
48     void on_actionRun_Single_Reflection_Validation_triggered();
49
50     void on_actionRun_Double_Reflection_Validation_triggered();
51
52     void on_actionRun_Triple_Reflection_Validation_triggered();
53
54     void on_actionRun_Recursive_Validation_triggered();
55
56     void on_removeAllWallsCheckBox_toggled(bool checked);
57
58     void on_actionView_path_loss_plot_triggered();
59
60 private:
61     Ui::MainWindow *ui;
62     bool runSimulation(QProgressBar* progress_bar);
63     void changeBaseStationPower(qreal value);
64     void changeBaseStationCoordinates(QPointF point);
65     void initFirstBaseStation();
66     void showFirstBaseStation();
67     void showBaseStationPower(int value_dBm);
68     void showBaseStationCoordinates(QPointF point);
69
70     void toggleCellParametersLayout(bool enabled);
71     void toggleCoverageParametersLayout(bool enabled);
72     // Put the default argument here, just use nullptr for safety
73     void saveImage(QGraphicsView* view = nullptr, bool isValid = false);
74 };
75 #endif // MAINWINDOW_H

```

C.3.2 mainwindow.cpp

```

1  #include "mainwindow.h"
2  #include "../ui_mainwindow.h"
3  #include <QDesktopServices>
4  #include <QProcess>
5  #include <stdio.h>
6  #include <QMessageBox>
7  #include <QFileDialog>
8  #include <QDir>
9  #include <QChartView>
10
11 #include "parameters.h"
12
13 #include "simulation.h"
14
15 #include "validate_freespace.h"
16 #include "validate_wall.h"
17 #include "validate_corridor.h"
18 QWidget* Validate_view;
19
20 QGraphicsView* Validation_view;
21 QChartView* plot_view;

```

```

22
23 Simulation simulation = Simulation(); // The global simulation object, use `extern Simulation simulation;` in other files
24
25 int currentEditingBaseStation_index = 0; // The base station that is currently selected for user edit
26
27 QList<qreal> resolutions = {1.0, 0.5, 0.25, 0.125}; //, 0.0625};
28
29 MainWindow::MainWindow(QWidget *parent)
30     : QMainWindow(parent)
31     , ui(new Ui::MainWindow)
32 {
33     // MainWindow constructor, is ran on program's UI launch
34     this->setWindowIcon(QIcon(":/assets/icon.png"));
35     initFirstBaseStation();
36     ui->setupUi(this);
37     ui->baseStationXspinBox->setMaximum(max_x);
38     ui->baseStationYspinBox->setMaximum(max_y);
39     ui->cellXspinBox->setMaximum(max_x);
40     ui->cellYspinBox->setMaximum(max_y);
41
42     showFirstBaseStation();
43
44     toggleCoverageParametersLayout(true);
45     toggleCellParametersLayout(false);
46
47     for (qreal resolution : std::as_const(resolutions)) {
48         ui->resolutionComboBox->addItem(QString("%1 m").arg(resolution));
49     }
50     ui->resolutionComboBox->setCurrentIndex(0);
51
52     this->setWindowIcon(QIcon(":/assets/icon.png"));
53 }
54
55 MainWindow::~MainWindow()
56 {
57     // MainWindow destructor
58     delete ui;
59 }
60
61 void MainWindow::on_runSimulationButton_clicked()
62 {
63     // User clicked on the "Run Simulation" button
64
65     if (simulation.is_running) {
66         return;
67     }
68
69     qInfo("\n### Starting simulation ###");
70     QProgressBar* progress_bar = ui->simulationProgressBar;
71     progress_bar->setValue(100);
72     ui->runSimulationButton->setEnabled(false);
73     ui->runSimulationButton->setStyleSheet("font-size: 14px;");
74     ui->runSimulationButton->setText("[Menu->Reset App] or (Ctrl+R) to reset");
75     bool res = runSimulation(progress_bar);
76     progress_bar->setValue(100);
77     // Turn the text green if simulation ran successfully, red otherwise
78     ui->runSuccessOrFailText->setStyleSheet(res ? "color: green;" : "color: red;");
79     ui->runSuccessOrFailText->setText(res ? QString("Success, took %1ms").arg(simulation.getSimulationTime()): "Failed");
80
81     simulation.is_running = false;
82 }
83
84 bool MainWindow::runSimulation(QProgressBar* progress_bar)
85 {
86

```

```

87     // Run the simulation and returns true if no errors occurred
88     simulation.ran = true;
89     simulation.is_running = true;
90     try {
91         simulation.run(progress_bar);
92
93         qDebug() << "Simulation ended successfully\n";
94         return true;
95     } catch (...) {
96         qDebug() << "Simulation failed\n";
97         return false;
98     }
99 }
100
101 void MainWindow::changeBaseStationPower(qreal value)
102 {
103     // Modify the current editing base station with the new user chosen power
104     qDebug() << "dBm value:" << value;
105     //Transmitter* base_station = simulation.getBaseStation(currentEditingBaseStation_index);
106     Transmitter* base_station = simulation.baseStations[currentEditingBaseStation_index];
107
108     //qreal power = pow(10.0, value / 10.0);
109     //base_station->power=power;
110
111     base_station->setPower_dBm(value);
112     simulation.baseStations[currentEditingBaseStation_index]->power_dBm=value;
113
114     //base_station->setPower_dBm(value);
115
116     showBaseStationPower(value);
117 }
118
119 void MainWindow::on_sliderBaseStationPower_valueChanged(int value)
120 {
121     // User changed the current editing base station power (with slider)
122     changeBaseStationPower(value);
123 }
124
125 void MainWindow::on_spinBoxBaseStationPower_valueChanged(int value)
126 {
127     // User changed the current editing base station power (with spin box)
128     changeBaseStationPower(value);
129 }
130
131 void MainWindow::changeBaseStationCoordinates(QPointF point)
132 {
133     // Modify the current editing base station with the new user chosen coordinates
134     Transmitter* base_station = simulation.baseStations.at(currentEditingBaseStation_index);
135     base_station->changeCoordinates(point);
136     showBaseStationCoordinates(point);
137 }
138
139 void MainWindow::on_baseStationXspinBox_valueChanged(double x)
140 {
141     // User changed the current editing base station X coordinate
142     Transmitter* base_station = simulation.baseStations.at(currentEditingBaseStation_index);
143     QPointF coordinates = base_station->toPointF();
144     coordinates.setX(x);
145     changeBaseStationCoordinates(coordinates);
146 }
147
148 void MainWindow::on_baseStationYspinBox_valueChanged(double y)
149 {
150     // User changed the current editing base station Y coordinate
151     Transmitter* base_station = simulation.baseStations.at(currentEditingBaseStation_index);

```

```

152     QPointF coordinates = base_station->toPointF();
153     coordinates.setY(y);
154     changeBaseStationCoordinates(coordinates);
155 }
156
157 void MainWindow::on_actionExit_triggered()
158 {
159     // User clicked on the menu's "Exit" button, or presse "CTRL+W"
160     printf("Closing app...\n");
161     qApp->quit();
162 }
163
164
165 void MainWindow::on_actionReset_triggered()
166 {
167     // User clicked on the menu's "Reset" button, or presse "CTRL+R"
168     qInfo("Resetting app...");
169
170     qApp->quit();
171     QProcess::startDetached(qApp->arguments()[0], qApp->arguments());
172 }
173
174
175 void MainWindow::on_actionSee_Github_triggered()
176 {
177     // User clicked on the menu's "See Github" button
178     QDesktopServices::openUrl(QUrl("https://github.com/LucasPlacentino/5G_channel_modeling_raytracing", QUrl::TolerantMode));
179 }
180
181
182 void MainWindow::on_actionAbout_triggered()
183 {
184     // User clicked on the menu's "About" button
185     QMessageBox::about(this, tr("About this application"),
186         tr("This <b>5G Channel Modeling Ray-Tracing Simulator</b> was made as a "
187             "project for the course <b>ELEC-H415</b> Communication Channels "
188             "at <b>École polytechnique de Bruxelles - ULB (BRUFACE Master program)</b>."
189             "<br>By Lucas Placentino, 2026"));
190 }
191
192 void MainWindow::saveImage(QGraphicsView* target_view, bool isValid) // bool isValid not used
193 {
194     /// Resolve default argument
195     //if (target_view == nullptr) {
196     //    if (simulation.view == nullptr) return;
197     //    target_view = simulation.view;
198     //}
199     if (target_view == nullptr) {
200         qWarning() << "No view, cannot save image.";
201         return;
202     }
203
204     if (!target_view || !target_view->scene()) {
205         qWarning() << "No view or scene available to save!";
206         return;
207     }
208
209     // 1. Ask for file location FIRST
210     QString img_filename = QFileDialog::getSaveFileName(
211         this,
212         tr("Save Simulation Image"),
213         QDir::currentPath(),
214         "PNG (*.png);;BMP Files (*.bmp);;JPEG (*.jpg)"
215     );
216

```

```

217     if (img_filename.isEmpty()) {
218         qInfo() << "Save Cancelled";
219         return; // Abort before wasting CPU
220     }
221
222     // 2. Setup Image (Format_ARGB32_Premultiplied is much faster for Qt drawing than RGBA64)
223     QGraphicsScene* scene = target_view->scene();
224     QSize size = scene->sceneRect().size().toSize() * 10;
225     QImage img(size, QImage::Format_ARGB32_Premultiplied);
226
227     // 3. Clear the garbage memory!
228     img.fill(Qt::black); // Use black since your UI uses a black background
229
230     // 4. Paint the SCENE, not the view
231     QPainter painter(&img);
232     painter.setRenderHint(QPainter::Antialiasing);
233
234     // Pass the target rect (the image) and source rect (the scene) to ensure perfect scaling
235     scene->render(&painter, QRectF(img.rect()), scene->sceneRect());
236     painter.end(); // Always end the painter before saving
237
238     // 5. Save
239     bool success = img.save(img_filename);
240     if (success) {
241         qInfo() << "Image successfully saved to:" << img_filename;
242     } else {
243         qWarning() << "Failed to save image!";
244     }
245 }
246
247 void MainWindow::on_actionSave_image_triggered()
248 {
249     // User clicked on the menu's "Save Image" button, captures an image of the simulation view.
250
251     if (simulation.scene == nullptr || !simulation.ran) { // checks if a simulation has run
252         qInfo("Cannot save simulation that has not already run.");
253         //throw std::runtime_error("Cannot save simulation that has not already run.");
254         return;
255     }
256     saveImage(simulation.view);
257 }
258
259
260 void MainWindow::initFirstBaseStation()
261 {
262     // Creates the first (non-deletable) Base Station
263     simulation.baseStations.append(new Transmitter(initial_BS_x, initial_BS_y, 0, "Base Station 1"));
264 }
265
266 void MainWindow::showFirstBaseStation()
267 {
268     // Makes sure the first (non-deletable) Base Station is shown on the base station user edit box
269     QString new_item = QString("Base Station 1");
270     ui->transmitterSelector->addItem(new_item);
271     ui->transmitterSelector->setCurrentIndex(0);
272     on_transmitterSelector_activated(0);
273 }
274
275 void MainWindow::showBaseStationPower(int value_dBm)
276 {
277     // Updates the UI to show this base station power
278     ui->spinBoxBaseStationPower->setValue(value_dBm);
279     ui->sliderBaseStationPower->setValue(value_dBm);
280 }
281

```

```

282 void MainWindow::showBaseStationCoordinates(QPointF point)
283 {
284     // Updates the UI to show these base station coordinates
285     ui->baseStationXspinBox->setValue(point.x());
286     ui->baseStationYspinBox->setValue(point.y());
287 }
288
289 void MainWindow::toggleCellParametersLayout(bool enabled)
290 {
291     //ui->cellParametersLayout->setEnabled(enabled);
292     ui->cellCoordinatesLabel->setEnabled(enabled);
293     ui->cellCoordinatesXLabel->setEnabled(enabled);
294     ui->cellCoordinatesYLabel->setEnabled(enabled);
295     ui->cellXspinBox->setEnabled(enabled);
296     ui->cellYspinBox->setEnabled(enabled);
297 }
298
299 void MainWindow::toggleCoverageParametersLayout(bool enabled)
300 {
301     ui->showCellOutlineCheckBox->setEnabled(enabled);
302 }
303
304 void MainWindow::on_transmitterSelector_activated(int index)
305 {
306     // User selected a base station in the dropdown
307     qDebug() << "Selected base station: " << index+1;
308
309     // set current editing transmitter to this one
310     currentEditingBaseStation_index = index;
311     showBaseStationPower(simulation.getBaseStation(currentEditingBaseStation_index)->getPower_dBm());
312
313     showBaseStationCoordinates(simulation.baseStations.at(currentEditingBaseStation_index)->toPointF());
314 }
315
316
317 void MainWindow::on_addTransmitterButton_clicked()
318 {
319     // User clicked "Add Base Station" button
320     if (ui->transmitterSelector->count() < ui->transmitterSelector->maxCount()) //check if has not reached the max number
321     {
322         qDebug("Added base station");
323
324         QString new_item_name = QString("Base Station %1").arg(ui->transmitterSelector->count()+1);
325         ui->transmitterSelector->addItem(new_item_name);
326
327         int new_item_index = ui->transmitterSelector->findText(new_item_name);
328
329         simulation.baseStations.append(new Transmitter(initial_BS_x,initial_BS_y, new_item_index, new_item_name));
330
331         on_transmitterSelector_activated(new_item_index);
332         ui->transmitterSelector->setCurrentIndex(new_item_index);
333
334     } else {
335         qDebug("There is already the maximum number of base stations");
336     }
337 }
338
339
340
341 void MainWindow::on_deleteBaseStationPushButton_clicked()
342 {
343     // User clicked on the "Delete Base Station" button
344     //currentEditingBaseStation_index
345     if (ui->transmitterSelector->count()>1 && currentEditingBaseStation_index != 0)
346     {

```

```

347         ui->transmitterSelector->removeItem(currentEditingBaseStation_index);
348         simulation.deleteBaseStation(currentEditingBaseStation_index);
349
350         int previous_item_index = currentEditingBaseStation_index-1;
351         on_transmitterSelector_activated(previous_item_index);
352         ui->transmitterSelector->setCurrentIndex(previous_item_index);
353     } else {
354         qDebug("Cannot delete Base Station 1");
355     }
356 }
357
358 void MainWindow::on_coverageHeatmapRadioButton_clicked(bool checked)
359 {
360     simulation.showRaySingleCell = !checked;
361     toggleCoverageParametersLayout(checked);
362     toggleCellParametersLayout(!checked);
363     qDebug() << "Coverage parameters" << (checked? "enabled," : "disabled,") << "Cell parameters" << (!checked? "enabled," : "disabled,");
364 }
365
366 void MainWindow::on_singleCellRadioButton_clicked(bool checked)
367 {
368     simulation.showRaySingleCell = checked;
369     toggleCellParametersLayout(checked);
370     toggleCoverageParametersLayout(!checked);
371     qDebug() << "Coverage parameters" << (!checked? "enabled," : "disabled,") << "Cell parameters" << (checked? "enabled," : "disabled,");
372 }
373
374 void MainWindow::on_liftOnFloorCheckBox_clicked(bool checked)
375 {
376     qDebug() << "Set lift:" << checked;
377     simulation.lift_is_on_floor = checked;
378 }
379
380 void MainWindow::on_showCellOutlineCheckBox_toggled(bool checked)
381 {
382     simulation.show_cell_outline = checked;
383     qDebug() << "Show cell outline:" << checked;
384 }
385
386 void MainWindow::on_resolutionComboBox_currentIndexChanged(int index)
387 {
388     simulation.resolution = resolutions[index];
389     qDebug() << "Changed resolution to" << resolutions[index] << "m";
390 }
391
392 // void MainWindow::on_actionRun_Validation_Simulation_triggered()
393 // {
394 //     Validation_view = runValidation();
395 // }
396
397 void MainWindow::on_actionSave_Validation_image_triggered()
398 {
399     saveImage(Validation_view);
400 }
401
402 void MainWindow::on_cellXspinBox_valueChanged(double x)
403 {
404     simulation.singleCellX = x;
405 }
406
407
408
409
410
411

```

```

412
413 void MainWindow::on_cellYspinBox_valueChanged(double y)
414 {
415     simulation.singleCellY = y;
416 }
417
418 void MainWindow::on_actionRun_Free_Space_Validation_triggered()
419 {
420     //Validation_view = runFreeSpaceValidation();
421     qDebug() << "Running Free Space Validation...";
422     Validate_view = createFreeSpaceValidationPlot();
423     Validate_view->show();
424     //runFreeSpaceValidation();
425 }
426
427
428 void MainWindow::on_actionRun_Single_Reflection_Validation_triggered()
429 {
430     //Validation_view = runSingleReflectionValidation();
431     qDebug() << "Running Single Reflection Validation...";
432     Validate_view = createTwoRayValidationPlot();
433     Validate_view->show();
434     //runSingleReflectionValidation();
435 }
436
437
438 void MainWindow::on_actionRun_Double_Reflection_Validation_triggered()
439 {
440     qDebug() << "Running Double Reflection Validation...";
441     //Validation_view = runDoubleReflectionValidation();
442 }
443
444
445 void MainWindow::on_actionRun_Triple_Reflection_Validation_triggered()
446 {
447     //Validation_view = runTripleReflectionValidation();
448     qDebug() << "Running Triple Reflection (Corridor) Validation...";
449     Validate_view = createThreeReflectionValidationPlot();
450     Validate_view->show();
451 }
452
453
454 void MainWindow::on_actionRun_Recursive_Validation_triggered()
455 {
456     qDebug() << "not implemented";
457     //Validation_view = runRecursiveValidation();
458 }
459
460
461 void MainWindow::on_removeAllWallsCheckBox_toggled(bool checked)
462 {
463     simulation.remove_all_walls = checked;
464     qDebug() << "Remove all walls:" << checked;
465 }
466
467
468 void MainWindow::on_actionView_path_loss_plot_triggered()
469 {
470     if (simulation.ran && simulation.baseStations.length() == 1) { // sim ran and only 1 BS
471         qDebug() << "Showing path loss plot";
472         plot_view = simulation.showPathLossScatterPlot();
473     } else {
474         qDebug() << "Simulation hasn't run yet! Cannot show path loss plot.";
475     }
476 }

```

C.4 mainwindow.ui

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <ui version="4.0">
3      <class>MainWindow</class>
4      <widget class="QMainWindow" name="MainWindow">
5          <property name="geometry">
6              <rect>
7                  <x>0</x>
8                  <y>0</y>
9                  <width>340</width>
10                 <height>604</height>
11             </rect>
12         </property>
13         <property name="sizePolicy">
14             <sizepolicy hsize="Preferred" vsize="Preferred">
15                 <horstretch>0</horstretch>
16                 <verstretch>0</verstretch>
17             </sizepolicy>
18         </property>
19         <property name="minimumSize">
20             <size>
21                 <width>340</width>
22                 <height>604</height>
23             </size>
24         </property>
25         <property name="maximumSize">
26             <size>
27                 <width>340</width>
28                 <height>604</height>
29             </size>
30         </property>
31         <property name="font">
32             <font>
33                 <family>Arial</family>
34             </font>
35         </property>
36         <property name="windowTitle">
37             <string>5G channel modeling raytracing</string>
38         </property>
39         <property name="windowIcon">
40             <iconset>
41                 <normaloff>assets/icon.png</normaloff>assets/icon.png</iconset>
42             </property>
43         <property name="autoFillBackground">
44             <bool>false</bool>
45         </property>
46         <property name="locale">
47             <locale language="English" country="Belgium"/>
48         </property>
49         <widget class="QWidget" name="centralwidget">
50             <widget class="QGroupBox" name="groupBox">
51                 <property name="geometry">
52                     <rect>
53                         <x>10</x>
54                         <y>0</y>
55                         <width>320</width>
56                         <height>281</height>
57                     </rect>
58                 </property>
59                 <property name="title">

```

```

60     <string>Simulation Settings</string>
61 </property>
62 <property name="alignment">
63     <set>Qt::AlignmentFlag::AlignLeading|Qt::AlignmentFlag::AlignLeft|Qt::AlignmentFlag::AlignVCenter</set>
64 </property>
65 <property name="flat">
66     <bool>false</bool>
67 </property>
68 <property name="checkable">
69     <bool>false</bool>
70 </property>
71 <widget class="QWidget" name="layoutWidget">
72     <property name="geometry">
73         <rect>
74             <x>10</x>
75             <y>20</y>
76             <width>301</width>
77             <height>258</height>
78         </rect>
79     </property>
80     <layout class="QVBoxLayout" name="verticalLayout_2">
81         <property name="spacing">
82             <number>0</number>
83         </property>
84         <property name="leftMargin">
85             <number>4</number>
86         </property>
87         <property name="topMargin">
88             <number>0</number>
89         </property>
90         <property name="rightMargin">
91             <number>5</number>
92         </property>
93         <property name="bottomMargin">
94             <number>0</number>
95         </property>
96         <item>
97             <layout class="QVBoxLayout" name="runSimulationLayout">
98                 <property name="spacing">
99                     <number>0</number>
100                 </property>
101                 <property name="sizeConstraint">
102                     <enum>QLayout::SizeConstraint::SetFixedSize</enum>
103                 </property>
104                 <item>
105                     <widget class="QLabel" name="runSuccessOrFailText">
106                         <property name="maximumSize">
107                             <size>
108                                 <width>16777215</width>
109                                 <height>30</height>
110                             </size>
111                         </property>
112                         <property name="frameShape">
113                             <enum>QFrame::Shape::Box</enum>
114                         </property>
115                         <property name="lineWidth">
116                             <number>1</number>
117                         </property>
118                         <property name="text">
119                             <string/>
120                         </property>
121                         <property name="alignment">
122                             <set>Qt::AlignmentFlag::AlignCenter</set>
123                         </property>
124                         <property name="margin">

```

```

125         <number>4</number>
126     </property>
127 </widget>
128 </item>
129 <item>
130     <widget class="QPushButton" name="runSimulationButton">
131         <property name="maximumSize">
132             <size>
133                 <width>16777215</width>
134                 <height>30</height>
135             </size>
136         </property>
137         <property name="font">
138             <font>
139                 <family>Arial</family>
140                 <pointsize>11</pointsize>
141             </font>
142         </property>
143         <property name="text">
144             <string>Run simulation</string>
145         </property>
146         <property name="shortcut">
147             <string>Ctrl+E</string>
148         </property>
149     </widget>
150 </item>
151 <item>
152     <widget class="QProgressBar" name="simulationProgressBar">
153         <property name="minimumSize">
154             <size>
155                 <width>20</width>
156                 <height>20</height>
157             </size>
158         </property>
159         <property name="baseSize">
160             <size>
161                 <width>20</width>
162                 <height>20</height>
163             </size>
164         </property>
165         <property name="value">
166             <number>0</number>
167         </property>
168         <property name="textVisible">
169             <bool>false</bool>
170         </property>
171     </widget>
172 </item>
173 </layout>
174 </item>
175 <item>
176     <layout class="QVBoxLayout" name="simulationParametersLayout">
177         <property name="spacing">
178             <number>4</number>
179         </property>
180     <item>
181         <widget class="Line" name="line">
182             <property name="frameShadow">
183                 <enum>QFrame::Shadow::Plain</enum>
184             </property>
185             <property name="lineWidth">
186                 <number>1</number>
187             </property>
188             <property name="orientation">
189                 <enum>Qt::Orientation::Horizontal</enum>

```

```

190     </property>
191 </widget>
192 </item>
193 <item>
194     <layout class="QHBoxLayout" name="horizontalLayout_10">
195         <property name="spacing">
196             <number>2</number>
197         </property>
198     <item>
199         <layout class="QVBoxLayout" name="verticalLayout_11">
200             <property name="spacing">
201                 <number>4</number>
202             </property>
203             <item>
204                 <widget class="QRadioButton" name="coverageHeatmapRadioButton">
205                     <property name="text">
206                         <string>Coverage Heatmap</string>
207                     </property>
208                     <property name="checked">
209                         <bool>true</bool>
210                     </property>
211                 </widget>
212             </item>
213             <item>
214                 <layout class="QVBoxLayout" name="verticalLayout">
215                     <item>
216                         <widget class="QCheckBox" name="showCellOutlineCheckBox">
217                             <property name="text">
218                                 <string>Show cells outline</string>
219                             </property>
220                         </widget>
221                     </item>
222                     <item>
223                         <layout class="QVBoxLayout" name="verticalLayout_9">
224                             <item>
225                                 <widget class="QLabel" name="resolutionLabel">
226                                     <property name="text">
227                                         <string>Resolution</string>
228                                     </property>
229                                 </widget>
230                             </item>
231                             <item>
232                                 <widget class="QComboBox" name="resolutionComboBox">
233                                     <property name="enabled">
234                                         <bool>true</bool>
235                                     </property>
236                                 </widget>
237                             </item>
238                             <item>
239                                 <widget class="QCheckBox" name="removeAllWallsCheckBox">
240                                     <property name="text">
241                                         <string>Remove all walls</string>
242                                     </property>
243                                 </widget>
244                             </item>
245                         </layout>
246                     </item>
247                     <item>
248                         <spacer name="verticalSpacer">
249                             <property name="orientation">
250                                 <enum>Qt::Orientation::Vertical</enum>
251                             </property>
252                             <property name="sizeType">
253                                 <enum>QSizePolicy::Policy::Minimum</enum>
254                             </property>

```

```

255         <property name="sizeHint" stdset="0">
256             <size>
257                 <width>0</width>
258                 <height>0</height>
259             </size>
260         </property>
261     </spacer>
262 </item>
263 </layout>
264 </item>
265 </layout>
266 </item>
267 <item>
268     <widget class="Line" name="line_2">
269         <property name="frameShadow">
270             <enum>QFrame::Shadow::Plain</enum>
271         </property>
272         <property name="orientation">
273             <enum>Qt::Orientation::Vertical</enum>
274         </property>
275     </widget>
276 </item>
277 <item>
278     <layout class="QVBoxLayout" name="verticalLayout_10">
279         <property name="spacing">
280             <number>6</number>
281         </property>
282         <item>
283             <widget class="QRadioButton" name="singleCellRadioButton">
284                 <property name="enabled">
285                     <bool>true</bool>
286                 </property>
287                 <property name="maximumSize">
288                     <size>
289                         <width>16777215</width>
290                         <height>16</height>
291                     </size>
292                 </property>
293                 <property name="text">
294                     <string>Rays to Single Cell</string>
295                 </property>
296             </widget>
297         </item>
298         <item>
299             <layout class="QVBoxLayout" name="cellParametersLayout">
300                 <property name="spacing">
301                     <number>0</number>
302                 </property>
303                 <item>
304                     <widget class="QLabel" name="cellCoordinatesLabel">
305                         <property name="maximumSize">
306                             <size>
307                                 <width>16777215</width>
308                                 <height>16</height>
309                             </size>
310                         </property>
311                         <property name="text">
312                             <string>Cell Coordinates</string>
313                         </property>
314                     </widget>
315                 </item>
316                 <item>
317                     <layout class="QHBoxLayout" name="horizontalLayout_9">
318                         <property name="spacing">
319                             <number>4</number>

```

```

320     </property>
321 </item>
322 <layout class="QVBoxLayout" name="verticalLayout_7">
323     <property name="spacing">
324         <number>0</number>
325     </property>
326     <item>
327         <widget class="QLabel" name="cellCoordinatesXLabel">
328             <property name="maximumSize">
329                 <size>
330                     <width>16777215</width>
331                     <height>14</height>
332                 </size>
333             </property>
334             <property name="text">
335                 <string>X</string>
336             </property>
337         </widget>
338     </item>
339     <item>
340         <widget class="QDoubleSpinBox" name="cellXspinBox">
341             <property name="maximumSize">
342                 <size>
343                     <width>16777215</width>
344                     <height>20</height>
345                 </size>
346             </property>
347             <property name="maximum">
348                 <double>15.000000000000000</double>
349             </property>
350             <property name="singleStep">
351                 <double>0.500000000000000</double>
352             </property>
353             <property name="value">
354                 <double>2.000000000000000</double>
355             </property>
356         </widget>
357     </item>
358 </layout>
359 </item>
360 <item>
361 <layout class="QVBoxLayout" name="verticalLayout_8">
362     <property name="spacing">
363         <number>0</number>
364     </property>
365     <item>
366         <widget class="QLabel" name="cellCoordinatesYLabel">
367             <property name="maximumSize">
368                 <size>
369                     <width>16777215</width>
370                     <height>14</height>
371                 </size>
372             </property>
373             <property name="text">
374                 <string>Y</string>
375             </property>
376         </widget>
377     </item>
378     <item>
379         <widget class="QDoubleSpinBox" name="cellYspinBox">
380             <property name="maximumSize">
381                 <size>
382                     <width>16777215</width>
383                     <height>20</height>
384                 </size>

```

```

385         </property>
386         <property name="minimum">
387             <double>0.000000000000000</double>
388         </property>
389         <property name="maximum">
390             <double>8.000000000000000</double>
391         </property>
392         <property name="singleStep">
393             <double>0.500000000000000</double>
394         </property>
395         <property name="value">
396             <double>2.000000000000000</double>
397         </property>
398     </widget>
399 </item>
400 </layout>
401 </item>
402 </layout>
403 </item>
404 </layout>
405 </item>
406 </layout>
407 </item>
408 </layout>
409 </item>
410 </layout>
411 </item>
412 </layout>
413 </widget>
414 </widget>
415 <widget class="QGroupBox" name="groupBox_2">
416     <property name="geometry">
417         <rect>
418             <x>10</x>
419             <y>290</y>
420             <width>320</width>
421             <height>260</height>
422         </rect>
423     </property>
424     <property name="title">
425         <string>Base Station Settings</string>
426     </property>
427     <widget class="QWidget" name="verticalLayoutWidget">
428         <property name="geometry">
429             <rect>
430                 <x>10</x>
431                 <y>20</y>
432                 <width>301</width>
433                 <height>231</height>
434             </rect>
435         </property>
436         <layout class="QVBoxLayout" name="verticalLayout_3">
437             <property name="spacing">
438                 <number>4</number>
439             </property>
440             <property name="leftMargin">
441                 <number>4</number>
442             </property>
443             <property name="topMargin">
444                 <number>0</number>
445             </property>
446             <property name="rightMargin">
447                 <number>5</number>
448             </property>
449             <property name="bottomMargin">

```

```

450     <number>0</number>
451 </property>
452 </item>
453 <layout class="QHBoxLayout" name="horizontalLayout_3">
454     <property name="spacing">
455         <number>2</number>
456     </property>
457     <property name="leftMargin">
458         <number>0</number>
459     </property>
460     <item>
461         <widget class="QPushButton" name="addTransmitterButton">
462             <property name="enabled">
463                 <bool>true</bool>
464             </property>
465             <property name="maximumSize">
466                 <size>
467                     <width>20</width>
468                     <height>20</height>
469                 </size>
470             </property>
471             <property name="text">
472                 <string>+</string>
473             </property>
474         </widget>
475     </item>
476     <item>
477         <widget class="QComboBox" name="transmitterSelector">
478             <property name="enabled">
479                 <bool>true</bool>
480             </property>
481             <property name="maximumSize">
482                 <size>
483                     <width>16777215</width>
484                     <height>20</height>
485                 </size>
486             </property>
487             <property name="currentText">
488                 <string/>
489             </property>
490             <property name="currentIndex">
491                 <number>-1</number>
492             </property>
493             <property name="maxCount">
494                 <number>4</number>
495             </property>
496         </widget>
497     </item>
498 </layout>
499 </item>
500 <item>
501     <layout class="QVBoxLayout" name="verticalLayout_6">
502         <item>
503             <layout class="QHBoxLayout" name="horizontalLayout_7">
504                 <item>
505                     <widget class="QLabel" name="baseStationSliderLabel">
506                         <property name="maximumSize">
507                             <size>
508                                 <width>16777215</width>
509                                 <height>20</height>
510                             </size>
511                         </property>
512                         <property name="text">
513                             <string>Base Station Transmitting Power</string>
514                         </property>

```

```

515         </widget>
516     </item>
517 </item>
518     <widget class="QLabel" name="unitDBmLabel">
519         <property name="maximumSize">
520             <size>
521                 <width>16777215</width>
522                 <height>20</height>
523             </size>
524         </property>
525         <property name="text">
526             <string>[dBm]</string>
527         </property>
528         <property name="alignment">
529             <set>Qt::AlignmentFlag::AlignRight|Qt::AlignmentFlag::AlignTrailing|Qt::AlignmentFlag::AlignVCenter</set>
530         </property>
531     </widget>
532 </item>
533 </layout>
534 </item>
535 <item>
536     <layout class="QHBoxLayout" name="horizontalLayout_2">
537         <item>
538             <widget class="QSlider" name="sliderBaseStationPower">
539                 <property name="enabled">
540                     <bool>true</bool>
541                 </property>
542                 <property name="maximumSize">
543                     <size>
544                         <width>16777215</width>
545                         <height>20</height>
546                     </size>
547                 </property>
548                 <property name="maximum">
549                     <number>30</number>
550                 </property>
551                 <property name="value">
552                     <number>20</number>
553                 </property>
554                 <property name="orientation">
555                     <enum>Qt::Orientation::Horizontal</enum>
556                 </property>
557                 <property name="tickPosition">
558                     <enum>QSlider::TickPosition::TicksAbove</enum>
559                 </property>
560                 <property name="tickInterval">
561                     <number>2</number>
562                 </property>
563             </widget>
564         </item>
565         <item>
566             <widget class="QSpinBox" name="spinBoxBaseStationPower">
567                 <property name="enabled">
568                     <bool>true</bool>
569                 </property>
570                 <property name="maximumSize">
571                     <size>
572                         <width>16777215</width>
573                         <height>20</height>
574                     </size>
575                 </property>
576                 <property name="suffix">
577                     <string/>
578                 </property>
579                 <property name="maximum">

```

```

580         <number>30</number>
581     </property>
582     <property name="value">
583         <number>20</number>
584     </property>
585 </widget>
586 </item>
587 </layout>
588 </item>
589 </layout>
590 </item>
591 <item>
592     <layout class="QHBoxLayout" name="horizontalLayout_6">
593         <item>
594             <layout class="QHBoxLayout" name="horizontalLayout_8">
595                 <item>
596                     <layout class="QVBoxLayout" name="verticalLayout_4">
597                         <item>
598                             <widget class="QLabel" name="baseStationXlabel">
599                                 <property name="maximumSize">
600                                     <size>
601                                         <width>16777215</width>
602                                         <height>12</height>
603                                     </size>
604                                 </property>
605                                 <property name="text">
606                                     <string>X Coordinate</string>
607                                 </property>
608                             </widget>
609                         </item>
610                         <item>
611                             <widget class="QDoubleSpinBox" name="baseStationXspinBox">
612                                 <property name="maximumSize">
613                                     <size>
614                                         <width>16777215</width>
615                                         <height>20</height>
616                                     </size>
617                                 </property>
618                                 <property name="decimals">
619                                     <number>1</number>
620                                 </property>
621                                 <property name="maximum">
622                                     <double>15.000000000000000</double>
623                                 </property>
624                                 <property name="singleStep">
625                                     <double>0.5000000000000000</double>
626                                 </property>
627                                 <property name="value">
628                                     <double>9.400000000000000</double>
629                                 </property>
630                             </widget>
631                         </item>
632                     </layout>
633                 </item>
634             <item>
635                 <layout class="QVBoxLayout" name="verticalLayout_5">
636                     <item>
637                         <widget class="QLabel" name="baseStationYlabel">
638                             <property name="maximumSize">
639                                 <size>
640                                     <width>16777215</width>
641                                     <height>12</height>
642                                 </size>
643                             </property>
644                             <property name="text">

```

```

645         <string>Y Coordinate</string>
646     </property>
647 </widget>
648 </item>
649 <item>
650     <widget class="QDoubleSpinBox" name="baseStationYspinBox">
651         <property name="maximumSize">
652             <size>
653                 <width>16777215</width>
654                 <height>20</height>
655             </size>
656         </property>
657         <property name="decimals">
658             <number>1</number>
659         </property>
660         <property name="minimum">
661             <double>0.000000000000000</double>
662         </property>
663         <property name="maximum">
664             <double>9.000000000000000</double>
665         </property>
666         <property name="singleStep">
667             <double>0.500000000000000</double>
668         </property>
669         <property name="value">
670             <double>0.000000000000000</double>
671         </property>
672     </widget>
673 </item>
674 </layout>
675 </item>
676 </layout>
677 </item>
678 </layout>
679 </item>
680 <item>
681     <layout class="QHBoxLayout" name="horizontalLayout_5">
682         <item>
683             <spacer name="horizontalSpacer">
684                 <property name="orientation">
685                     <enum>Qt::Orientation::Horizontal</enum>
686                 </property>
687                 <property name="sizeHint" stdset="0">
688                     <size>
689                         <width>40</width>
690                         <height>20</height>
691                     </size>
692                 </property>
693             </spacer>
694         </item>
695         <item>
696             <widget class="QPushButton" name="deleteBaseStationPushButton">
697                 <property name="enabled">
698                     <bool>true</bool>
699                 </property>
700                 <property name="maximumSize">
701                     <size>
702                         <width>16777215</width>
703                         <height>20</height>
704                     </size>
705                 </property>
706                 <property name="text">
707                     <string>Delete BS</string>
708                 </property>
709             </widget>

```

```

710         </item>
711     </layout>
712 </item>
713 </layout>
714 </widget>
715 </widget>
716 </widget>
717 <widget class="QMenuBar" name="menubar">
718     <property name="geometry">
719         <rect>
720             <x>0</x>
721             <y>0</y>
722             <width>340</width>
723             <height>18</height>
724         </rect>
725     </property>
726     <widget class="QMenu" name="menuTest">
727         <property name="title">
728             <string>Menu</string>
729         </property>
730         <addaction name="separator"/>
731         <addaction name="actionSave_image"/>
732         <addaction name="actionView_path_loss_plot"/>
733         <addaction name="separator"/>
734         <addaction name="actionExit"/>
735         <addaction name="actionReset"/>
736     </widget>
737     <widget class="QMenu" name="menuInfo">
738         <property name="title">
739             <string>Info</string>
740         </property>
741         <addaction name="actionAbout"/>
742         <addaction name="separator"/>
743         <addaction name="actionSee_Github"/>
744     </widget>
745     <widget class="QMenu" name="menuValidation_Simulation">
746         <property name="title">
747             <string>Validations</string>
748         </property>
749         <addaction name="actionRun_Free_Space_Validation"/>
750         <addaction name="actionRun_Single_Reflection_Validation"/>
751         <addaction name="actionRun_Double_Reflection_Validation"/>
752         <addaction name="actionRun_Triple_Reflection_Validation"/>
753         <addaction name="actionRun_Recursive_Validation"/>
754         <addaction name="actionSave_Validation_image"/>
755     </widget>
756     <addaction name="menuTest"/>
757     <addaction name="menuInfo"/>
758     <addaction name="menuValidation_Simulation"/>
759 </widget>
760 <widget class="QStatusBar" name="statusbar"/>
761 <action name="actionExit">
762     <property name="text">
763         <string>Exit</string>
764     </property>
765     <property name="shortcut">
766         <string>Ctrl+W</string>
767     </property>
768 </action>
769 <action name="actionReset">
770     <property name="text">
771         <string>Restart App</string>
772     </property>
773     <property name="shortcut">
774         <string>Ctrl+R</string>

```

```

775     </property>
776 </action>
777 <action name="actionMade_by_Lucas_Placentino">
778   <property name="text">
779     <string>Made by Lucas Placentino</string>
780   </property>
781 </action>
782 <action name="actionSee_Github">
783   <property name="text">
784     <string>See Github</string>
785   </property>
786 </action>
787 <action name="actionand_Salman_Houdaibi">
788   <property name="text">
789     <string>and Salman Houdaibi</string>
790   </property>
791 </action>
792 <action name="actionAbout">
793   <property name="text">
794     <string>About</string>
795   </property>
796   <property name="shortcut">
797     <string>Ctrl+A</string>
798   </property>
799 </action>
800 <action name="actionSave_image">
801   <property name="text">
802     <string>Save Simulation Image</string>
803   </property>
804   <property name="shortcut">
805     <string>Ctrl+S</string>
806   </property>
807 </action>
808 <action name="actionRun_Free_Space_Validation">
809   <property name="text">
810     <string>Run Free Space Validation</string>
811   </property>
812   <property name="shortcut">
813     <string>Ctrl+T</string>
814   </property>
815 </action>
816 <action name="actionSave_Validation_image">
817   <property name="enabled">
818     <bool>false</bool>
819   </property>
820   <property name="text">
821     <string>Save Validation image</string>
822   </property>
823 </action>
824 <action name="actionRun_Single_Reflection_Validation">
825   <property name="text">
826     <string>Run Single Reflection Validation</string>
827   </property>
828   <property name="shortcut">
829     <string>Ctrl+Y</string>
830   </property>
831 </action>
832 <action name="actionRun_Double_Reflection_Validation">
833   <property name="enabled">
834     <bool>false</bool>
835   </property>
836   <property name="text">
837     <string>Run Double Reflection Validation</string>
838   </property>
839   <property name="shortcut">

```

```

840     <string>Ctrl+U</string>
841 </property>
842 </action>
843 <action name="actionRun_Triple_Reflection_Validation">
844     <property name="text">
845         <string>Run Corridor Validation</string>
846     </property>
847     <property name="shortcut">
848         <string>Ctrl+I</string>
849     </property>
850 </action>
851 <action name="actionRun_Recursive_Validation">
852     <property name="enabled">
853         <bool>false</bool>
854     </property>
855     <property name="text">
856         <string>Run Recursive Validation</string>
857     </property>
858     <property name="shortcut">
859         <string>Ctrl+O</string>
860     </property>
861 </action>
862 <action name="actionView_path_loss_plot">
863     <property name="text">
864         <string>View Path Loss Plot</string>
865     </property>
866     <property name="shortcut">
867         <string>Ctrl+P</string>
868     </property>
869 </action>
870 </widget>
871 <resources/>
872 <connections/>
873 </ui>

```

C.5 parameters.h

```

1  #ifndef PARAMETERS_H
2  #define PARAMETERS_H
3
4  #include <complex>
5  #include <QtTypes>
6  #include <qmath.h>
7
8  using namespace std;
9
10 static constexpr complex<qreal> j(0, 1); // useful
11 //static constexpr qulonglong c = 3*1e8;
12 static constexpr qulonglong c = 299792458; // is more precise needed ?
13
14 static constexpr qreal epsilon_0 = 8.854187817e-12;
15 static constexpr qreal mu_0 = 1.256637e-6; // 4 * M_PI * 1e-7; // 1.256637e-6; // ?
16
17 // 5G
18 static constexpr qreal freq = 26e9; // 26 GHz (f)
19 static constexpr qreal f = freq;
20 constexpr qreal bandwidth = 200e6; // 200MHz (B) // TODO: impl
21 constexpr qreal B = bandwidth;
22 static constexpr qreal omega = 2 * M_PI * freq; // pulse
23 static constexpr qreal wavelength = c/freq;
24 static constexpr qreal lambda = wavelength;
25 static const qreal Z_0 = sqrt(mu_0 / epsilon_0); // vacuum impedance
26

```

```

27 static constexpr qreal epsilon_r = 4; // relative permittivity of ALL materials
28 static constexpr qreal sigma_cond = 0.0; // lossless material (to use dielectric slab model for walls)
29 //static constexpr qreal sigma_cond = 1;
30
31 /*
32 Sensitivity / Bit rate
33 -80 dBm / 100 Mb/s (lower: comms impossible)
34 -60 dBm / 1 Gb/s (higher: data rate capped)
35 */
36 static constexpr qreal min_sensitivity_dBm = -80; // minimum sensitivity in dBm (communication impossible)
37 static constexpr qreal max_sensitivity_dBm = -60; // maximum sensitivity in dBm (max capped data rate)
38 static constexpr qreal min_bitrate_bps = 100e6; // 100 Mbps
39 static constexpr qreal max_bitrate_bps = 1e9; // 1 Gbps
40 static constexpr qreal max_bitrate_Mbps = 1000; // 1 Gbps
41 static constexpr qreal min_bitrate_Mbps = 100; // 100 Mbps
42 // assume that the UE sensitivity (in dBm) varies linearly with the maximum achievable bit rate when both quantities are e
43
44 // far-field, plane waves, check:
45 static constexpr qreal D = 0.1; // max physical dimension of antenna e.g. ~10cm
46 static const qreal far_field_min_distance = qMax(qMax(1.6*lambda,5*D),2*qPow(D,2)/lambda); // r_{ff} border of exclusion
47
48 // TODO: change
49 static constexpr qreal max_x = 120;
50 static constexpr qreal min_x = 0;
51 static constexpr qreal min_y = 0;
52 static constexpr qreal max_y = 70;
53 // resolution is set in simulation->resolution, user chosen at runtime
54
55 static constexpr qreal beta_0 = 2*M_PI*freq/c; // beta
56
57 // TX and RX antennas are lossless lambda/2 dipoles
58 static constexpr qreal G_TX = 1.64; // transmitter antenna gain, 1.64 or 1.7
59 static constexpr qreal G_RX = 1.64; // same as TX
60 constexpr qreal P_TX = 0.1; // transmitter power in Watts
61 constexpr qreal P_TX_dBm = 20; // transmitter power in dBm
62 static const qreal effective_height = lambda/M_PI; // h_{e\perp} (because halfwave dipole in horizontal plane)
63 static constexpr qreal R_a = 73; // antenna resistance : 73 Ohm
64
65 static constexpr qreal initial_BS_x = 48.7;
66 static constexpr qreal initial_BS_y = 27.7;
67
68 #endif //PARAMETERS_H

```

C.6 obstacle

C.6.1 obstacle.h

```

1 #ifndef OBSTACLE_H
2 #define OBSTACLE_H
3
4 #include <QGraphicsRectItem>
5 #include <complex>
6 #include <QVector2D>
7
8 using namespace std;
9
10 struct WallProperties {
11     qreal relative_permittivity; // epsilon_r
12     qreal conductivity; // sigma
13     qreal epsilon; // permittivity = epsilon_0 * epsilon_r
14     complex<qreal> Z_m; // impedance
15     qreal alpha_m;
16     qreal beta_m;

```

```

17     complex<qreal> gamma_m; // gamma_m = alpha_m + j*beta_m
18 };
19
20 enum ObstacleType {
21     GenericWall,
22     BrickWall,
23     Window,
24     MetalWall,
25     DryWall,
26     ConcreteWall
27 };
28
29 struct Wall {
30     ObstacleType material;
31     WallProperties properties;
32 };
33
34 class Obstacle: public Wall
35 {
36 public:
37     Obstacle(
38         QVector2D start,
39         QVector2D end,
40         ObstacleType material,
41         qreal thickness//,
42         //int id
43     );
44     QGraphicsLineItem* graphics = new QGraphicsLineItem(); // wall's QGraphicsItem
45     QLineF line; // wall's QLineF, for intersects()
46     QVector2D normal; // wall's normal vector ! normalized !
47     QVector2D unitary; // wall's unitary vector ! normalized !
48     int id=0; // only used for debugging
49     qreal thickness; // in meters
50
51     ObstacleType getMaterial();
52
53     //~Obstacle(); // destructor to avoid memory leaks
54 };
55
56 #endif // OBSTACLE_H

```

C.6.2 obstacle.cpp

```

1  #include "obstacle.h"
2
3  #include <QPen>
4  #include <QVector2D>
5  #include "parameters.h"
6
7  /*
8   * The floorplan is 8m x 15m (x=0,y=0 top left to x=15,y=8 bottom right) :
9   * -----
10  * |      |      |      |      |
11  * |      |      |      |      |
12  * |      |      |      |      |
13  * |      |_  _|_  _|_  _|      |
14  * |_  _  _  _  _  _  _  _  _  _/
15  * |      |_  _  _  _  _  _  _  _/
16  * |      |##I      |      |      |
17  * |      |##I      |      |      |
18  * -----
19  *
20  * _ or | : wall (concrete or drywall)

```

```

21 * # : lift (metal walls)
22 * I : metal door
23 * / : glass window
24 *
25 */
26
27 Obstacle::Obstacle(QVector2D start, QVector2D end, ObstacleType material, qreal thickness)//, int id)
28 {
29     // Wall object constructor
30     //qDebug("Creating wall...");
31     this->material = material;
32     //this->id = id;
33     this->thickness = thickness; // in meters
34     this->line = QLineF(start.x(),start.y(), end.x(), end.y());
35     QLineF graphics_line = QLineF(10*start.x(), 10*start.y(), 10*end.x(), 10*end.y());
36     //qDebug() << "Wall" << id << "line:" << this->line ;
37     QLineF normal_line = this->line.normalVector();
38     //qDebug() << "Line" << id << "normal:" << normal_line;
39     this->normal = QVector2D(normal_line.dx(),normal_line.dy()).normalized(); // ! normalized !
40     //qDebug() << "Wall" << id << "normal:" << this->normal;
41     QLineF unit_line = this->line.unitVector();
42     //qDebug() << "Line" << id << "unitary:" << unit_line;
43     this->unitary = QVector2D(unit_line.dx(),unit_line.dy()).normalized(); // ! normalized !
44     //qDebug() << "Wall" << id << "unitary:" << this->unitary;
45     //qDebug("Setting Wall graphics line...");
46     this->graphics->setLine(graphics_line);
47
48     QPen pen(Qt::lightGray);
49     //pen.setWidthF(10*0.2);
50
51     pen.setColor(Qt::lightGray);
52     pen.setWidth(40*0.2);
53     this->properties.relative_permittivity = epsilon_r; // in parameters.h
54     this->properties.conductivity = sigma_cond; // in parameters.h
55
56     // switch (this->material) {
57     // case GenericWall:
58     //     pen.setColor(Qt::lightGray);
59     //     pen.setWidth(10*0.2);
60     //     this->properties.relative_permittivity = epsilon_r;
61     //     this->properties.conductivity = 0.01; // TODO: FIXME: value ?
62     //     break;
63     // case BrickWall:
64     //     pen.setColor(Qt::darkRed);
65     //     pen.setWidthF(10*0.18);
66     //     this->properties.relative_permittivity = 3.95;
67     //     this->properties.conductivity = 0.073;
68     //     break;
69     // case Window:
70     //     pen.setColor(Qt::white);
71     //     pen.setWidthF(10*0.08);
72     //     this->properties.relative_permittivity = 6.3919;
73     //     this->properties.conductivity = 0.00107;
74     //     break;
75     // case MetalWall:
76     //     pen.setColor(Qt::gray);
77     //     pen.setWidthF(10*0.1);
78     //     this->properties.relative_permittivity = 1;
79     //     this->properties.conductivity = 1e7;
80     //     break;
81     // case DryWall:
82     //     pen.setColor(Qt::lightGray);
83     //     pen.setWidthF(10*0.12);
84     //     this->properties.relative_permittivity = 2.7;
85     //     this->properties.conductivity = 0.05349;

```

```

86         // break;
87         // case ConcreteWall:
88         //     pen.setColor(Qt::darkMagenta);
89         //     pen.setWidthF(10*0.2);
90         //     this->properties.relative_permittivity = 6.4954;
91         //     this->properties.conductivity = 1.43;
92         // break;
93         // default:
94         //     pen.setColor(Qt::white);
95         //     pen.setWidthF(10*0.12);
96         //     this->properties.relative_permittivity = 1e-10;
97         //     this->properties.conductivity = 1e-10;
98         // break;
99     // }
100     this->graphics->setPen(pen);
101
102     this->properties.epsilon = epsilon_0 * this->properties.relative_permittivity;
103     complex<qreal> epsilon_tilde = this->properties.epsilon - j * (this->properties.conductivity / omega);
104     this->properties.Z_m = sqrt(mu_0 / epsilon_tilde);
105
106     this->properties.gamma_m = j * omega * sqrt(mu_0 * epsilon_tilde);
107
108     // qDebug("Wall created.");
109     // qDebug() << this->material; // 1: glass, 4: concrete, 2: metal, 3: drywall, 0: brick
110     // qDebug() << "Z_m" << this->properties.Z_m.real() << "+j" << this->properties.Z_m.imag();
111     // qDebug() << "gamma_m" << this->properties.gamma_m.real() << "+j" << this->properties.gamma_m.imag();
112 }
113
114 // Obstacle::~Obstacle() {
115 //     // Obstacle destructor to avoid memory leaks
116 //     // Only delete the graphics if it is NOT managed by a scene
117 //     if (this->graphics && this->graphics->scene() == nullptr) {
118 //         delete this->graphics;
119 //     }
120 // }
121
122 ObstacleType Obstacle::getMaterial()
123 {
124     return this->material;
125 }

```

C.7 ray

C.7.1 ray.h

```

1  #ifndef RAY_H
2  #define RAY_H
3
4  #include "raysegment.h"
5
6  #include <QGraphicsLineItem>
7  #include <complex>
8
9  using namespace std;
10
11 class Ray
12 {
13 public:
14     int tx_selector_index;
15     Ray(QPointF start, QPointF end, int tx_selector_index);
16     QList<RaySegment*> segments; // list of this ray's segment(s)
17     int num_reflections = 0;
18     QPointF start; // ray's starting point (TX)

```

```

19     QPointF end; // ray's end point (RX)
20     QList<complex<qreal>> coeffsList; // list of ray's transmission and reflection coefficients
21     //qreal totalCoeffs=1; // product of all of the ray's coefficients (|T_1|^2 * |G_1|^2 * ...)
22     qreal distance=1e12; // default distance, WILL be changed anyway programmatically
23     void addCoeff(complex<qreal> coeff);
24
25     qreal getTotalCoeffs();
26
27     QList<QGraphicsLineItem*> getSegmentsGraphics();
28
29     qreal getTotalDistance();
30
31     //~Ray(); // destructor to avoid memory leaks
32 };
33
34 #endif // RAY_H

```

C.7.2 ray.cpp

```

1  #include "ray.h"
2
3  #include <QPen>
4  #include "parameters.h"
5
6  Ray::Ray(QPointF start, QPointF end, int tx_selector_index) {
7      // Ray object constructor
8      this->start=start;
9      this->end=end;
10     this->tx_selector_index=tx_selector_index;
11 }
12
13 // Ray::~~Ray() {
14 //     // Ray destructor to avoid memory leaks
15 //     // for (RaySegment* segment : std::as_const(this->segments)) {
16 //         // delete segment;
17 //     }
18 //     qDeleteAll(this->segments);
19 // }
20
21 void Ray::addCoeff(complex<qreal> coeff) {
22     // add a Transmission or Reflection coefficient to this ray's list of coeffs
23     //qDebug() << "Adding coeff to ray:" << coeff_module;
24     // which one to use ? :
25     if (coeff.real() != coeff.real() || coeff.imag() != coeff.imag() || coeff != coeff) {
26         qDebug() << "NaN coeff";
27     }
28     this->coeffsList.append(coeff);
29     //this->totalCoeffs*=pow(abs(coeff),2);
30 }
31
32 qreal Ray::getTotalCoeffs() {
33     // returns the total product of all of the ray's coefficients multiplied by the exponent term
34     //qreal res = totalCoeffs;
35     //res *= pow(abs(exp(-j*beta_0*this->distance)/this->distance),2); // exp term
36     ///qDebug() << "getTotalCoeffs ray:" << res;
37     //return res;
38
39     qreal distance_ray = getTotalDistance();
40     //qDebug() << distance_ray;
41     qreal res = 1;
42     complex<qreal> res_q = 1;
43     for (complex<qreal> coeff : std::as_const(this->coeffsList)) {
44         res_q*=coeff; // all coeffs

```

```

45     }
46     res = pow(abs(res_q),2);
47     res*=pow(abs(exp(-j*beta_0*distance_ray)/distance_ray),2); // exp term
48     //qDebug() << "computeAllCoeffs:" << res;
49     if (res != res) {
50         qDebug() << "NaN Total Coeff";
51     }
52     return res;
53 }
54
55 QList<QGraphicsLineItem*> Ray::getSegmentsGraphics(){
56     // returns this ray's graphics: list of its segment's QGraphicsItems
57     //qDebug() << "Getting ray segments graphics";
58     QPen ray_pen;
59     ray_pen.setWidthF(0.1);
60     // set ray graphics color depending on number of reflections
61     //qDebug() << "This ray has" << this->segments.length() << "segments, so" << this->segments.length()-1 << "reflections";
62     this->num_reflections=this->segments.length()-1;
63     switch (this->num_reflections){
64     case 0: // no reflections: direct ray
65         //qDebug() << "This ray is direct";
66         ray_pen.setColor(Qt::green);
67         break;
68     case 1: // 1 reflection
69         //qDebug() << "This ray has 1 reflection";
70         ray_pen.setColor(Qt::red);
71         break;
72     case 2: // 2 reflections
73         //qDebug() << "This ray has 2 reflections";
74         ray_pen.setColor(Qt::yellow);
75         break;
76     case 3: // 3 reflections
77         //qDebug() << "This ray has 3 reflections";
78         ray_pen.setColor(Qt::cyan);
79         break;
80     }
81     QList<QGraphicsLineItem*> ray_graphics;
82     for (RaySegment* ray_segment : std::as_const(this->segments)) {
83         //for (int i=0; i<this->segments.length(); i++) {
84         //qDebug() << "Adding this segment to list ray_graphics";
85         //ray_segment->graphics->setPen(ray_pen);
86         //qDebug() << "before" << ray_segment->graphics;
87         //ray_graphics.append(ray_segment->graphics);
88
89         QGraphicsLineItem* seg_graphic = new QGraphicsLineItem(
90             ray_segment->x1() * 10,
91             ray_segment->y1() * 10,
92             ray_segment->x2() * 10,
93             ray_segment->y2() * 10
94         );
95         seg_graphic->setPen(ray_pen);
96         ray_graphics.append(seg_graphic);
97
98         //qDebug() << "after:" << ray_graphics;
99     }
100     return ray_graphics;
101 }
102
103 qreal Ray::getTotalDistance() {
104     // returns the total distance of this ray (which is the either the sum of the distances of each segments,
105     // or the distance between the last image and RX)
106
107     // TODO: ?
108     //qreal d = 0;
109     //for (RaySegment* segment : this->segments) {

```

```

110     //    d += segment->distance;
111     //}
112     //return d;
113
114     // ray's distance has already been changed
115     //QDebug() << "Ray getTotalDistance:" << this->distance;
116     return this->distance;
117 }

```

C.8 raysegment

C.8.1 raysegment.h

```

1  #ifndef RAYSEGMENT_H
2  #define RAYSEGMENT_H
3
4  #include <QGraphicsLineItem>
5  #include <QLineF>
6
7
8  class RaySegment : public QLineF
9  {
10 public:
11     qreal distance; ///not used ?
12
13     //QGraphicsLineItem* graphics = new QGraphicsLineItem(); // segment's QGraphicsItem // removed from here to reduce mem
14
15     RaySegment(qreal start_x, qreal start_y, qreal end_x, qreal end_y);
16
17     //~RaySegment(); // destructor to avoid memory leaks
18 };
19
20 #endif // RAYSEGMENT_H

```

C.8.2 raysegment.cpp

```

1  #include "raysegment.h"
2
3  RaySegment::RaySegment(qreal start_x, qreal start_y, qreal end_x, qreal end_y) {
4      // RaySegment object constructor
5      this->setLine(start_x, start_y, end_x, end_y);
6      //this->graphics->setLine(start_x*10, start_y*10, end_x*10, end_y*10);
7      this->distance = this->length();
8  }
9
10 // RaySegment::~~RaySegment() {
11 //     // RaySegment destructor to avoid memory leaks
12 //     // Only delete the graphics if it is NOT managed by a scene
13 //     if (this->graphics && this->graphics->scene() == nullptr) {
14 //         delete this->graphics;
15 //     }
16 // }

```

C.9 receiver

C.9.1 receiver.h

```

1  #ifndef RECEIVER_H

```

```

2  #define RECEIVER_H
3
4  #include "ray.h"
5  #include "transmitter.h"
6
7  #include <QGraphicsRectItem>
8  #include <QVector2D>
9
10 #include <QMap> // new
11
12 #include <QGraphicsSceneMouseEvent> // Required for mouse click events
13
14 class Receiver; // Forward declaration
15
16
17 // Custom graphics item to handle clicks
18 class ReceiverCellGraphics : public QGraphicsRectItem {
19 public:
20     Receiver* parentReceiver;
21     ReceiverCellGraphics(Receiver* parent) : parentReceiver(parent) {}
22
23 protected:
24     void mousePressEvent(QGraphicsSceneMouseEvent *event) override;
25 };
26
27
28 class Receiver : public QVector2D // each cell of the simulation grid acts like a receiver
29 {
30 public:
31
32     Receiver(qreal x, qreal y, qreal resolution, bool showOutline = false);
33
34     // It must be initialized in receiver.cpp
35     QGraphicsRectItem* graphics; // RX's QGraphicsItem
36
37     qreal power; // ! in Watts
38     qreal gain;
39     qulonglong bitrate_Mbps; // bitrate in Mbps
40     qreal Ra = 73; // antenna resistance
41     QColor cell_color = QColor(Qt::transparent); // receiver cell color, based on power/bitrate
42
43     int connected_tx_selector_index = -1; // new
44     Transmitter* connected_tx = nullptr; // new
45
46     QList<Ray*> all_rays; // list of all rays that go to this receiver
47
48     qreal computeTotalPower(Transmitter* transmitter); // returns final total power computation for this RX
49
50     void updateBitrateAndColor(); // update this receiver's bitrate and cell color
51
52     QColor computeColor(qreal value_normalized);
53
54     // new:
55     QMap<int, qreal> computeTDL(Transmitter* transmitter);
56
57     //~Receiver(); // destructor to avoid memory leaks
58 };
59
60 #endif // RECEIVER_H

```

C.9.2 receiver.cpp

```

1  #include "receiver.h"

```

```

2
3 #include <QBrush>
4 #include <QPen>
5 #include "parameters.h"
6
7 // new: (for TDL/PDP charts)
8 #include <QtCharts/QChartView>
9
10 #include <QtCharts/QScatterSeries>
11 #include <QtCharts/QLineSeries>
12 #include <QtCharts/QLegendMarker>
13 #include <QtCharts/QCategoryAxis>
14 #include <QFont>
15
16 #include <QShortcut>
17 #include <QKeySequence>
18 #include <QFileDialog>
19 #include <QPixmap>
20 #include <QPushButton>
21 #include <QVBoxLayout>
22 #include <QHBoxLayout>
23 #include <QPdfWriter>
24 #include <QPainter>
25 #include <QPageSize>
26
27 // #include <QtCharts/QValueAxis>
28 #include <cmath>
29 void showPDPChartWindow(const QMap<int, qreal>& pdp_dBm);
30
31 /*
32  Sensitivity | Bit rate
33  -80 dBm | 100 Mb/s (lower: comms impossible)
34  -60 dBm | 1 Gb/s (higher: data rate capped)
35 */
36
37 // Conversion to bitrate (beware log scale)
38 static const qreal max_sensitivity_mW = std::pow(10.0, max_sensitivity_dBm / 10.0);
39 static const qreal min_sensitivity_mW = std::pow(10.0, min_sensitivity_dBm / 10.0);
40 static const qreal max_bitrate_dB = 10*log10(max_bitrate_Mbps);
41 static const qreal min_bitrate_dB = 10*log10(min_bitrate_Mbps);
42
43 Receiver::Receiver(qreal x, qreal y, qreal resolution, bool showOutline) {
44     // Receiver object constructor
45     QBrush rxBrush(Qt::black);
46     QPen rxPen;
47     if (showOutline){
48         rxPen.setColor(Qt::black);
49         rxPen.setWidthF(10*0.01);
50     } else {
51         rxPen.setColor(Qt::transparent);
52         rxPen.setWidth(0);
53     }
54
55     // Initialize the custom graphics item and pass 'this' receiver to it
56     this->graphics = new ReceiverCellGraphics(this);
57
58     this->setX(x);
59     this->setY(y);
60     this->graphics->setToolTip(QString("Receiver x=%1 y=%2").arg(this->x(), this->y()));
61     this->graphics->setBrush(rxBrush);
62     this->graphics->setPen(rxPen);
63     this->graphics->setRect(10*(x-resolution/2), 10*(y-resolution/2), 10*resolution, 10*resolution);
64     this->graphics->setAcceptHoverEvents(true);
65 }
66

```

```

67 // Receiver::~Receiver() {
68 //     // Receiver destructor to avoid memory leaks
69 //     qDeleteAll(this->all_rays);
70 //     // Only delete the graphics if it is NOT managed by a scene
71 //     if (this->graphics && this->graphics->scene() == nullptr) {
72 //         delete this->graphics;
73 //     }
74 // }
75
76 void Receiver::updateBitrateAndColor()
77 {
78     qreal bitrate;
79     qreal power_dBm = 10*std::log10(this->power*1000);
80     if (this->power != this->power) {
81         bitrate = 999999999999999.9;
82         this->cell_color = QColor::fromRgb(255,192,203); // pink
83         qDebug() << "--- ! ERROR: Cell has NaN power ! ---";
84     } else if (power_dBm > max_sensitivity_dBm) {
85         bitrate = max_bitrate_Mbps; //in Mbps, 40 Gbps max from -40 dBm
86         this->cell_color = QColor::fromRgb(255,0,0); // red
87     } else if (power_dBm < min_sensitivity_dBm) { // 50 Mbps at -90 dBm
88         bitrate = 0; //in Mbps, no connection (0 Mbps)
89         this->cell_color = Qt::transparent; // Qt::transparent or Qt::black or Qt::darkBlue ?
90     } else {
91
92
93         qreal bitrate_dB = min_bitrate_dB + ((power_dBm - min_sensitivity_dBm) / (max_sensitivity_dBm - min_sensitivity_dBm));
94         //qDebug() << "bitrate dB:" << bitrate_dB;
95         bitrate = pow(10.0, bitrate_dB / 10.0); // back to linear
96         //qDebug() << "bitrate (Mbps):" << bitrate;
97
98         qreal value_normalized = (bitrate_dB - min_bitrate_dB) / (max_bitrate_dB - min_bitrate_dB);
99         //qDebug() << "value_normalized:" << value_normalized;
100
101         QColor color = computeColor(value_normalized);
102         // testing in monochrome :
103         //QColor color = QColor::fromRgbF(value_normalized,value_normalized,value_normalized);
104         //qDebug() << "cell color:" << color;
105
106         this->cell_color = color;
107     }
108     //// DEBUG:
109     ////this->cell_color = Qt::black; // set ALL cells to black
110
111     this->bitrate_Mbps = bitrate;
112 }
113
114 QColor Receiver::computeColor(qreal value)
115 {
116     // Maps a normalized value to a 5 color gradient
117
118     // Define colors for the heatmap gradient
119     QColor colors[] = {
120         QColor(0, 0, 255), // Blue
121         QColor(0, 255, 255), // Cyan
122         QColor(0, 255, 0), // Green
123         QColor(255, 255, 0), // Yellow
124         QColor(255, 0, 0) // Red
125     };
126
127     // Determine which two colors to interpolate between
128     int index1 = value * 4;
129     int index2 = index1 + 1;
130
131     // Calculate interpolation factor

```

```

132     qreal factor = (value * 4) - index1;
133
134     // Interpolate between the two colors
135     QColor color = colors[index1].toRgb();
136     QColor next_color = colors[index2].toRgb();
137
138     int red = color.red() + factor * (next_color.red() - color.red());
139     int green = color.green() + factor * (next_color.green() - color.green());
140     int blue = color.blue() + factor * (next_color.blue() - color.blue());
141
142     return QColor(red, green, blue);
143 }
144
145
146 qreal Receiver::computeTotalPower(Transmitter* transmitter) // returns final total power computation for this RX
147
148 {
149     // TODO: far-field cut-off protection ? -> done elsewhere
150     qreal distance_to_tx = this->distanceToPoint(*transmitter);
151     if (distance_to_tx < far_field_min_distance) {
152         return 0.0; // Return 0 power; model is invalid here
153     }
154
155     // Initial E-field amplitude at 1m
156     // for THIS transmitter
157     qreal E0 = sqrt(60.0 * transmitter->gain * transmitter->power);
158
159     complex<qreal> E_tot(0, 0);
160
161     // Get the coordinates of this specific transmitter
162     QPointF tx_pos(transmitter->x(), transmitter->y());
163
164     for (Ray* ray : std::as_const(this->all_rays)) {
165
166         // FILTER: Only add E-fields of rays that belong to THIS transmitter.
167         if (ray->tx_selector_index != transmitter->selector_index) {
168             continue;
169         }
170
171         complex<qreal> ray_coeff(1, 0);
172         for (complex<qreal> coeff : std::as_const(ray->coeffsList)) {
173             ray_coeff *= coeff; // Product of all Gamma_m and T_m for this ray
174         }
175
176         qreal d = ray->getTotalDistance();
177
178         // nchoerent addition: E_n = E0 * (Coeffs) * exp(-j*beta*d) / d
179         complex<qreal> E_ray = E0 * ray_coeff * exp(-j * beta_0 * d) / d;
180         E_tot += E_ray;
181     }
182
183     qreal h_e = lambda / M_PI; // Effective height for lambda/2 dipole
184
185     // P_RX = (1 / 8*Ra) * |h_e * E_tot|^2
186     qreal res = (1.0 / (8.0 * Ra)) * pow(abs(h_e * E_tot), 2);
187
188     if (res != res) qDebug() << "computeTotalPower: NaN !!!";
189
190     return res;
191 }
192
193
194 // new:
195 QMap<int, qreal> Receiver::computeTDL(Transmitter* transmitter)
196 {

```

```

197 QMap<int, complex<qreal>> complex_taps;
198 qreal E0 = sqrt(60.0 * transmitter->gain * transmitter->power);
199 qreal tap_distance_resolution = double(c) / B; // 1.5 meters
200
201 // 1. Bin rays into taps
202 for (Ray* ray : std::as_const(this->all_rays)) {
203     if (ray->tx_selector_index != transmitter->selector_index) {
204         continue;
205     }
206
207     qreal d = ray->getTotalDistance();
208
209     // Find which tap this ray falls into based on its distance
210     int tap_index = qFloor(d/tap_distance_resolution);
211
212     complex<qreal> ray_coeff(1, 0);
213     for (complex<qreal> coeff : std::as_const(ray->coeffsList)) {
214         ray_coeff *= coeff;
215     }
216
217     complex<qreal> E_ray = E0 * ray_coeff * exp(-j * beta_0 * d) / d;
218
219     // Add E-field coherently to the existing tap
220     complex_taps[tap_index] += E_ray;
221 }
222
223 // 2. Compute power for each tap
224 QMap<int, qreal> power_delay_profile;
225
226 for (auto it = complex_taps.constBegin(); it != complex_taps.constEnd(); ++it) {
227     int tap = it.key();
228     complex<qreal> E_tot_tap = it.value();
229
230     qreal tap_power = (1.0 / (8.0 * R_a)) * pow(abs(effective_height * E_tot_tap), 2);
231
232     // Store as dBm for plotting
233     power_delay_profile[tap] = 10.0 * std::log10(tap_power * 1000.0);
234
235 }
236
237 return power_delay_profile; // Key: Tap Index, Value: Power in dBm
238 }
239
240 void ReceiverCellGraphics::mousePressEvent(QGraphicsSceneMouseEvent *event)
241 {
242     if (event->button() == Qt::LeftButton) {
243         qDebug() << "Clicked cell at" << parentReceiver->x() << ", " << parentReceiver->y();
244
245         Transmitter* tx = parentReceiver->connected_tx;
246         if (tx == nullptr) {
247             qDebug() << "Cell has no connected base station (Dead zone).";
248             event->accept();
249             return;
250         }
251
252         QMap<int, qreal> RX_TDL_pdp = parentReceiver->computeTDL(tx);
253
254         showPDPChartWindow(RX_TDL_pdp);
255
256         qDebug() << "--- Power Delay Profile ---";
257         for (auto it = RX_TDL_pdp.constBegin(); it != RX_TDL_pdp.constEnd(); ++it) {
258             qDebug() << "Tap" << it.key() << "(Delay:" << it.key() * 5 << "ns) : " << it.value() << "dBm";
259         }
260
261         event->accept(); // Tell Qt this click was handled

```

```

262     } else {
263         QGraphicsRectItem::mousePressEvent(event); // Pass other clicks (right-click) down
264     }
265 }
266
267
268 // new (for TDL charts):
269 void showPDPChartWindow(const QMap<int, qreal>& pdp_dBm)
270 {
271     if (pdp_dBm.isEmpty()) return;
272
273     // int max_tap = pdp_dBm.lastKey();
274     ////qreal tap_duration_ns = 5.0; // 1 / 200MHz = 5 ns
275     qreal tap_duration_ns = 1 / (B/1e9);
276
277     qreal max_power_dBm = -999.0;
278
279     qreal max_delay_ns = 0.0; // ?
280
281     QScatterSeries *scatterSeries = new QScatterSeries();
282     scatterSeries->setName("Multipath Components");
283     scatterSeries->setMarkerShape(QScatterSeries::MarkerShapeCircle);
284     scatterSeries->setMarkerSize(8.0);
285     scatterSeries->setColor(Qt::blue);
286
287     QLineSeries *stemSeries = new QLineSeries();
288     QPen stemPen(Qt::blue);
289     stemPen.setWidth(2);
290     stemSeries->setPen(stemPen);
291     stemSeries->setName("Stems");
292
293     QCategoryAxis *axisX = new QCategoryAxis();
294     axisX->setTitleText("Delay (ns)");
295     QFont axisFont;
296     axisFont.setPointSize(12); // Larger than default, smaller than main title
297     axisX->setTitleFont(axisFont);
298     axisX->setLabelsPosition(QCategoryAxis::AxisLabelsPositionOnValue);
299     axisX->setLabelsAngle(-90);
300
301     // 1. Process ONLY the taps that actually exist in the map
302     for (auto it = pdp_dBm.constBegin(); it != pdp_dBm.constEnd(); ++it) {
303         int tap = it.key();
304         qreal pwr_dBm = it.value();
305         qreal delay_ns = tap * tap_duration_ns;
306
307         if (pwr_dBm > max_power_dBm) max_power_dBm = pwr_dBm;
308         if (delay_ns > max_delay_ns) max_delay_ns = delay_ns;
309
310         // Add peak dot to chart
311         scatterSeries->append(delay_ns, pwr_dBm);
312
313         // Draw the vertical stem down to the noise floor (-120 dBm)
314         stemSeries->append(delay_ns, -120.0);
315         stemSeries->append(delay_ns, pwr_dBm);
316         stemSeries->append(delay_ns, -120.0); // Return to floor so it doesn't draw a diagonal line to the next peak
317
318         // NEW: Add a specific marker/label for this exact tap
319         axisX->append(QString::number(delay_ns), delay_ns);
320     }
321
322     // 3. Build the Chart
323     QChart *chart = new QChart();
324     chart->addSeries(stemSeries); // Add stem first so it draws behind the dots
325     chart->addSeries(scatterSeries);
326     // Hide the stem legend marker to keep the UI clean

```

```

327     const auto markers = chart->legend()->markers(stemSeries);
328     for (QLegendMarker *marker : markers) {
329         marker->setVisible(false);
330     }
331
332     // Title
333     //QString title = QString("Power Delay Profile (RMS Delay Spread: %1 ns)").arg(rms_delay_spread, 0, 'f', 2);
334     //chart->setTitle(title);
335     chart->setTitle("Power Delay Profile");
336     // Increase Chart Title size
337     QFont titleFont = chart->titleFont();
338     titleFont.setPointSize(16); // Adjust size here (e.g., 14, 16, 18)
339     titleFont.setBold(true);
340     chart->setTitleFont(titleFont);
341     chart->setAnimationOptions(QChart::SeriesAnimations);
342
343     // X-Axis (Delay in ns) now using the CategoryAxis built in the loop
344     axisX->setRange(0, max_delay_ns + 20.0);
345     chart->addAxis(axisX, Qt::AlignBottom);
346     stemSeries->attachAxis(axisX);
347     scatterSeries->attachAxis(axisX);
348
349     // Y-Axis (Power in dBm)
350     QValueAxis *axisY = new QValueAxis();
351     axisY->setTitleText("Power (dBm)");
352     axisY->setTitleFont(axisFont);
353     axisY->setRange(-120.0, max_power_dBm + 5.0); // Dynamic range based on strongest signal
354     chart->addAxis(axisY, Qt::AlignLeft);
355     stemSeries->attachAxis(axisY);
356     scatterSeries->attachAxis(axisY);
357
358     // 4. Create the Chart View
359     QChartView *chartView = new QChartView(chart);
360     chartView->setRenderHint(QPainter::Antialiasing);
361
362     // 5. Create a Wrapper Window with a Layout
363     QWidget *window = new QWidget();
364     window->setWindowTitle("TDL Impulse Response");
365     window->resize(850, 500);
366     window->setAttribute(Qt::WA_DeleteOnClose);
367
368     QVBoxLayout *mainLayout = new QVBoxLayout(window);
369     mainLayout->addWidget(chartView, 1);
370
371     // 6. Create the Export Buttons
372     QHBoxLayout *buttonLayout = new QHBoxLayout();
373     QPushButton *btnSavePng = new QPushButton("Save as PNG");
374     //QPushButton *btnSavePdf = new QPushButton("Save as PDF");
375
376     btnSavePng->setMinimumHeight(30);
377     //btnSavePdf->setMinimumHeight(30);
378
379     buttonLayout->addWidget(btnSavePng);
380     //buttonLayout->addWidget(btnSavePdf);
381     buttonLayout->addStretch();
382
383     mainLayout->addLayout(buttonLayout);
384
385     // 7. Connect Button Clicks to Save Functions
386     QObject::connect(btnSavePng, &QPushButton::clicked, [window, chartView]() {
387         QString fileName = QFileDialog::getSaveFileName(window, "Save Chart as Image", "", "PNG Image (*.png)");
388         if (!fileName.isEmpty()) {
389             QPixmap pixmap = chartView->grab();
390             pixmap.save(fileName);
391         }

```

```

392     });
393
394     // QObject::connect(btnSavePdf, &QPushButton::clicked, [window, chartView]() {
395     //     QString fileName = QFileDialog::getSaveFileName(window, "Save Chart as PDF", "", "PDF Document (*.pdf)");
396     //     if (!fileName.isEmpty()) {
397     //         QPdfWriter writer(fileName);
398     //         writer.setCreator("5G Channel Ray Tracer");
399     //         writer.setPageSize(QPageSize(QPageSize::A4));
400     //         writer.setPageOrientation(QPageLayout::Landscape);
401     //
402     //         QPainter painter(&writer);
403     //         chartView->render(&painter);
404     //         painter.end();
405     //     }
406     // });
407
408     // --- Add Keyboard Shortcut (Ctrl+S) ---
409     QShortcut *saveShortcut = new QShortcut(QKeySequence::Save, window); // Automatically maps to Ctrl+S
410
411     QObject::connect(saveShortcut, &QShortcut::activated, [window, chartView]() {
412         QString fileName = QFileDialog::getSaveFileName(
413             window,
414             "Save Chart as Image",
415             QDir::currentPath(),
416             "PNG Image (*.png);;JPEG (*.jpg)"
417         );
418
419         if (!fileName.isEmpty()) {
420             bool success = chartView->grab().save(fileName);
421             if (success) {
422                 qDebug() << "Chart successfully saved to:" << fileName;
423             }
424         }
425     });
426
427     // show the window
428     //chartView->show();
429     window->show();
430 }

```

C.10 transmitter

C.10.1 transmitter.h

```

1  #ifndef TRANSMITTER_H
2  #define TRANSMITTER_H
3
4  #include <QGraphicsEllipseItem>
5  #include <QVector2D>
6  #include <QPen>
7  #include <QBrush>
8
9  #include "parameters.h"
10
11 class Transmitter : public QVector2D
12 {
13 public:
14     int getPower_dBm() const;
15     qreal getPower() const;
16     qreal getG_TXP_TX() const;
17     void setPower_dBm(int power_dBm);
18     QPointF getCoordinates() const;
19     void changeCoordinates(QPointF new_coordinates);

```

```

20
21     int selector_index;
22     QString name;
23     qreal coverage_area_sqm = 0.0;
24
25     Transmitter(qreal x, qreal y, int selector_index, QString name);
26
27     QGraphicsEllipseItem* graphics = new QGraphicsEllipseItem(); // TX's QGraphicsItem
28     qreal power = P_TX; // ! in Watts, 20dBm = 0.1W
29     qreal power_dBm = P_TX_dBm; // 20dBm
30     qreal gain = G_TX; // 1.64 or 1.7 ?
31
32     //~Transmitter(); // destructor to avoid memory leaks
33
34 private:
35     void setGraphicsRect(qreal x,qreal y);
36 };
37
38 #endif // TRANSMITTER_H

```

C.10.2 transmitter.cpp

```

1  #include "transmitter.h"
2
3  qreal _factor = 3;
4
5  Transmitter::Transmitter(qreal x, qreal y, int selector_index, QString name){
6      // Transmitter object constructor
7      QBrush txBrush(Qt::white);
8      QPen txPen(Qt::gray);
9      txPen.setWidthF(_factor*5*0.07);
10
11      this->selector_index = selector_index;
12      this->name = name;
13
14      this->setX(x);
15      this->setY(y);
16      this->graphics->setToolTip(QString("Test transmitter x=%1 y=%2").arg(this->x(),this->y()));
17      this->graphics->setBrush(txBrush);
18      this->graphics->setPen(txPen);
19      //this->graphics->setRect(10*x-1.5,10*y-1.5,3,3);
20      this->setGraphicsRect(x,y);
21      this->graphics->setAcceptHoverEvents(true);
22  };
23
24  // Transmitter::~Transmitter() {
25  //     // Transmitter destructor to avoid memory leaks
26  //     // Only delete the graphics if it is NOT managed by a scene
27  //     if (this->graphics && this->graphics->scene() == nullptr) {
28  //         delete this->graphics;
29  //     }
30  // }
31
32  void Transmitter::setGraphicsRect(qreal x,qreal y)
33  {
34      qreal _nb = 2.4;
35      this->graphics->setRect(10*x-(_factor*_nb/2),10*y-(_factor*_nb/2),_factor*_nb,_factor*_nb);
36  }
37
38  int Transmitter::getPower_dBm() const
39  {
40      return 10*std::log10(this->power*1000);
41  }

```

```

42
43 double Transmitter::getPower() const
44 {
45     return this->power;
46 }
47
48 qreal Transmitter::getG_TXP_TX() const
49 {
50     return this->gain*this->power;
51 }
52
53 void Transmitter::setPower_dBm(int power_dBm)
54 {
55     this->power = pow(10.0, power_dBm / 10.0) / 1000.0; // dBm to Watts
56     qDebug() << "setPower:" << this->power << "Watts";
57 }
58
59 void Transmitter::changeCoordinates(QPointF new_coordinates)
60 {
61     this->setX(new_coordinates.x());
62     this->setY(new_coordinates.y());
63     this->setGraphicsRect(this->x(),this->y());
64 }
65
66 QPointF Transmitter::getCoordinates() const {
67     return this->toPointF();
68 }

```

C.11 simulation

C.11.1 simulation.h

```

1  #ifndef SIMULATION_H
2  #define SIMULATION_H
3
4  #include <QElapsedTimer>
5  #include <QGraphicsView>
6  #include <QProgressBar>
7
8  #include "obstacle.h"
9  #include "ray.h"
10 #include "receiver.h"
11 #include "transmitter.h"
12
13 // new: for TDL/PDP on cell click
14 #include <QObject>
15 #include <QEvent>
16 #include <QGraphicsSceneMouseEvent>
17 #include <QChartView>
18
19 class Simulation {
20 public:
21     Simulation(bool show = true); // constructor
22     bool ran = false;
23     bool is_running = false;
24     qreal resolution = 1.0; // meters, default is (0.5m x 0.5m, no 1x1 now) 1sqm cells
25
26     void run(QProgressBar* progress_bar);
27     void createBaseStation(Transmitter* transmitter);
28     Transmitter* getBaseStation(int index);
29     void deleteBaseStation(int index);
30     QList<Obstacle*>* getObstacles();
31     unsigned int getNumberOfObstacles();

```

```

32     unsigned int getNumberOfBaseStations();
33
34     qint64 getSimulationTime() const;
35
36     QGraphicsScene* scene = nullptr;
37     QGraphicsView* view = nullptr;
38
39     bool lift_is_on_floor = false;
40     bool show_cell_outline = false;
41     int max_ray_reflections = 3; // TODO: 3 reflections
42     bool showRaySingleCell = false;
43     bool remove_all_walls = false;
44
45     // --- NEW : ---
46     QList<Transmitter*> baseStations; // TX list
47     QList<QList<Receiver*>> cells; // RX matrix
48     QList<Obstacle*> obstacles; // walls list
49
50     QChartView* showPathLossScatterPlot();
51
52
53     qreal singleCellX=2;
54     qreal singleCellY=2;
55     QList<QVector2D> singleCellSimReflectionPoints = {};
56     // -----
57     bool show = true;
58 private:
59     QElapsedTimer timer;
60     qint64 simulation_time;
61
62     void createCellsMatrix();
63
64     // --- new : ---
65     void createWalls();
66     QGraphicsScene* createGraphicsScene();
67     void computeReflections(Receiver* _RX, const QVector2D& _TX, int tx_selector_index);
68     void computeDirect(Receiver* _RX, const QVector2D& _TX, int tx_selector_index);
69     void addReflection(Ray* _ray, const QVector2D& _p1, const QVector2D& _p2, Obstacle* wall);
70     complex<qreal> makeTransmission(RaySegment* ray_segment, Obstacle* wall);
71     void checkTransmissions(Ray* _ray, QList<Obstacle*> _reflection_walls);
72     //complex<qreal> computePerpendicularGamma(qreal _cos_theta_i, qreal _cos_theta_t, Obstacle* wall);
73     complex<qreal> computeReflectionCoeff(qreal _cos_theta_i, qreal _sin_theta_i, qreal _cos_theta_t, qreal _sin_theta_t);
74     complex<qreal> computeTransmissionCoeff(qreal _cos_theta_i, qreal _sin_theta_i, qreal _cos_theta_t, qreal _sin_theta_t);
75     bool checkSameSideOfWall(const QVector2D& _normal, const QVector2D& _TX, const QVector2D& _RX);
76     bool checkRaySegmentIntersectsWall(const Obstacle* wall, RaySegment* ray_segment, QPointF* intersection_point=nullptr);
77     QVector2D computeImage(const QVector2D& _point, Obstacle* wall);
78     QVector2D calculateReflectionPoint(const QVector2D& _start, const QVector2D& _end, Obstacle* wall);
79
80     void showView();
81     void addLegend(QGraphicsScene* scene);
82
83 };
84
85 #endif // SIMULATION_H

```

C.11.2 simulation.cpp

```

1  #include "simulation.h"
2
3  #include <QDir>
4  #include <QLineSeries>
5  #include <QtMath>
6

```

```

7  #include "parameters.h"
8
9  // for path loss plot:
10 #include <QtCharts/QChartView>
11 #include <QtCharts/QScatterSeries>
12 #include <QtCharts/QLogValueAxis>
13 #include <QtCharts/QValueAxis>
14 #include <QPushButton>
15 #include <QVBoxLayout>
16 #include <QHBoxLayout>
17 #include <QFileDialog>
18 #include <QShortcut>
19 #include <QtCharts/QLegendMarker>
20 #include <QtCharts/QAreaSeries>
21
22
23 Simulation::Simulation(bool show) {
24     // class constructor
25     this->show = show;
26
27     /// createWalls(); // done in Simulation::run()
28 }
29
30 void Simulation::createWalls()
31 {
32     // qDebug("Creating walls...");
33
34     // Delete old obstacles vector objects (avoid memory leak)
35     if(!this->obstacles.empty()) {
36         // The vector is not empty
37         for(auto ptr : std::as_const(this->obstacles)) {
38             delete ptr;
39         }
40         this->obstacles.clear();
41     }
42
43     // --- Outdoor Urban Environment (120x70m Layout) ---
44     QList<Obstacle*> urban_walls;
45     ObstacleType building_material = GenericWall;
46     qreal thickness = 0.5; // 50cm thick exterior concrete walls
47
48     // The layout features a Main Avenue (Y = 30 to 45) and a Cross Street (X = 50 to 70).
49
50     // 1. Building Block 1 (Top-Left): Features a chamfered intersection corner
51     urban_walls.append(new Obstacle(QVector2D(0, 30), QVector2D(45, 30), building_material, thickness)); // South facing
52     urban_walls.append(new Obstacle(QVector2D(45, 30), QVector2D(50, 25), building_material, thickness)); // Chamfered corner
53     urban_walls.append(new Obstacle(QVector2D(50, 25), QVector2D(50, 0), building_material, thickness)); // East facing
54
55     // 2. Building Block 2 (Top-Right): Features a 10m wide, 20m deep alleyway
56     urban_walls.append(new Obstacle(QVector2D(70, 0), QVector2D(70, 30), building_material, thickness)); // West facing
57     urban_walls.append(new Obstacle(QVector2D(70, 30), QVector2D(90, 30), building_material, thickness)); // South facing
58     urban_walls.append(new Obstacle(QVector2D(100, 30), QVector2D(120, 30), building_material, thickness)); // South facing
59
60     // Alleyway walls
61     urban_walls.append(new Obstacle(QVector2D(90, 30), QVector2D(90, 10), building_material, thickness)); // Alley West facing
62     urban_walls.append(new Obstacle(QVector2D(100, 30), QVector2D(100, 10), building_material, thickness)); // Alley East facing
63     urban_walls.append(new Obstacle(QVector2D(90, 10), QVector2D(100, 10), building_material, thickness)); // Alley back wall
64
65     // 3. Building Block 3 (Bottom-Left): Standard 90-degree block
66     urban_walls.append(new Obstacle(QVector2D(0, 45), QVector2D(50, 45), building_material, thickness)); // North facing
67     urban_walls.append(new Obstacle(QVector2D(50, 45), QVector2D(50, 70), building_material, thickness)); // East facing
68
69     // 4. Building Block 4 (Bottom-Right): Features a recessed corner plaza
70     urban_walls.append(new Obstacle(QVector2D(85, 45), QVector2D(120, 45), building_material, thickness)); // North facing
71     urban_walls.append(new Obstacle(QVector2D(85, 45), QVector2D(85, 55), building_material, thickness)); // Plaza West facing

```

```

72     urban_walls.append(new Obstacle(QVector2D(70, 55), QVector2D(85, 55), building_material, thickness)); // Plaza North
73     urban_walls.append(new Obstacle(QVector2D(70, 55), QVector2D(70, 70), building_material, thickness)); // West facing
74
75     QList<Obstacle*> all_obstacles;
76
77     //all_obstacles.append(boundary_walls);
78     //all_obstacles.append(interior_walls);
79     all_obstacles.append(urban_walls);
80     //this->obstacles = walls;
81
82     this->obstacles = all_obstacles;
83
84     //    // ----- DEBUG: remove all walls -----
85     // #if REMOVE_ALL_WALLS
86     //     for(auto ptr : this->obstacles) {
87     //         delete ptr;
88     //     }
89     //     this->obstacles.clear();
90     // #endif
91     //    // -----
92
93     qDebug() << QString::number(this->obstacles.length()) << "Walls created";
94 }
95
96 void Simulation::run(QProgressBar* progress_bar)
97 {
98     // WARNING: CLEAN UP PREVIOUS QT LAYOUTS FIRST before destroying math!
99     if (this->view) {
100         delete this->view;
101         this->view = nullptr;
102     }
103     if (this->scene) {
104         // Deleting the scene will automatically destroy every QGraphicsItem inside it
105         delete this->scene;
106         this->scene = nullptr;
107     }
108
109     this->timer.start();
110     qDebug() << "Simulation::run() - single cell simulation: " << (this->showRaySingleCell);
111
112     qDebug() << "P_TX:" << P_TX << "W," << P_TX_dBm << "dBm";
113     qDebug() << "G_TX:" << G_TX;
114     qDebug() << "beta:" << beta_0;
115     qDebug() << "lambda:" << lambda << "m";
116     qDebug() << "frequency:" << freq << "Hz";
117     qDebug() << "omega:" << omega << "rad/s";
118     qDebug() << "Exclusion zone for not far-field:" << far_field_min_distance << "m";
119
120
121     for (QList<Receiver*>& cells_line : this->cells) {
122         qDebug() << "Deleting cells_line";
123     }
124     this->cells.clear();
125
126     qDebug() << "Deleting obstacles";
127     this->obstacles.clear();
128
129     if (this->remove_all_walls) {
130         qDebug() << "Running without any wall";
131     } else {
132         createWalls();
133     }
134
135     if (!this->showRaySingleCell) {
136         createCellsMatrix();

```

```

137     } else {
138         // single cell simulation
139         Receiver* rx = new Receiver(this->singleCellX, this->singleCellY, 0.5, true);
140         QPen singleCellPen = QPen(Qt::magenta);
141         singleCellPen.setWidthF(10*0.03);
142         rx->graphics->setPen(singleCellPen);
143         this->cells = {{rx}};
144     }
145
146     if (this->cells.isEmpty()) {
147         qWarning("no cells provided (simulation.cells matrix is empty)");
148         throw std::exception();
149     }
150
151     for (Transmitter* tx : std::as_const(this->baseStations)) {
152         tx->coverage_area_sqm = 0.0; // reset
153         int i=0;
154         for (QList<Receiver*>& cells_line : this->cells){
155             // loops over each line
156             for (Receiver* cell : cells_line) {
157                 computeDirect(cell, *tx, tx->selector_index);
158                 computeReflections(cell, *tx, tx->selector_index);
159                 progress_bar->setValue(i/(cells_line.length()*this->cells.length()));
160                 i++;
161             }
162         }
163     }
164     qDebug() << this->cells.length()*this->cells[0].length() << "cells,"; // << i << "computed";
165
166     //end of simulation
167     this->simulation_time = this->timer.elapsed();
168     qDebug() << "Simulation time:" << this->simulation_time << "ms";
169
170     if (this->show) {
171         showView();
172     }
173
174 }
175
176 void Simulation::computeReflections(Receiver* _RX, const QVector2D& _TX, int tx_selector_index)
177 {
178     // 1st reflection :
179     for (int i=0; i<this->obstacles.length(); i++) {
180         Obstacle* wall = this->obstacles[i];
181
182         if (checkSameSideOfWall(wall->normal, _TX, *_RX)) {
183             QVector2D _imageTX = computeImage(_TX, wall);
184             QVector2D _P_r = calculateReflectionPoint(_imageTX, *_RX, wall);
185
186             // CHECK IF REFLECTION IS ON THE WALL AND NOT ITS EXTENSION:
187             RaySegment* test_segment = new RaySegment(_imageTX.x(), _imageTX.y(), _RX->x(), _RX->y());
188             bool valid_1st_reflection = checkRaySegmentIntersectsWall(wall, test_segment);
189             //delete test_segment->graphics; // fix memory leak
190             delete test_segment;
191
192             if (valid_1st_reflection) {
193                 Ray* ray_1_reflection = new Ray(_TX.toPointF(), _RX->toPointF(), tx_selector_index);
194                 QList<RaySegment*> ray_segments;
195                 ray_segments.append(new RaySegment(_TX.x(), _TX.y(), _P_r.x(), _P_r.y()));
196                 ray_segments.append(new RaySegment(_P_r.x(), _P_r.y(), _RX->x(), _RX->y()));
197
198                 ray_1_reflection->segments = ray_segments;
199                 addReflection(ray_1_reflection, _imageTX, *_RX, wall);
200                 checkTransmissions(ray_1_reflection, {wall});
201

```

```

202         if (this->showRaySingleCell) {
203             this->singleCellSimReflectionPoints.append(_P_r);
204         }
205
206         ray_1_reflection->distance = QVector2D(*_RX-_imageTX).length();
207         _RX->all_rays.append(ray_1_reflection);
208     }
209
210     // 2nd reflection (we always process this if TX and RX are on the same side,
211     // even if the 1st reflection was invalid, because the image still exists)
212     for (Obstacle* wall_2 : std::as_const(this->obstacles)) {
213         if (wall_2 != wall && checkSameSideOfWall(wall_2->normal, _imageTX, *_RX)) {
214             QVector2D _image_imageTX = computeImage(_imageTX, wall_2);
215             QVector2D _P_r_2_last = calculateReflectionPoint(_image_imageTX, *_RX, wall_2);
216             QVector2D _P_r_2_first = calculateReflectionPoint(_imageTX, _P_r_2_last, wall);
217
218             RaySegment* test_segment_1 = new RaySegment(_image_imageTX.x(), _image_imageTX.y(), _RX->x(), _RX->y());
219             RaySegment* test_segment_2 = new RaySegment(_imageTX.x(), _imageTX.y(), _P_r_2_last.x(), _P_r_2_last.y());
220
221             bool valid_2nd_reflection = checkRaySegmentIntersectsWall(wall_2, test_segment_1) &&
222                                     checkRaySegmentIntersectsWall(wall, test_segment_2);
223             //delete test_segment_1->graphics; // fix memory leak
224             delete test_segment_1;
225             //delete test_segment_2->graphics; // fix memory leak
226             delete test_segment_2;
227
228             if (valid_2nd_reflection) {
229                 Ray* ray_2_reflection = new Ray(_TX.toPointF(), _RX->toPointF(), tx_selector_index);
230                 QList<RaySegment*> ray_segments_2;
231                 ray_segments_2.append(new RaySegment(_TX.x(), _TX.y(), _P_r_2_first.x(), _P_r_2_first.y()));
232                 ray_segments_2.append(new RaySegment(_P_r_2_first.x(), _P_r_2_first.y(), _P_r_2_last.x(), _P_r_2_last.y()));
233                 ray_segments_2.append(new RaySegment(_P_r_2_last.x(), _P_r_2_last.y(), _RX->x(), _RX->y()));
234
235                 ray_2_reflection->segments = ray_segments_2;
236                 addReflection(ray_2_reflection, _imageTX, _P_r_2_last, wall);
237                 addReflection(ray_2_reflection, _image_imageTX, *_RX, wall_2);
238                 checkTransmissions(ray_2_reflection, {wall, wall_2});
239
240                 if (this->showRaySingleCell) {
241                     this->singleCellSimReflectionPoints.append(_P_r_2_first);
242                     this->singleCellSimReflectionPoints.append(_P_r_2_last);
243                 }
244
245                 ray_2_reflection->distance = QVector2D(*_RX-_image_imageTX).length();
246                 _RX->all_rays.append(ray_2_reflection);
247             }
248
249     // 3rd reflection
250     for (Obstacle* wall_3 : std::as_const(this->obstacles)) {
251         if (wall_3 != wall_2 && checkSameSideOfWall(wall_3->normal, _image_imageTX, *_RX)) {
252             QVector2D _image_image_imageTX = computeImage(_image_imageTX, wall_3);
253
254             QVector2D _P_r_3_last = calculateReflectionPoint(_image_image_imageTX, *_RX, wall_3);
255             QVector2D _P_r_3_mid = calculateReflectionPoint(_image_imageTX, _P_r_3_last, wall_2);
256             QVector2D _P_r_3_first = calculateReflectionPoint(_imageTX, _P_r_3_mid, wall);
257
258             RaySegment* test_segment_3_1 = new RaySegment(_image_image_imageTX.x(), _image_image_imageTX.y(), _RX->x(), _RX->y());
259             RaySegment* test_segment_3_2 = new RaySegment(_image_imageTX.x(), _image_imageTX.y(), _P_r_3_mid.x(), _P_r_3_mid.y());
260             RaySegment* test_segment_3_3 = new RaySegment(_imageTX.x(), _imageTX.y(), _P_r_3_first.x(), _P_r_3_first.y());
261
262             bool valid_3rd_reflection = checkRaySegmentIntersectsWall(wall_3, test_segment_3_1) &&
263                                     checkRaySegmentIntersectsWall(wall_2, test_segment_3_2) &&
264                                     checkRaySegmentIntersectsWall(wall, test_segment_3_3);
265
266             //delete test_segment_3_1->graphics; // fix memory leak

```

```

267         delete test_segment_3_1;
268         //delete test_segment_3_2->graphics; // fix memory leak
269         delete test_segment_3_2;
270         //delete test_segment_3_3->graphics; // fix memory leak
271         delete test_segment_3_3;
272
273         if (valid_3rd_reflection) {
274             Ray* ray_3_reflection = new Ray(_TX.toPointF(), _RX->toPointF(), tx_selector_index);
275             QList<RaySegment*> ray_segments_3;
276             ray_segments_3.append(new RaySegment(_TX.x(), _TX.y(), _P_r_3_first.x(), _P_r_3_first.y()));
277             ray_segments_3.append(new RaySegment(_P_r_3_first.x(), _P_r_3_first.y(), _P_r_3_mid.x(),
278             ray_segments_3.append(new RaySegment(_P_r_3_mid.x(), _P_r_3_mid.y(), _P_r_3_last.x(), _P_r_3_last.y()));
279             ray_segments_3.append(new RaySegment(_P_r_3_last.x(), _P_r_3_last.y(), _RX->x(), _RX->y()));
280
281             ray_3_reflection->segments = ray_segments_3;
282
283             addReflection(ray_3_reflection, _imageTX, _P_r_3_mid, wall);
284             addReflection(ray_3_reflection, _image_imageTX, _P_r_3_last, wall_2);
285             addReflection(ray_3_reflection, _image_image_imageTX, *_RX, wall_3);
286             checkTransmissions(ray_3_reflection, {wall, wall_2, wall_3});
287
288             if (this->showRaySingleCell) {
289                 this->singleCellSimReflectionPoints.append(_P_r_3_first);
290                 this->singleCellSimReflectionPoints.append(_P_r_3_mid);
291                 this->singleCellSimReflectionPoints.append(_P_r_3_last);
292             }
293
294             ray_3_reflection->distance = QVector2D(*_RX - _image_image_imageTX).length();
295             _RX->all_rays.append(ray_3_reflection);
296         }
297     }
298 }
299 }
300 }
301 }
302 }
303 }
304
305 // compute the image position
306 QVector2D Simulation::computeImage(const QVector2D& _point, Obstacle* wall) {
307     // returns the coordinates of _point's image with wall
308     QVector2D new_origin = QVector2D(wall->line.p1()); // set origin to point1 of wall
309     //qDebug() << "new coords" << wall->line.p1();
310     QVector2D _normal = wall->normal; // normal to the wall (is normalized so it is relative to any origin)
311     //qDebug() << "normal" << wall->normal;
312     QVector2D new_point_coords = _point - new_origin; // initial point in new coordinates relative to point1 of wall
313     double _dotProduct = QVector2D::dotProduct(new_point_coords, _normal);
314     QVector2D _image_new_coords = new_point_coords - 2 * _dotProduct * _normal; // image point in new coordinates relative to point1 of wall
315     QVector2D _image = new_origin + _image_new_coords; // image point in absolute coordinates
316     //qDebug() << "image:" << _image.x() << _image.y();
317
318     return _image;
319 }
320
321 // compute the reflection point on the wall
322 QVector2D Simulation::calculateReflectionPoint(const QVector2D& _start, const QVector2D& _end, Obstacle* wall) {
323     // returns the intersection between the line from _start to _end and the wall
324     QVector2D d = _end - _start;
325     //QVector2D x0(0,0); // TODO: always this ?
326     QVector2D x0 = QVector2D(wall->line.p1()); // TODO: FIXME: correct ?
327     qreal t = ((d.y()*(_start.x()-x0.x()))-(d.x()*(_start.y()-x0.y())))/(wall->unitary.x()*d.y()-wall->unitary.y()*d.x());
328     QVector2D P_r = x0 + t * wall->unitary;
329     return P_r;
330 }
331

```

```

332 void Simulation::addReflection(Ray* _ray, const QVector2D& _p1, const QVector2D& _p2, Obstacle* wall){
333     // computes the final Gamma coeff for the ray_segment's reflection with this wall, and adds it to this ray's coeffs list
334     QVector2D _d = _p2-_p1;
335     qreal _cos_theta_i = abs(QVector2D::dotProduct(_d.normalized(),wall->normal));
336     qreal _sin_theta_i = sqrt(1.0 - pow(_cos_theta_i,2));
337     qreal _sin_theta_t = _sin_theta_i / sqrt(wall->properties.relative_permittivity);
338     qreal _cos_theta_t = sqrt(1.0 - pow(_sin_theta_t,2));
339     complex<qreal> Gamma_coeff = computeReflectionCoeff(_cos_theta_i,_sin_theta_i,_cos_theta_t,_sin_theta_t, wall);
340     //qDebug() << "addReflection, Gamma_coeff:" << Gamma_coeff;
341     if (Gamma_coeff.real() != Gamma_coeff.real() || Gamma_coeff.imag() != Gamma_coeff.imag()) {
342         qDebug() << "Gamma_coeff = NaN";
343         qDebug() << "_d" << _d;
344         qDebug() << "_cos_theta_i" << _cos_theta_i;
345         qDebug() << "_sin_theta_i" << _sin_theta_i;
346         qDebug() << "_sin_theta_t" << _sin_theta_t;
347         qDebug() << "_cos_theta_t" << _cos_theta_t;
348     }
349     _ray->addCoeff(Gamma_coeff);
350 }
351
352 complex<qreal> Simulation::makeTransmission(RaySegment* ray_segment, Obstacle* wall) {
353     // computes the final T coeff for the ray_segment's transmission with this wall, and adds it to this ray's coeffs list
354     QVector2D _d = QVector2D(ray_segment->p1())-QVector2D(ray_segment->p2());
355     qreal _cos_theta_i = abs(QVector2D::dotProduct(_d.normalized(),wall->normal));
356     qreal _sin_theta_i = sqrt(1.0 - pow(_cos_theta_i,2));
357     qreal _sin_theta_t = _sin_theta_i / sqrt(wall->properties.relative_permittivity);
358     qreal _cos_theta_t = sqrt(1.0 - pow(_sin_theta_t,2));
359
360     complex<qreal> T_coeff = computeTransmissionCoeff(_cos_theta_i,_sin_theta_i,_cos_theta_t,_sin_theta_t,wall);
361     if (T_coeff.real() != T_coeff.real() || T_coeff.imag() != T_coeff.imag()) {
362         qDebug() << "Gamma_coeff = NaN";
363         qDebug() << "_d" << _d;
364         qDebug() << "_cos_theta_i" << _cos_theta_i;
365         qDebug() << "_sin_theta_i" << _sin_theta_i;
366         qDebug() << "_sin_theta_t" << _sin_theta_t;
367         qDebug() << "_cos_theta_t" << _cos_theta_t;
368     }
369     return T_coeff;
370 }
371
372 void Simulation::checkTransmissions(Ray* _ray, QList<Obstacle*> _reflection_walls) {
373     // checks for every segment in this ray if they intersect a wall (which isn't a wall already used for a reflection by
374     // if so: adds the Transmission coefficient to this ray's coeffs list
375     for (RaySegment* ray_segment : std::as_const(_ray->segments)) {
376         for (Obstacle* wall : std::as_const(this->obstacles)) {
377             //qDebug() << "pwall" << &wall;
378             if (!_reflection_walls.contains(wall)) { // is NOT reflection wall
379                 if (checkRaySegmentIntersectsWall(wall, ray_segment,nullptr)) {
380                     complex<qreal> T_coeff = makeTransmission(ray_segment,wall);
381                     //qDebug() << "checkTransmission, T_coeff:" << T_coeff;
382                     _ray->addCoeff(T_coeff);
383                 }
384             }
385         }
386     }
387
388 void Simulation::computeDirect(Receiver* _RX, const QVector2D& _TX, int tx_selector_index)
389 {
390     // Computes the direct ray: checks all walls between RX and TX and adds
391     // their computed transmission coefficients to the direct ray list of coeffs
392     Ray* direct_ray = new Ray(_TX.toPointF(), _RX->toPointF(), tx_selector_index);
393     RaySegment* _direct_line = new RaySegment(_RX->x(), _RX->y(), _TX.x(), _TX.y());
394     for (Obstacle* wall : std::as_const(this->obstacles)) {
395         QPointF* intersection_point = nullptr; // not used
396         if (checkRaySegmentIntersectsWall(wall, _direct_line, intersection_point)) {

```

```

397         // transmission through this wall, compute the transmission coeff
398         complex<qreal> T_coeff = makeTransmission(_direct_line,wall);
399         direct_ray->addCoeff(T_coeff);
400     }
401     //else {
402     //     continue;
403     //}
404 }
405 //QDebug() << "ray_direct distance:" << QVector2D(*_RX - _TX).length();
406 direct_ray->distance = QVector2D(*_RX-_TX).length();
407 //QDebug() << "Ray's (direct) total coeffs:" << direct_ray->getTotalCoeffs();
408 direct_ray->segments = {_direct_line};
409 _RX->all_rays.append(direct_ray);
410 }
411
412
413 complex<qreal> Simulation::computeReflectionCoeff(qreal _cos_theta_i, qreal _sin_theta_i, qreal _cos_theta_t, qreal _sin_theta_t)
414 {
415     // returns the reflection coefficient Gamma_m
416
417     // Assume walls are dielectric slabs (depends on the Fresnel reflection coefficient of the interface and the slab's thickness)
418     qreal eps_r = wall->properties.relative_permittivity;
419
420     // Normal polarization Fresnel reflection (Gamma_perp)
421     complex<qreal> Gamma_perp = (_cos_theta_i - sqrt(eps_r - pow(_sin_theta_i, 2))) /
422                               (_cos_theta_i + sqrt(eps_r - pow(_sin_theta_i, 2)));
423
424     // Phase thickness (q)
425     qreal q = beta_0 * wall->thickness * sqrt(eps_r - pow(_sin_theta_i, 2));
426
427     // Slab reflection (Gamma_m) !
428     complex<qreal> num = Gamma_perp * (1.0 - exp(-2.0 * j * q));
429     complex<qreal> den = 1.0 - pow(Gamma_perp, 2) * exp(-2.0 * j * q);
430     return num / den;
431 }
432
433 complex<qreal> Simulation::computeTransmissionCoeff(qreal _cos_theta_i, qreal _sin_theta_i, qreal _cos_theta_t, qreal _sin_theta_t)
434 {
435     // returns the transmission coefficient T_m
436
437     // Assume walls are dielectric slabs (depends on the Fresnel reflection coefficient of the interface and the slab's thickness)
438     qreal eps_r = wall->properties.relative_permittivity;
439
440     // Normal polarization Fresnel reflection (Gamma_perp)
441     complex<qreal> Gamma_perp = (_cos_theta_i - sqrt(eps_r - pow(_sin_theta_i, 2))) /
442                               (_cos_theta_i + sqrt(eps_r - pow(_sin_theta_i, 2)));
443
444     // Phase thickness (q)
445     qreal q = beta_0 * wall->thickness * sqrt(eps_r - pow(_sin_theta_i, 2));
446
447     // Slab transmission (T_m)
448     complex<qreal> num = (1.0 - pow(Gamma_perp, 2)) * exp(-j * q);
449     complex<qreal> den = 1.0 - pow(Gamma_perp, 2) * exp(-2.0 * j * q);
450     return num / den;
451 }
452
453 bool Simulation::checkSameSideOfWall(const QVector2D& _normal, const QVector2D& _TX, const QVector2D& _RX) {
454     // returns true if _TX and _RX are on the same side of the wall (using the wall's normal vector)
455     // must be same sign to be true:
456     bool res = (QVector2D::dotProduct(_normal,_RX)>0 == QVector2D::dotProduct(_normal,_TX)>0);
457     return res;
458 }
459
460 bool Simulation::checkRaySegmentIntersectsWall(const Obstacle* wall, RaySegment* ray_segment, QPointF* intersection_point)
461     // returns true if ray_segment intersects wall

```

```

462 // the intersection_point pointer's value is set with the intersection point coordinates if they intersect
463 int _intersection_type = ray_segment->intersects(wall->line, intersection_point); // also writes to intersection point
464 bool intersects_wall = _intersection_type==1 ? true: false; //0: no intersection (parallel), 1: intersects directly
465 return intersects_wall;
466 }
467
468 void Simulation::showView()
469 {
470     qDebug() << "Creating graphics view...";
471     QGraphicsScene* sim_scene = createGraphicsScene();
472     this->scene = sim_scene;
473     this->view = new QGraphicsView(sim_scene); // create user's view showing the graphics scene
474     this->view->setAttribute(Qt::WA_DeleteOnClose); // close the view when user clicks on the close button (and not just t
475
476     this->view->setAttribute(Qt::WA_AlwaysShowToolTips); //? maybe necessary ?
477
478     //this->view->setFixedSize(990, 720);
479     int view_width = (max_x+10)*10; int view_height = (max_y+20)*10;
480     this->view->setFixedSize(view_width, view_height); // new map
481     //this->view->scale(6, 6);
482     this->view->scale(1, 1); // new map
483     qDebug() << "Showing graphics view";
484     //QIcon _icon = QIcon(QDir::currentPath()+"/icon.png");
485     //view->setWindowIcon(QIcon("./assets/icon.png"));
486     view->setWindowIcon(QIcon(":/assets/icon.png"));
487     this->view->show(); // shows the graphics scene to the user
488 }
489
490 void Simulation::createCellsMatrix()
491 {
492     int max_x_count = ceil(max_x/this->resolution); // -1 ?
493     //qDebug() << "Max count of cells X:" << max_x_count;
494     int max_y_count = ceil(max_y/this->resolution); // -1 ?
495     //qDebug() << "Max count of cells Y:" << max_y_count;
496
497     qDebug() << "Creating cells matrix" << max_x_count << "x" << max_y_count << "...";
498     //qDebug() << "cells matrix initial size:" << this->cells.size();
499     for (int x_count=0; x_count < max_x_count; x_count++) {
500         //qDebug() << "Creating new line of cells_matrix...";
501         qreal x = this->resolution/2+(this->resolution*x_count);
502         QList<Receiver*> temp_list;
503         for (int y_count=0; y_count < max_y_count; y_count++) {
504             qreal y = this->resolution/2+(this->resolution*y_count);
505             temp_list.append(new Receiver(x,y,this->resolution, this->show_cell_outline));
506             //qDebug() << "cells_matrix line"<< x_count << "size:" << temp_list.size();
507         }
508         this->cells.append(temp_list);
509         //qDebug() << "cells_matrix size:" << this->cells.size();
510     }
511     qDebug() << "cells_matrix created";
512 }
513
514 Transmitter* Simulation::getBaseStation(int index)
515 {
516     if (index < 0 || index >= this->baseStations.length()) {
517         qDebug() << "baseStations index out of range";
518         throw std::out_of_range("baseStations index out of range");
519     }
520     //return &this->baseStations.at(index);
521     return this->baseStations.at(index);
522 }
523
524 void Simulation::deleteBaseStation(int index)
525 {
526     if (index > 0 || index < this->baseStations.size())

```

```

527     {
528         this->baseStations.erase(this->baseStations.begin()+index);
529     } else if (index == 0)
530     {
531         qDebug("Cannot delete Base Station 1");
532     } else {
533         qWarning("deleteBaseStation error: index out of range");
534         throw std::out_of_range("deleteBaseStation error: index out of range");
535     }
536 }
537
538 QList<Obstacle*>* Simulation::getObstacles()
539 {
540     return &this->obstacles;
541 }
542
543 unsigned int Simulation::getNumberOfObstacles()
544 {
545     return this->obstacles.size();
546 }
547
548 unsigned int Simulation::getNumberOfBaseStations()
549 {
550     return this->baseStations.size();
551 }
552
553 quint64 Simulation::getSimulationTime() const
554 {
555     return this->simulation_time;
556 }
557
558 QGraphicsScene *Simulation::createGraphicsScene()//std::vector<Transmitter*>* TX
559 {
560     // creates the QGraphicsScene (to give to QGraphicsView) and adds all graphics to it
561     qDebug() << "Creating graphics scene...";
562
563     QGraphicsScene* scene = new QGraphicsScene();
564
565     //Transmitter* TX = this->baseStations[0];
566
567     for (QList<Receiver*>& cells_line : this->cells) {
568         for (Receiver* RX : cells_line) {
569             // compute total power and set it in RX
570             qreal _rx_power = 0.0;
571             //int _best_tx_id = -1;
572             Transmitter* _best_tx = nullptr;
573
574             for (Transmitter* tx : std::as_const(this->baseStations)) {
575                 qreal _pwr = RX->computeTotalPower(tx);
576
577                 // If it has higher power, update the best TX
578                 if (_pwr > _rx_power) {
579                     _rx_power = _pwr;
580                     _best_tx = tx;
581                 }
582             }
583
584             qreal _rx_power_dBm = 10.0 * std::log10(_rx_power * 1000.0);
585             RX->power = _rx_power;
586
587             // Only connect if the power is physically valid
588             if (_rx_power > 0.0) {
589                 RX->connected_tx = _best_tx;
590                 RX->connected_tx_selector_index = _best_tx ? _best_tx->selector_index : -1;
591             } else {

```

```

592         RX->connected_tx = nullptr;
593         RX->connected_tx_selector_index = -1;
594     }
595
596     // Coverage Area Calculation:
597     if (_best_tx != nullptr && _rx_power > 0.0) {
598         if (_rx_power_dBm >= min_sensitivity_dBm) {
599             // area of one square cell is resolution * resolution
600             qreal cell_area = resolution * resolution;
601             _best_tx->coverage_area_sqm += cell_area;
602         }
603     }
604
605     RX->updateBitrateAndColor();
606     QBrush _rxBrush = RX->graphics->brush();
607     if (this->showRaySingleCell) {
608         _rxBrush.setColor(Qt::black);
609     } else {
610         _rxBrush.setColor(RX->cell_color);
611     }
612     RX->graphics->setBrush(_rxBrush);
613
614     // Draw RX and add its tooltip
615     //QDebug() << "RX power:" << _rx_power << "W," << _rx_power_dBm << "dBm";
616     //QDebug() << "RX bitrate:" << RX->bitrate_Mbps << "Mbps";
617
618     if (RX->connected_tx_selector_index != -1) {
619         qreal bitrate_Mbps = RX->bitrate_Mbps;
620         if (bitrate_Mbps >= 1000) {
621             qreal bitrate_Gbps = bitrate_Mbps/1000;
622             RX->graphics->setToolTip(
623                 //QString("Receiver cell:\nx=%1 y=%2\nPower: %3 mW | %4 dBm\nBitrate: %5 Gbps\nDirect ray (%7 segm
624                 QString("Receiver cell:\nx=%1 y=%2\n(Total Incoherent Power)\nPower: %3 mW | %4 dBm\nBitrate: %5
625                 QString::number(RX->x()),
626                 QString::number(RX->y()),
627                 QString::number(_rx_power*1000),
628                 QString::number(_rx_power_dBm, 'f', 2),
629                 QString::number(bitrate_Gbps, 'f', 2),
630                 QString::number(RX->connected_tx_selector_index)
631                 //QString::number(RX->all_rays.first()->coeffsList.length()),
632                 //QString::number(RX->all_rays.first()->segments.length())
633                 );
634         } else {
635             RX->graphics->setToolTip(
636                 //QString("Receiver cell:\nx=%1 y=%2\nPower: %3 mW | %4 dBm\nBitrate: %5 Mbps\nDirect ray (%7 segm
637                 QString("Receiver cell:\nx=%1 y=%2\n(Total Incoherent Power)\nPower: %3 mW | %4 dBm\nBitrate: %5
638                 QString::number(RX->x()),
639                 QString::number(RX->y()),
640                 QString::number(_rx_power*1000),
641                 QString::number(_rx_power_dBm, 'f', 2),
642                 QString::number(bitrate_Mbps),
643                 QString::number(RX->connected_tx_selector_index)
644                 //QString::number(RX->all_rays.first()->coeffsList.length()),
645                 //QString::number(RX->all_rays.first()->segments.length())
646                 )
647             );
648         }
649     } else {
650         RX->graphics->setToolTip(
651             //QString("Receiver cell:\nx=%1 y=%2\nPower: %3 mW | %4 dBm\nBitrate: %5 Mbps\nDirect ray (%7 segments
652             QString("Receiver cell:\nx=%1 y=%2\nDead Zone (Near-Field / No Signal)").arg(
653                 QString::number(RX->x()),
654                 QString::number(RX->y())
655                 //QString::number(RX->all_rays.first()->coeffsList.length()),
656                 //QString::number(RX->all_rays.first()->segments.length())

```

```

657         )
658     );
659 }
660 scene->addItem(RX->graphics);
661 //qDebug() << "RX.graphics:" << RX->graphics->rect();
662 }
663 }
664 qDebug() << "All receiver cells added to scene.";
665
666 // Draw all walls in wall_list
667 for (Obstacle* wall : std::as_const(this->obstacles)){
668     //qDebug() << "Adding wall to scene...";
669     scene->addItem(wall->graphics);
670 }
671 qDebug() << "All walls added to scene.";
672
673 for (Transmitter* TX : std::as_const(this->baseStations)) {
674     TX->graphics->setToolTip(
675         QString("Transmitter:\nx=%1 y=%2\nGain: %3\nPower: %4W\nCoverage Area: %5 m² (%6%)").arg(
676             QString::number(TX->x()),
677             QString::number(TX->y()),
678             QString::number(TX->gain),
679             QString::number(TX->power),
680             QString::number(TX->coverage_area_sqm),
681             QString::number(100*(TX->coverage_area_sqm)/(max_x*max_y), 'f', 2) // percentage of max possible area of ma
682         )
683     );
684     //qDebug() << "TX.graphics:" << TX->graphics->rect();
685     scene->addItem(TX->graphics);
686     qDebug() << "Transmitter added to scene.";
687 }
688
689 if (this->showRaySingleCell) {
690     // Draw all rays (their segments) from the all_rays list
691
692     for (Ray* ray : std::as_const(this->cells[0][0]->all_rays)) {
693         qDebug() << "Drawing ray";
694         for (QGraphicsLineItem* segment_graphics : ray->getSegmentsGraphics()) {
695             scene->addItem(segment_graphics);
696         }
697     }
698
699     /// -- TEST only direct --
700     //#ifdef DEBUG_SINGLE_CELL_DIRECT_RAY
701     //for (QGraphicsLineItem* gra : this->cells[0][0]->all_rays.first()->getSegmentsGraphics()) {
702     //    scene->addItem(gra);
703     //}
704     //qDebug() << "number of coeffs:" << this->cells[0][0]->all_rays.first()->coeffsList.length();
705     //qDebug() << "T_m=" << this->cells[0][0]->all_rays.first()->coeffsList.first().real() << "+j" << this->cells[0][0]
706     //#endif
707     /// --
708
709     qDebug() << "All rays added to scene.";
710
711     for (QVector2D point : std::as_const(this->singleCellSimReflectionPoints)) {
712         qreal width = 0.04;
713         QGraphicsEllipseItem* reflection_graphics = new QGraphicsEllipseItem(10*(point.x()-width),10*(point.y()-width),
714             10*(point.x()+width),10*(point.y()+width));
715         reflection_graphics->setPen(QPen(Qt::green));
716         reflection_graphics->setBrush(QBrush(Qt::green));
717         reflection_graphics->setToolTip("Reflection point");
718         scene->addItem(reflection_graphics);
719     }
720     qDebug() << "All reflection points added to scene.";
721 }

```

```

722
723     addLegend(scene);
724
725     scene->setBackgroundBrush(QBrush(Qt::black)); // black background
726
727     qDebug() << "Scene created.";
728
729     return scene;
730 }
731
732 void Simulation::addLegend(QGraphicsScene* scene)
733 {
734     qDebug() << "Adding legend to scene";
735
736     QPen legendPen(Qt::white);
737     legendPen.setWidthF(1.5);
738
739     QFont legendFont;
740     legendFont.setPointSize(13); // Standard readable font size instead of using setScale()
741     QFont smallFont;
742     smallFont.setPointSize(9);
743
744     if (!this->showRaySingleCell) {
745
746         QLinearGradient gradient;
747
748         //QRectF rect((max_x * 10) / 2.0 - 138.0 / 2.0, (max_y * 10) + 8, 138, 10); // gradient rectangle scene coordinate
749         qreal bar_width = 600.0;
750         qreal bar_height = 40.0;
751         QRectF rect((max_x * 10) / 2.0 - bar_width / 2.0, (max_y * 10) + 40, bar_width, bar_height);
752
753         QGraphicsRectItem* gradient_graphics = new QGraphicsRectItem(rect);
754
755         gradient.setCoordinateMode(QGradient::ObjectBoundingMode);
756         gradient.setColorAt(0.0, QColor::fromRgb(255,0,0)); // blue
757         gradient.setColorAt(0.25, QColor::fromRgb(255,255,0)); // cyan
758         gradient.setColorAt(0.5, QColor::fromRgb(0,255,0)); // green
759         gradient.setColorAt(0.75, QColor::fromRgb(0,255,255)); // yellow
760         gradient.setColorAt(1.0, QColor::fromRgb(0,0,255)); // red
761         gradient.setStart(1.0, 0.0); // start left
762         gradient.setFinalStop(0.0, 0.0); // end right
763
764         QBrush gradientBrush(gradient);
765         gradient_graphics->setPen(legendPen);
766         gradient_graphics->setBrush(gradientBrush);
767         scene->addItem(gradient_graphics); // draw gradient rectangle
768
769         qreal rect_width = rect.width();
770         for (int i=0; i<5; i++) {
771             QGraphicsLineItem* small_line = new QGraphicsLineItem(rect.bottomLeft().x()+0.25*i*rect_width,rect.bottomLeft
772             small_line->setPen(legendPen);
773             scene->addItem(small_line);
774         }
775
776         // new: Function to easily add aligned text
777         auto addLegendText = [&](QString text, qreal x, qreal y) {
778             QGraphicsTextItem* t = new QGraphicsTextItem(text);
779             t->setDefaultTextColor(Qt::white);
780             t->setFont(legendFont);
781             // Center the text roughly on the X coordinate
782             t->setPos(x - (t->boundingRect().width() / 2.0), y);
783             scene->addItem(t);
784         };
785
786         // Draw small lines and text underneath the gradient

```

```

787     for (int i=0; i<5; i++) {
788         qreal x_pos = rect.bottomLeft().x() + 0.25 * i * bar_width;
789
790         QGraphicsLineItem* small_line = new QGraphicsLineItem(
791             x_pos, rect.bottomLeft().y(),
792             x_pos, rect.bottomLeft().y() + 8.0);
793         small_line->setPen(legendPen);
794         scene->addItem(small_line);
795     }
796
797     qreal quarter_bitrate_Mbps = pow(10.0, (log10(min_bitrate_Mbps) + 0.25*(log10(max_bitrate_Mbps)-log10(min_bitrate_Mbps)));
798     qreal middle_bitrate_Mbps = pow(10.0, (log10(min_bitrate_Mbps) + 0.5*(log10(max_bitrate_Mbps)-log10(min_bitrate_Mbps)));
799     qreal three_quarters_bitrate_Mbps = pow(10.0, (log10(min_bitrate_Mbps) + 0.75*(log10(max_bitrate_Mbps)-log10(min_bitrate_Mbps)));
800
801     qreal text_y = rect.bottomLeft().y() + 10.0;
802     addLegendText(QString::number(min_bitrate_Mbps)+" Mbps", rect.bottomLeft().x(), text_y);
803     addLegendText(QString::number(quarter_bitrate_Mbps, 'f', 0)+" Mbps", rect.bottomLeft().x() + 0.25*bar_width, text_y);
804     addLegendText(QString::number(middle_bitrate_Mbps, 'f', 0)+" Mbps", rect.bottomLeft().x() + 0.5*bar_width, text_y);
805     addLegendText(QString::number(three_quarters_bitrate_Mbps, 'f', 0)+" Mbps", rect.bottomLeft().x() + 0.75*bar_width, text_y);
806     addLegendText(QString::number(max_bitrate_Mbps/1000)+" Gbps", rect.bottomRight().x(), text_y);
807
808 } else {
809     QGraphicsTextItem* ray_colors = new QGraphicsTextItem("Green line: Direct ray\nRed line: One-reflection ray\nYellow line: Two-reflection ray");
810     ray_colors->setPos((max_x * 10) / 2.0 - 100.0, (max_y * 10) + 20);
811     ray_colors->setScale(0.4);
812     ray_colors->setDefaultTextColor(Qt::white);
813     scene->addItem(ray_colors);
814 }
815
816 // axes legends :
817 qreal offset = -15.0; // Push axes slightly further out to fit text
818 QGraphicsLineItem* x_line = new QGraphicsLineItem(0, offset, max_x*10, offset);
819 QGraphicsLineItem* y_line = new QGraphicsLineItem(offset, 0, offset, max_y*10);
820 x_line->setPen(legendPen);
821 y_line->setPen(legendPen);
822 scene->addItem(x_line);
823 scene->addItem(y_line);
824
825 QGraphicsTextItem* x_label = new QGraphicsTextItem("x");
826 x_label->setPos(-25.0, offset - 30.0);
827 x_label->setFont(legendFont);
828 x_label->setDefaultTextColor(Qt::white);
829 scene->addItem(x_label);
830
831 QGraphicsTextItem* m_label = new QGraphicsTextItem("[m]");
832 m_label->setPos(offset - 35.0, offset - 20.0);
833 m_label->setFont(smallFont);
834 m_label->setDefaultTextColor(Qt::white);
835 scene->addItem(m_label);
836
837 QGraphicsTextItem* y_label = new QGraphicsTextItem("y");
838 y_label->setPos(offset - 40.0, -20.0);
839 y_label->setFont(legendFont);
840 y_label->setDefaultTextColor(Qt::white);
841 scene->addItem(y_label);
842
843 // X-Axis markers:
844 for (int x = 0; x <= max_x; x++) {
845     qreal tick_length = 2.0; // Small tick for every 1m
846
847     if (x % 10 == 0) {
848         tick_length = 6.0; // Large tick for every 10m
849     } else if (x % 5 == 0) {
850         tick_length = 4.0; // Medium tick for every 5m
851     }

```

```

852
853 // Draw the tick line
854 QGraphicsLineItem* tick_line = new QGraphicsLineItem(x * 10, offset, x * 10, offset - tick_length);
855 tick_line->setPen(legendPen);
856 scene->addItem(tick_line);
857
858 // Only draw the text label if it is a 10m mark
859 if (x % 10 == 0) {
860     QGraphicsTextItem* x_index = new QGraphicsTextItem(QString::number(x));
861     x_index->setFont(smallFont);
862     x_index->setDefaultTextColor(Qt::white);
863     // Center text on the tick
864     x_index->setPos((x * 10) - (x_index->boundingRect().width() / 2.0), offset - 30.0);
865     scene->addItem(x_index);
866 }
867 }
868
869 // Y-Axis Markers:
870 for (int y = 0; y <= max_y; y++) {
871     qreal tick_length = 2.0; // Small tick for every 1m
872
873     if (y % 10 == 0) {
874         tick_length = 6.0; // Large tick for every 10m
875     } else if (y % 5 == 0) {
876         tick_length = 4.0; // Medium tick for every 5m
877     }
878
879 // Draw the tick line
880 QGraphicsLineItem* tick_line = new QGraphicsLineItem(offset, y * 10, offset - tick_length, y * 10);
881 tick_line->setPen(legendPen);
882 scene->addItem(tick_line);
883
884 // Only draw the text label if it is a 10m mark
885 if (y % 10 == 0) {
886     QGraphicsTextItem* y_index = new QGraphicsTextItem(QString::number(y));
887     y_index->setFont(smallFont);
888     y_index->setDefaultTextColor(Qt::white);
889     // Align text to the left of the tick
890     y_index->setPos(offset - 8.0 - y_index->boundingRect().width(), (y * 10) - (y_index->boundingRect().height() / 2.0));
891     scene->addItem(y_index);
892 }
893 }
894
895 }
896
897 // path loss scatter plot
898 QChartView* Simulation::showPathLossScatterPlot() // ONLY IF 1 BS !
899 {
900     if (this->baseStations.size() != 1) {
901         qWarning() << "Path Loss Scatter Plot is only available for exactly 1 Base Station.";
902         return nullptr;
903     }
904     QScatterSeries *scatterSeries = new QScatterSeries();
905     scatterSeries->setName("Simulated UE");
906     scatterSeries->setMarkerShape(QScatterSeries::MarkerShapeCircle);
907     scatterSeries->setMarkerSize(4.0);
908
909     // slightly transparent (alpha = 150) so overlapping points create a "density heat" effect
910     scatterSeries->setColor(QColor(0, 100, 255, 150));
911     scatterSeries->setBorderColor(Qt::transparent);
912
913     qreal min_pwr = 0.0;
914     qreal max_pwr = -200.0;
915     qreal max_dist = 1.0;
916     qreal safe_log_zero = 0.001; // Prevents log10(0) math crashes

```

```

917
918     qreal tx_pwr_dBm = this->baseStations[0]->getPower_dBm();
919     //qreal min_pl_threshold = tx_pwr_dBm - max_sensitivity_dBm;
920     //qreal max_pl_threshold = tx_pwr_dBm - min_sensitivity_dBm;
921
922     // Variables for Least Squares Linear Regression
923     qreal sum_x = 0;
924     qreal sum_y = 0;
925     qreal sum_xy = 0;
926     qreal sum_x2 = 0;
927     int N = 0;
928     QVector<QPointF> regression_data;
929
930     // 1. Extract the Data
931     for (const QList<Receiver*>& row : std::as_const(this->cells)) {
932         for (Receiver* rx : row) {
933             // Only plot valid cells that are connected to a transmitter
934             if (rx->power > 0.0 && rx->connected_tx != nullptr) {
935
936                 qreal d = rx->distanceToPoint(*(rx->connected_tx));
937
938                 // Enforce the far-field minimum distance (1m) so the Log axis doesn't crash on d=0
939                 if (d < 1.0) d = 1.0;
940
941                 qreal p_dBm = 10.0 * std::log10(rx->power * 1000.0); // received power
942                 //qreal pl_dB = tx_pwr_dBm - p_dBm; // PL
943
944                 scatterSeries->append(d, p_dBm); // received power
945                 //scatterSeries->append(d, pl_dB); // PL
946
947                 // Accumulate data for the log-normal regression fit
948                 // x = 10 * log10(d), y = P_dBm
949                 qreal x_val = 10.0 * std::log10(d);
950                 qreal y_val = p_dBm;
951                 //qreal y_val = pl_dB; // PL
952
953                 sum_x += x_val;
954                 sum_y += y_val;
955                 sum_xy += (x_val * y_val);
956                 sum_x2 += (x_val * x_val);
957                 N++;
958                 regression_data.append(QPointF(x_val, y_val));
959
960                 // Track bounds for dynamic axes
961                 if (p_dBm < min_pwr) min_pwr = p_dBm;
962                 if (p_dBm > max_pwr) max_pwr = p_dBm;
963                 //if (pl_dB < min_pwr) min_pwr = pl_dB; // PL
964                 //if (pl_dB > max_pwr) max_pwr = pl_dB; // PL
965
966                 if (d > max_dist) max_dist = d;
967             }
968         }
969     }
970
971     if (scatterSeries->count() == 0) {
972         qDebug() << "No valid data to plot.";
973         return nullptr;
974     }
975
976     // -- Compute log-normal regression (path loss law)
977     QLineSeries *fitSeries = new QLineSeries();
978     QLineSeries *sigmaPlusSeries = new QLineSeries();
979     QLineSeries *sigmaMinusSeries = new QLineSeries();
980     QLineSeries *friisSeries = new QLineSeries();
981     if (N > 1) {

```

```

982 // Calculate slope (m) and intercept (C) for  $y = mx + C$ 
983 qreal m = (N * sum_xy - sum_x * sum_y) / (N * sum_x2 - sum_x * sum_x);
984 qreal C = (sum_y - m * sum_x) / N;
985 qreal n = -m; // received power, Path Loss Exponent (PLE?)
986 //qreal n = m; // PL, Path Loss Exponent (PLE)
987
988 // Calculate Standard Deviation of Shadowing (sigma_L)
989 qreal variance_sum = 0.0;
990 for (const QPointF& pt : regression_data) {
991     qreal predicted_y = C + m * pt.x(); // x is already 10*log10(d)
992     variance_sum += std::pow(pt.y() - predicted_y, 2);
993 }
994 qreal sigma_L = std::sqrt(variance_sum / N);
995
996 // Setup Pens
997 QPen fitPen(Qt::red);
998 fitPen.setWidth(3);
999 fitSeries->setPen(fitPen);
1000 //fitSeries->setName(QString("Fit: Path loss exponent (n) = %1").arg(n, 0, 'f', 2));
1001 //fitSeries->setName(QString("Empirical fit: < P_RX(1m) > = %1 dBm, n = %2")
1002 //    .arg(C, 0, 'f', 1)
1003 fitSeries->setName(QString("Empirical fit (n = %1)")
1004     .arg(n, 0, 'f', 2));
1005
1006 QPen sigmaPen(Qt::green);
1007 sigmaPen.setWidth(2);
1008 sigmaPen.setStyle(Qt::DotLine);
1009 sigmaPlusSeries->setPen(sigmaPen);
1010 sigmaMinusSeries->setPen(sigmaPen);
1011 //sigmaPlusSeries->setName(QString("+/- 1\u03C3 Shadowing (%1 dB)").arg(sigma_L, 0, 'f', 1));
1012 sigmaPlusSeries->setName(QString("\u00B1\u03C3 (Shadowing Variability \u03C3_L = %1 dB)")
1013     .arg(sigma_L, 0, 'f', 1));
1014
1015 QPen friisPen;
1016 friisPen.setColor("gold");
1017 friisPen.setWidth(2);
1018 friisPen.setStyle(Qt::DashLine);
1019 friisSeries->setPen(friisPen);
1020 friisSeries->setName("Theoretical: Friis Free Space (n=2)");
1021
1022 // Generate the curve using 100 points so it bends correctly on a linear axis
1023 int num_points = 100;
1024 qreal step = (max_dist - 1.0) / num_points;
1025 // Convert linear gains (1.64) to decibels
1026 qreal tx_gain_dB = 10.0 * std::log10(G_TX);
1027 qreal rx_gain_dB = 10.0 * std::log10(G_RX);
1028 for (int i = 0; i <= num_points; i++) {
1029     qreal d_val = 1.0 + i * step;
1030     qreal log_d = 10.0 * std::log10(d_val);
1031
1032     // Regression Mean & Sigma bands
1033     qreal p_val_mean = C + m * log_d;
1034     fitSeries->append(d_val, p_val_mean);
1035     sigmaPlusSeries->append(d_val, p_val_mean + sigma_L);
1036     sigmaMinusSeries->append(d_val, p_val_mean - sigma_L);
1037
1038     // qreal p_val = C - n * 10.0 * std::log10(d_val); // received power
1039     // qreal pl_val = C + n * 10.0 * std::log10(d_val); // PL
1040     // fitSeries->append(d_val, p_val); // received power
1041     // //fitSeries->append(d_val, pl_val); // PL
1042
1043     // theoretical Friis equation
1044     // PRX = PTX + G_TX + G_RX - FSPL
1045     qreal p_val_friis = tx_pwr_dBm + tx_gain_dB + rx_gain_dB - (20.0 * std::log10((4.0 * M_PI * d_val) / lambda));
1046     friisSeries->append(d_val, p_val_friis);

```

```

1047     }
1048 }
1049
1050 // 2. Build the Chart
1051 QChart *chart = new QChart();
1052 qreal x_max_rounded = std::ceil(max_dist / 10.0) * 10.0;
1053
1054 // -- Add UE Sensitivity Threshold Lines
1055 QPen thresholdPen(Qt::black);
1056 thresholdPen.setWidth(1);
1057 thresholdPen.setStyle(Qt::DashLine);
1058 // Min Sensitivity Line
1059 QLineSeries *minLimitSeries = new QLineSeries();
1060 //minLimitSeries->setName("UE Min Sensitivity Threshold");
1061 minLimitSeries->setPen(thresholdPen);
1062 minLimitSeries->append(safe_log_zero, min_sensitivity_dBm);
1063 minLimitSeries->append(x_max_rounded, min_sensitivity_dBm);
1064 //minLimitSeries->append(safe_log_zero, min_pl_threshold); // PL
1065 //minLimitSeries->append(x_max_rounded, min_pl_threshold); // PL
1066 // Max Sensitivity (Saturation) Line
1067 QLineSeries *maxLimitSeries = new QLineSeries();
1068 //maxLimitSeries->setName("UE Max Sensitivity Threshold");
1069 maxLimitSeries->setPen(thresholdPen);
1070 maxLimitSeries->append(safe_log_zero, max_sensitivity_dBm);
1071 maxLimitSeries->append(x_max_rounded, max_sensitivity_dBm);
1072 //maxLimitSeries->append(safe_log_zero, max_pl_threshold); // PL
1073 //maxLimitSeries->append(x_max_rounded, max_pl_threshold); // PL
1074
1075 // -- Add Near-Field Blackout Zone
1076 QLineSeries *nearFieldLower = new QLineSeries();
1077 nearFieldLower->append(safe_log_zero, -300.0); // Arbitrary extreme low
1078 nearFieldLower->append(far_field_min_distance, -300.0);
1079 QLineSeries *nearFieldUpper = new QLineSeries();
1080 nearFieldUpper->append(safe_log_zero, 100.0); // Arbitrary extreme high
1081 nearFieldUpper->append(far_field_min_distance, 100.0);
1082 QAreaSeries *nearFieldArea = new QAreaSeries(nearFieldUpper, nearFieldLower);
1083 nearFieldArea->setPen(Qt::NoPen); // No border line
1084 nearFieldArea->setBrush(QBrush(Qt::darkGray)); // Solid black fill
1085
1086 // -- Add series to chart
1087 // "bg" layer
1088 //chart->addSeries(nearFieldArea); // painted 1st
1089 // "mid" layer
1090 chart->addSeries(minLimitSeries); // painted 2nd
1091 chart->addSeries(maxLimitSeries); // painted 3rd
1092 // "top" layer
1093 chart->addSeries(scatterSeries); // painted 4th
1094 chart->addSeries(friisSeries);
1095 chart->addSeries(fitSeries);
1096 chart->addSeries(sigmaPlusSeries);
1097 chart->addSeries(sigmaMinusSeries);
1098 //
1099 chart->addSeries(nearFieldArea);
1100
1101 chart->setTitle(QString("Path Loss Scatter Plot (Power vs. Distance), TX power: %1 dBm").arg(QString::number(this->ba
1102
1103 // -- Larger Chart Title
1104 QFont titleFont = chart->titleFont();
1105 titleFont.setPointSize(16);
1106 titleFont.setBold(true);
1107 chart->setTitleFont(titleFont);
1108
1109 chart->legend()->setVisible(true);
1110 //chart->legend()->setAlignment(Qt::AlignBottom);
1111

```

```

1112 // -- hide threshold lines from legend
1113 const auto minMarkers = chart->legend()->markers(minLimitSeries);
1114 for (QLegendMarker *marker : minMarkers) {
1115     marker->setVisible(false);
1116 }
1117 const auto maxMarkers = chart->legend()->markers(maxLimitSeries);
1118 for (QLegendMarker *marker : maxMarkers) {
1119     marker->setVisible(false);
1120 }
1121 // -- hide near-field bg box
1122 const auto nearFieldMarkers = chart->legend()->markers(nearFieldArea);
1123 for (QLegendMarker *marker : nearFieldMarkers) {
1124     marker->setVisible(false);
1125 }
1126 // -- hide one of the -sigma legend because redundant with sigma (put +/-sigma later in legend)
1127 const auto sigmaMinusMarkers = chart->legend()->markers(sigmaMinusSeries);
1128 for (QLegendMarker *marker : sigmaMinusMarkers) {
1129     marker->setVisible(false);
1130 }
1131
1132 QFont font;
1133 font.setPointSize(12);
1134
1135 // -- X-Axis (Linear scale for Distance)
1136 // Round max distance up to the nearest 10 (e.g., 123m becomes 130m)
1137 QValueAxis *axisX = new QValueAxis();
1138 axisX->setTitleText("Distance (m)");
1139 axisX->setTitleFont(font);
1140 axisX->setRange(0.0, x_max_rounded);
1141 // Set a major tick (with text) every 10 meters
1142 axisX->setTickCount(static_cast<int>(x_max_rounded / 10.0) + 1);
1143 // Add 1 minor tick between majors (creates a visual mark every 5 meters)
1144 axisX->setMinorTickCount(1);
1145 axisX->setLabelFormat("%d"); // force integer display
1146 chart->addAxis(axisX, Qt::AlignBottom);
1147
1148 nearFieldArea->attachAxis(axisX);
1149 minLimitSeries->attachAxis(axisX);
1150 maxLimitSeries->attachAxis(axisX);
1151 scatterSeries->attachAxis(axisX);
1152 friisSeries->attachAxis(axisX);
1153 sigmaPlusSeries->attachAxis(axisX);
1154 sigmaMinusSeries->attachAxis(axisX);
1155 fitSeries->attachAxis(axisX);
1156
1157 // -- Y-Axis (Linear scale for dBm)
1158 // Round min down to nearest 10, and max up to nearest 10
1159 //qreal y_min_rounded = std::floor((min_pwr - 5.0) / 10.0) * 10.0;
1160 //qreal y_max_rounded = std::ceil((max_pwr + 5.0) / 10.0) * 10.0;
1161 qreal y_min_rounded = std::floor((min_pwr - 0.5) / 10.0) * 10.0;
1162 qreal y_max_rounded = std::ceil((max_pwr + 0.5) / 10.0) * 10.0;
1163 QValueAxis *axisY = new QValueAxis();
1164 axisY->setTitleText("Received Power (dBm)"); // received power
1165 //axisY->setTitleText("Path Loss (dB)"); // PL
1166 axisY->setTitleFont(font);
1167 axisY->setRange(y_min_rounded, y_max_rounded);
1168 // Set a major tick (with text) every 10 dBm
1169 int y_tick_count = static_cast<int>((y_max_rounded - y_min_rounded) / 10.0) + 1;
1170 axisY->setTickCount(y_tick_count);
1171 // Add 1 minor tick between majors (creates a visual mark every 5 dBm)
1172 axisY->setMinorTickCount(1);
1173 axisY->setLabelFormat("%d"); // force integer display
1174 chart->addAxis(axisY, Qt::AlignLeft);
1175
1176 nearFieldArea->attachAxis(axisY);

```

```

1177 minLimitSeries->attachAxis(axisY);
1178 maxLimitSeries->attachAxis(axisY);
1179 scatterSeries->attachAxis(axisY);
1180 friisSeries->attachAxis(axisY);
1181 sigmaPlusSeries->attachAxis(axisY);
1182 sigmaMinusSeries->attachAxis(axisY);
1183 fitSeries->attachAxis(axisY);
1184
1185 // 3. Create the UI Window (reuse image export code from TDL)
1186 QChartView *chartView = new QChartView(chart);
1187 chartView->setRenderHint(QPainter::Antialiasing);
1188
1189 QWidget *window = new QWidget();
1190 window->setWindowTitle("Path Loss Analysis");
1191 window->resize(1000, 600);
1192 window->setAttribute(Qt::WA_DeleteOnClose);
1193
1194 QVBoxLayout *mainLayout = new QVBoxLayout(window);
1195 mainLayout->addWidget(chartView, 1);
1196
1197 QHBoxLayout *buttonLayout = new QHBoxLayout();
1198
1199
1200 QPushButton *btnSavePng = new QPushButton("Save as PNG");
1201 btnSavePng->setMinimumHeight(30);
1202
1203 QPushButton *btnToggleX = new QPushButton("Toggle Log/Linear X-Axis");
1204 btnToggleX->setMinimumHeight(30);
1205
1206 buttonLayout->addWidget(btnSavePng);
1207 buttonLayout->addWidget(btnToggleX);
1208
1209 buttonLayout->addStretch();
1210 mainLayout->addLayout(buttonLayout);
1211
1212 // lambda function
1213 QObject::connect(btnSavePng, &QPushButton::clicked, [window, chartView]() {
1214     QString fileName = QFileDialog::getSaveFileName(window, "Save Chart", "", "PNG Image (*.png)");
1215     if (!fileName.isEmpty()) {
1216         chartView->grab().save(fileName);
1217     }
1218 });
1219
1220 QShortcut *saveShortcut = new QShortcut(QKeySequence::Save, window);
1221 // lambda function
1222 QObject::connect(saveShortcut, &QShortcut::activated, [window, chartView]() {
1223     QString fileName = QFileDialog::getSaveFileName(window, "Save Chart", "", "PNG Image (*.png)");
1224     if (!fileName.isEmpty()) chartView->grab().save(fileName);
1225 });
1226
1227 // lambda function
1228 QObject::connect(btnToggleX, &QPushButton::clicked, [chart, scatterSeries, minLimitSeries, maxLimitSeries, nearFieldArea]() {
1229
1230     // 1. Get the current X-Axis
1231     QAbstractAxis *oldAxisX = chart->axes(Qt::Horizontal).at(0); // or .constFirst(); rather than .first()
1232     bool isCurrentlyLinear = (oldAxisX->type() == QAbstractAxis::AxisTypeCategory || oldAxisX->type() == QAbstractAxis::AxisTypeLog);
1233
1234     // 2. Detach old axis from ALL series
1235     nearFieldArea->detachAxis(oldAxisX);
1236     minLimitSeries->detachAxis(oldAxisX);
1237     maxLimitSeries->detachAxis(oldAxisX);
1238     scatterSeries->detachAxis(oldAxisX);
1239     friisSeries->detachAxis(oldAxisX);
1240     sigmaPlusSeries->detachAxis(oldAxisX);
1241     sigmaMinusSeries->detachAxis(oldAxisX);

```

```

1242     fitSeries->detachAxis(oldAxisX);
1243
1244     // 3. Remove and delete the old axis
1245     chart->removeAxis(oldAxisX);
1246     delete oldAxisX;
1247
1248     // 4. Create and attach the new axis
1249     if (isCurrentlyLinear) {
1250         // --- Switch to LOGARITHMIC ---
1251         QLogValueAxis *newAxisX = new QLogValueAxis();
1252         newAxisX->setTitleText("Distance (m) [Log Scale]");
1253         newAxisX->setTitleFont(font);
1254         newAxisX->setBase(10.0);
1255         newAxisX->setLabelFormat("%g");
1256
1257         // add 8 minor ticks for the Log grid look
1258         newAxisX->setMinorTickCount(8);
1259
1260         // Log axis cannot start at 0. Start at 1.0 (or your far_field_min_distance).
1261         newAxisX->setRange(1.0, max_dist + 20.0);
1262
1263         chart->addAxis(newAxisX, Qt::AlignBottom);
1264
1265         nearFieldArea->attachAxis(newAxisX);
1266         minLimitSeries->attachAxis(newAxisX);
1267         maxLimitSeries->attachAxis(newAxisX);
1268         scatterSeries->attachAxis(newAxisX);
1269         friisSeries->attachAxis(newAxisX);
1270         sigmaPlusSeries->attachAxis(newAxisX);
1271         sigmaMinusSeries->attachAxis(newAxisX);
1272         fitSeries->attachAxis(newAxisX);
1273
1274     } else {
1275         // --- Switch to LINEAR ---
1276         QValueAxis *newAxisX = new QValueAxis();
1277         newAxisX->setTitleText("Distance (m)");
1278         newAxisX->setTitleFont(font);
1279         newAxisX->setRange(0.0, x_max_rounded);
1280         newAxisX->setTickCount(static_cast<int>(x_max_rounded / 10.0) + 1);
1281         newAxisX->setMinorTickCount(1);
1282         newAxisX->setLabelFormat("%d");
1283
1284         chart->addAxis(newAxisX, Qt::AlignBottom);
1285
1286         nearFieldArea->attachAxis(newAxisX);
1287         minLimitSeries->attachAxis(newAxisX);
1288         maxLimitSeries->attachAxis(newAxisX);
1289         scatterSeries->attachAxis(newAxisX);
1290         friisSeries->attachAxis(newAxisX);
1291         sigmaPlusSeries->attachAxis(newAxisX);
1292         sigmaMinusSeries->attachAxis(newAxisX);
1293         fitSeries->attachAxis(newAxisX);
1294     }
1295 });
1296
1297 window->show();
1298 return chartView;
1299 }
1300
1301

```

C.12 validate_freespace

C.12.1 validate_freespace.h

```
1  #ifndef VALIDATE_FREESPACE_H
2  #define VALIDATE_FREESPACE_H
3
4  #include <QWidget>
5
6
7  QWidget* createFreeSpaceValidationPlot();
8
9  #endif // VALIDATE_FREESPACE_H
```

C.12.2 validate_freespace.cpp

```
1  #include "validate_freespace.h"
2  #include "validate_common_utils.h"
3
4  #include <QtCharts/QChartView>
5  #include <QtCharts/QLineSeries>
6  #include <QtCharts/QValueAxis>
7  #include <QtCharts/QLogValueAxis>
8  #include <QFileDialog>
9  #include <QPushButton>
10 #include <QVBoxLayout>
11 #include <cmath>
12 #include <complex>
13
14 QWidget* createFreeSpaceValidationPlot() {
15     // 0. Simulation Parameters
16     constexpr double freq = 26e9; // 26 GHz
17     constexpr double c = 3e8;
18     constexpr double lambda = c / freq;
19     constexpr double beta = (2.0 * M_PI) / lambda; // Phase constant
20
21     constexpr double p_tx_dBm = 20.0;
22     constexpr double g_tx_linear = 1.64; // lambda/2 dipole
23     constexpr double g_rx_linear = 1.64;
24
25     double g_tx_dB = 10.0 * std::log10(g_tx_linear);
26     double g_rx_dB = 10.0 * std::log10(g_rx_linear);
27
28     // 1. Generate Simulated Ray-Tracer Output (Thick Solid Line)
29     QLineSeries *simSeries = new QLineSeries();
30     simSeries->setName("Simulated");
31     QPen simPen(Qt::red);
32     simPen.setWidth(5); // thick line so the dotted one shows on top
33     simSeries->setPen(simPen);
34
35     // 2. Generate Theoretical Data
36     QLineSeries *friisSeries = new QLineSeries();
37     friisSeries->setName("Friis Free-Space (n=2)");
38     QPen friisPen(Qt::cyan);
39     friisPen.setWidth(2);
40     friisPen.setStyle(Qt::DotLine);
41     friisSeries->setPen(friisPen);
42
43     double max_dist = 100.0;
44     int num_points = 500;
45     double step = max_dist / num_points;
46 }
```

```

47     double min_pwr = 0; // For dynamic Y-axis
48     double max_pwr = -200;
49
50     for (int i = 1; i <= num_points; i++) {
51         double d = i * step;
52
53         // Simulation logic: calculate via Complex E-Field (Like your ray engine)
54         std::complex<double> E_direct = std::polar(1.0 / d, -beta * d);
55         double power_factor = std::norm(E_direct); // |E|^2
56
57         double p_rx_sim_dBm = p_tx_dBm + g_tx_dB + g_rx_dB +
58             20.0 * std::log10(lambda / (4.0 * M_PI)) +
59             10.0 * std::log10(power_factor);
60
61         simSeries->append(d, p_rx_sim_dBm);
62
63         // Friis theoretical equation: PRX = PTX + GTX + GRX - 20*log10(4*pi*d / lambda)
64         double friis_dB = 20.0 * std::log10((4.0 * M_PI * d) / lambda);
65         double p_rx_dBm = p_tx_dBm + g_tx_dB + g_rx_dB - friis_dB;
66
67         friisSeries->append(d, p_rx_dBm);
68
69         if (p_rx_dBm < min_pwr) min_pwr = p_rx_dBm;
70         if (p_rx_dBm > max_pwr) max_pwr = p_rx_dBm;
71     }
72
73     // 3. Build the Chart
74     QChart *chart = new QChart();
75     chart->addSeries(simSeries);
76     chart->addSeries(friisSeries);
77     chart->setTitle("Validation: Free-Space Propagation (26 GHz)");
78     QFont titleFont = chart->titleFont();
79     titleFont.setPointSize(16);
80     titleFont.setBold(true);
81     chart->setTitleFont(titleFont);
82
83     // X-Axis (log scale to match slide 26)
84     QLogValueAxis *axisX = new QLogValueAxis();
85     axisX->setTitleText("Distance (m) [Log Scale]");
86     axisX->setBase(10.0);
87     axisX->setRange(1.0, 100.0);
88     axisX->setMinorTickCount(8);
89     axisX->setLabelFormat("%g");
90     chart->addAxis(axisX, Qt::AlignBottom);
91     simSeries->attachAxis(axisX);
92     friisSeries->attachAxis(axisX);
93
94     // Y-Axis
95     QValueAxis *axisY = new QValueAxis();
96     axisY->setTitleText("Received Power (dBm)");
97     axisY->setRange(std::floor(min_pwr / 10.0) * 10.0, std::ceil(max_pwr / 10.0) * 10.0);
98     chart->addAxis(axisY, Qt::AlignLeft);
99     simSeries->attachAxis(axisY);
100    friisSeries->attachAxis(axisY);
101
102    // 4. Create the UI Window
103    QChartView *chartView = new QChartView(chart);
104    chartView->setRenderHint(QPainter::Antialiasing);
105
106    QWidget *window = new QWidget();
107    window->setWindowTitle("Validation: Free-Space");
108    window->resize(1000, 600);
109    window->setAttribute(Qt::WA_DeleteOnClose);
110
111    QVBoxLayout *mainLayout = new QVBoxLayout(window);

```

```

112     mainLayout->addWidget(chartView, 1);
113
114     QHBoxLayout *buttonLayout = new QHBoxLayout();
115     QPushButton *btnSavePng = new QPushButton("Save Chart PNG");
116     QPushButton *btnScene = new QPushButton("View 2D Scene (d=50m)");
117     QPushButton *btnToggleAxis = new QPushButton("Toggle X-Axis (Log/Linear)");
118     buttonLayout->addWidget(btnToggleAxis);
119     buttonLayout->addWidget(btnSavePng);
120     buttonLayout->addWidget(btnScene);
121     mainLayout->addLayout(buttonLayout);
122
123     QObject::connect(btnSavePng, &QPushButton::clicked, [window, chartView]() {
124         QString fileName = QFileDialog::getSaveFileName(window, "Save Chart", "", "PNG Image (*.png)");
125         if (!fileName.isEmpty()) chartView->grab().save(fileName);
126     });
127
128     QObject::connect(btnScene, &QPushButton::clicked, []() {
129         QGraphicsScene* scene = new QGraphicsScene();
130         double scale = 10.0; // 10 pixels per meter
131         double d = 50.0 * scale;
132
133         // TX (Red) and RX (Blue)
134         scene->addEllipse(-5, -5, 10, 10, QPen(Qt::red), QBrush(Qt::red));
135         scene->addEllipse(d-5, -5, 10, 10, QPen(Qt::blue), QBrush(Qt::blue));
136
137         // Direct Ray
138         scene->addLine(0, 0, d, 0, QPen(Qt::green, 1, Qt::DashLine));
139
140         showValidationScene(scene, "Free-Space Geometry Snapshot (d=50m)");
141     });
142
143     QObject::connect(btnToggleAxis, &QPushButton::clicked, [chart, simSeries, friisSeries]() {
144         // Use constFirst() to avoid Clazy detaching warnings
145         QAbstractAxis *oldAxisX = chart->axes(Qt::Horizontal).constFirst();
146         bool isLog = (oldAxisX->type() == QAbstractAxis::AxisTypeLogValue);
147
148         // 1. Detach old axis
149         simSeries->detachAxis(oldAxisX);
150         friisSeries->detachAxis(oldAxisX);
151
152         // 2. Remove and delete
153         chart->removeAxis(oldAxisX);
154         delete oldAxisX;
155
156         // 3. Create and attach new axis
157         if (isLog) {
158             QValueAxis *newAxisX = new QValueAxis();
159             newAxisX->setTitleText("Distance (m)");
160             newAxisX->setRange(1.0, 100.0);
161             newAxisX->setTickCount(11);
162             chart->addAxis(newAxisX, Qt::AlignBottom);
163             simSeries->attachAxis(newAxisX);
164             friisSeries->attachAxis(newAxisX);
165         } else {
166             QLogValueAxis *newAxisX = new QLogValueAxis();
167             newAxisX->setTitleText("Distance (m) [Log Scale]");
168             newAxisX->setBase(10.0);
169             newAxisX->setRange(1.0, 100.0);
170             newAxisX->setMinorTickCount(8);
171             newAxisX->setLabelFormat("%g");
172             chart->addAxis(newAxisX, Qt::AlignBottom);
173             simSeries->attachAxis(newAxisX);
174             friisSeries->attachAxis(newAxisX);
175         }
176     });

```

```

177
178     return window;
179 }
180

```

C.13 validate_wall

C.13.1 validate_wall.h

```

1  #ifndef VALIDATE_WALL_H
2  #define VALIDATE_WALL_H
3
4  #include <QWidget>
5
6  QWidget* createTwoRayValidationPlot();
7
8  #endif // VALIDATE_WALL_H

```

C.13.2 validate_wall.cpp

```

1  #include "validate_wall.h"
2  #include "validate_common_utils.h"
3
4  #include <QtCharts/QChartView>
5  #include <QtCharts/QLineSeries>
6  #include <QtCharts/QValueAxis>
7  #include <QFileDialog>
8  #include <QLogValueAxis>
9  #include <QPushButton>
10 #include <QVBoxLayout>
11 #include <cmath>
12 #include <complex>
13
14 QWidget* createTwoRayValidationPlot() {
15     // 1. Simulation Parameters
16     constexpr double freq = 26e9; // 26 GHz
17     constexpr double c = 3e8;
18     constexpr double lambda = c / freq;
19     constexpr double beta = (2.0 * M_PI) / lambda;
20
21     constexpr double p_tx_dBm = 20.0;
22     constexpr double g_tx_linear = 1.64;
23     constexpr double g_rx_linear = 1.64;
24
25     double g_tx_dB = 10.0 * std::log10(g_tx_linear);
26     double g_rx_dB = 10.0 * std::log10(g_rx_linear);
27
28     // Environment: Wall is parallel to the TX-RX line
29     constexpr double epsilon_r = 4.0; // Relative permittivity of the wall
30     constexpr double w = 5.0; // Perpendicular distance from TX/RX to the wall (meters)
31
32     // 2. Generate Data
33     QLineSeries *twoRaySeries = new QLineSeries();
34     twoRaySeries->setName("Two-Ray Interference (Wall at 5m)");
35     QPen twoRayPen(Qt::red);
36     twoRayPen.setWidth(2);
37     twoRaySeries->setPen(twoRayPen);
38
39     // -- Theoretical
40     // Friis
41     QLineSeries *friisSeries = new QLineSeries(); // Baseline for comparison

```

```

42     friisSeries->setName("Friis Free-Space (n=2)");
43     QPen friisPen(Qt::blue);
44     friisPen.setStyle(Qt::DotLine);
45     friisPen.setWidth(2);
46     friisSeries->setPen(friisPen);
47
48     // Over-The-Ground Asymptote (n=4)
49     QLineSeries *otgSeries = new QLineSeries();
50     otgSeries->setName("Over-the-Ground model (n=4)");
51     QPen otgPen(Qt::green);
52     otgPen.setStyle(Qt::DashLine);
53     otgPen.setWidth(2);
54     otgSeries->setPen(otgPen);
55
56     double max_dist = 100000.0; // /\ QUITE SLOW !!!
57     //int num_points = 2000; // High resolution needed to capture high-frequency 26 GHz fringes
58     int num_points = 200000; // for increased max dist to 10000m
59     double step = max_dist / num_points;
60
61     qreal min_y_range = -150.0;
62     qreal max_y_range = 60.0;
63
64     double d_breakpoint = 4 * M_PI * (w * w) / lambda;
65     // Create the vertical line series
66     QLineSeries *bpSeries = new QLineSeries();
67     bpSeries->setName(QString("Breakpoint (d = %1 m)").arg(d_breakpoint, 0, 'f', 1));
68     QPen bpPen(Qt::darkGray);
69     bpPen.setStyle(Qt::DotLine);
70     bpPen.setWidth(2);
71     bpSeries->setPen(bpPen);
72     bpSeries->append(d_breakpoint, min_y_range);
73     bpSeries->append(d_breakpoint, max_y_range);
74
75     for (int i = 1; i <= num_points; i++) {
76         double d_horizontal = i * step; // x-axis separation
77
78         // Ray 1: Direct Path
79         double d1 = d_horizontal;
80         std::complex<double> E1 = std::polar(1.0 / d1, -beta * d1);
81
82         // Ray 2: Reflected Path (Image Method)
83         double d2 = std::sqrt(std::pow(d_horizontal, 2) + std::pow(2.0 * w, 2));
84
85         // Angle of incidence with the normal of the wall
86         double theta_i = std::atan(d_horizontal / (2.0 * w));
87
88         // Perpendicular reflection coefficient (assuming vertical dipoles, E-field is normal to incidence plane)
89         double cos_theta_i = std::cos(theta_i);
90         double sin_theta_i = std::sin(theta_i);
91         double gamma = (cos_theta_i - std::sqrt(epsilon_r - std::pow(sin_theta_i, 2))) /
92             (cos_theta_i + std::sqrt(epsilon_r - std::pow(sin_theta_i, 2)));
93
94         std::complex<double> E2 = std::polar(gamma / d2, -beta * d2);
95
96         // Complex sum of the E-field phasors
97         std::complex<double> E_total = E1 + E2;
98
99         // Power calculation: std::norm returns the squared magnitude |z|^2
100        double power_factor = std::norm(E_total);
101
102        // P_RX = P_TX * G_TX * G_RX * (lambda / 4*pi)^2 * |E1 + E2|^2
103        double p_rx_dBm = p_tx_dBm + g_tx_dB + g_rx_dB +
104            20.0 * std::log10(lambda / (4.0 * M_PI)) +
105            10.0 * std::log10(power_factor);
106

```

```

107         twoRaySeries->append(d_horizontal, p_rx_dBm);
108
109         // Compute Friis for plotting
110         double p_rx_friis = p_tx_dBm + g_tx_dB + g_rx_dB - 20.0 * std::log10((4.0 * M_PI * d_horizontal) / lambda);
111         friisSeries->append(d_horizontal, p_rx_friis);
112
113         // Compute Over-the-ground fro plotting
114         double p_rx_otg = p_tx_dBm + g_tx_dB + g_rx_dB + 40.0 * std::log10(w) - 40.0 * std::log10(d_horizontal);
115         otgSeries->append(d_horizontal, p_rx_otg);
116     }
117
118     // 3. Build the Chart
119     QChart *chart = new QChart();
120     chart->addSeries(twoRaySeries);
121     chart->addSeries(bpSeries); // breakpoint line
122     chart->addSeries(friisSeries);
123     chart->addSeries(otgSeries);
124     chart->setTitle("Validation: Single Reflection Path (26 GHz)");
125     QFont titleFont = chart->titleFont();
126     titleFont.setPointSize(16);
127     titleFont.setBold(true);
128     chart->setTitleFont(titleFont);
129
130     // X-Axis (Linear to clearly see the spatial frequency of the fringes)
131     QValueAxis *axisX = new QValueAxis();
132     axisX->setTitleText("Distance (m)");
133     axisX->setRange(1.0, 100000.0);
134     axisX->setTickCount(11);
135     chart->addAxis(axisX, Qt::AlignBottom);
136     friisSeries->attachAxis(axisX);
137     twoRaySeries->attachAxis(axisX);
138     otgSeries->attachAxis(axisX);
139     bpSeries->attachAxis(axisX);
140
141     // Y-Axis
142     QValueAxis *axisY = new QValueAxis();
143     axisY->setTitleText("Received Power (dBm)");
144     //axisY->setRange(-110.0, -30.0); // Fixed bounds to clearly show the deep fades
145     axisY->setRange(min_y_range, max_y_range); // Raise the roof and lower the floor fro the super long distances
146     chart->addAxis(axisY, Qt::AlignLeft);
147     friisSeries->attachAxis(axisY);
148     twoRaySeries->attachAxis(axisY);
149     otgSeries->attachAxis(axisY);
150     bpSeries->attachAxis(axisY);
151
152     // 3. Create the UI Window
153     QChartView *chartView = new QChartView(chart);
154     chartView->setRenderHint(QPainter::Antialiasing);
155
156     QWidget *window = new QWidget();
157     window->setWindowTitle("Validation: Two-Ray");
158     window->resize(1000, 600);
159     window->setAttribute(Qt::WA_DeleteOnClose);
160
161     QVBoxLayout *mainLayout = new QVBoxLayout(window);
162     mainLayout->addWidget(chartView, 1);
163
164     QHBoxLayout *buttonLayout = new QHBoxLayout();
165     QPushButton *btnSavePng = new QPushButton("Save Chart PNG");
166     QPushButton *btnScene = new QPushButton("View 2D Scene (d=50m)");
167     QPushButton *btnToggleAxis = new QPushButton("Toggle X-Axis (Log/Linear)");
168     buttonLayout->addWidget(btnToggleAxis);
169     buttonLayout->addWidget(btnSavePng);
170     buttonLayout->addWidget(btnScene);
171     mainLayout->addLayout(buttonLayout);

```

```

172
173 QObject::connect(btnSavePng, &QPushButton::clicked, [window, chartView]() {
174     QString fileName = QFileDialog::getSaveFileName(window, "Save Chart", "", "PNG Image (*.png)");
175     if (!fileName.isEmpty()) chartView->grab().save(fileName);
176 });
177
178 QObject::connect(btnScene, &QPushButton::clicked, []() {
179     QGraphicsScene* scene = new QGraphicsScene();
180     double scale = 10.0;
181     double w = 5.0 * scale; // Wall at Y = 5m
182     double d = 50.0 * scale;
183
184     // Wall
185     scene->addLine(0, w, d, w, QPen(Qt::black, 3));
186
187     // TX and RX
188     scene->addEllipse(-5, -5, 10, 10, QPen(Qt::red), QBrush(Qt::red));
189     scene->addEllipse(d-5, -5, 10, 10, QPen(Qt::blue), QBrush(Qt::blue));
190
191     // Direct Ray
192     scene->addLine(0, 0, d, 0, QPen(Qt::green, 1, Qt::DashLine));
193
194     // Reflected Ray (Specular point is exactly at d/2 due to symmetry)
195     scene->addLine(0, 0, d/2, w, QPen(Qt::magenta, 2));
196     scene->addLine(d/2, w, d, 0, QPen(Qt::magenta, 2));
197
198     scene->setSceneRect(-20, -20, d + 40, w + 40);
199     showValidationScene(scene, "Single Reflection Geometry Snapshot (d=50m)");
200 });
201
202 QObject::connect(btnToggleAxis, &QPushButton::clicked, [chart, twoRaySeries, friisSeries, otgSeries, bpSeries]() {
203     QAbstractAxis *oldAxisX = chart->axes(Qt::Horizontal).constFirst();
204     bool isLog = (oldAxisX->type() == QAbstractAxis::AxisTypeLogValue);
205
206     twoRaySeries->detachAxis(oldAxisX);
207     friisSeries->detachAxis(oldAxisX);
208     otgSeries->detachAxis(oldAxisX);
209     bpSeries->detachAxis(oldAxisX);
210
211     chart->removeAxis(oldAxisX);
212     delete oldAxisX;
213
214     if (isLog) {
215         QValueAxis *newAxisX = new QValueAxis();
216         newAxisX->setTitleText("Distance (m)");
217         newAxisX->setRange(1.0, 100000.0);
218         newAxisX->setTickCount(11);
219         chart->addAxis(newAxisX, Qt::AlignBottom);
220         twoRaySeries->attachAxis(newAxisX);
221         friisSeries->attachAxis(newAxisX);
222         otgSeries->attachAxis(newAxisX);
223         bpSeries->attachAxis(newAxisX);
224     } else {
225         QLogValueAxis *newAxisX = new QLogValueAxis();
226         newAxisX->setTitleText("Distance (m) [Log Scale]");
227         newAxisX->setBase(10.0);
228         newAxisX->setRange(1.0, 100000.0);
229         newAxisX->setMinorTickCount(8);
230         newAxisX->setLabelFormat("%g");
231         chart->addAxis(newAxisX, Qt::AlignBottom);
232         twoRaySeries->attachAxis(newAxisX);
233         friisSeries->attachAxis(newAxisX);
234         otgSeries->attachAxis(newAxisX);
235         bpSeries->attachAxis(newAxisX);
236     }

```

```

237     });
238
239     return window;
240 }
241

```

C.14 validate_corridor

C.14.1 validate_corridor.h

```

1  #ifndef VALIDATE_CORRIDOR_H
2  #define VALIDATE_CORRIDOR_H
3
4  #include <QWidget>
5
6
7  QWidget* createThreeReflectionValidationPlot();
8
9  #endif // VALIDATE_CORRIDOR_H

```

C.14.2 validate_corridor.cpp

```

1  #include "validate_corridor.h"
2  #include "validate_common_utils.h"
3
4  #include <QtCharts/QChartView>
5  #include <QtCharts/QLineSeries>
6  #include <QtCharts/QValueAxis>
7  #include <QFileDialog>
8  #include <QLogValueAxis>
9  #include <QPushButton>
10 #include <QVBoxLayout>
11 #include <cmath>
12 #include <complex>
13
14 // Computes the Dielectric Slab Reflection Coefficient (Gamma_m)
15 std::complex<double> calcDielectricSlabReflection(double theta_i, double freq, double epsilon_r, double thickness) {
16     constexpr double c = 3e8;
17     double lambda = c / freq;
18     double beta = (2.0 * M_PI) / lambda;
19
20     double cos_theta = std::cos(theta_i);
21     double sin_theta = std::sin(theta_i);
22
23     // 1. Single interface perpendicular reflection coefficient
24     double root_term = std::sqrt(epsilon_r - std::pow(sin_theta, 2));
25     double gamma_perp = (cos_theta - root_term) / (cos_theta + root_term);
26
27     // 2. Phase delay inside the slab (q)
28     double q = beta * thickness * root_term;
29
30     // 3. Infinite internal reflections sum (Slab Coefficient)
31     std::complex<double> j(0.0, 1.0);
32     std::complex<double> exp_term = std::exp(-2.0 * j * q);
33
34     std::complex<double> gamma_m = gamma_perp * (1.0 - exp_term) / (1.0 - (std::pow(gamma_perp, 2) * exp_term));
35
36     return gamma_m;
37 }
38
39 QWidget* createThreeReflectionValidationPlot() {

```

```

40 // 1. Simulation Parameters
41 constexpr double freq = 26e9; // 26 GHz
42 constexpr double c = 3e8;
43 constexpr double lambda = c / freq;
44 constexpr double beta = (2.0 * M_PI) / lambda;
45
46 constexpr double p_tx_dBm = 20.0;
47 double g_tx_dB = 10.0 * std::log10(1.64); // lambda/2 dipole
48 double g_rx_dB = 10.0 * std::log10(1.64);
49
50 // Corridor Geometry & Material
51 constexpr double epsilon_r = 4.0;
52 constexpr double wall_thickness = 0.40; // 40cm concrete slab
53 constexpr double W = 5.0; // Distance from centerline to each wall (10m wide corridor)
54
55 // 2. Generate Theoretical Data
56 QLineSeries *corridorSeries = new QLineSeries();
57 corridorSeries->setName("Corridor/Canyon (up to 3 reflections)");
58 QPen corridorPen(Qt::red);
59 corridorPen.setWidth(2);
60 corridorSeries->setPen(corridorPen);
61
62 QLineSeries *friisSeries = new QLineSeries();
63 friisSeries->setName("Friis Free-Space (n=2)");
64 QPen friisPen(Qt::blue);
65 friisPen.setStyle(Qt::DashLine);
66 friisSeries->setPen(friisPen);
67
68 double max_dist = 200.0;
69 int num_points = 4000;
70 double step = max_dist / num_points;
71
72 for (int i = 1; i <= num_points; i++) {
73     double x = i * step;
74
75     std::complex<double> E_total(0.0, 0.0);
76
77     // Ray 0: Direct Path (0 Bounces)
78     E_total += std::polar(1.0 / x, -beta * x);
79
80     // Loop over 1-bounce, 2-bounce, and 3-bounce paths
81     for (int bounces = 1; bounces <= 3; bounces++) {
82         // Using image method: The apparent source moves 2*W further away per bounce
83         double image_y = bounces * (2.0 * W);
84         double d_n = std::sqrt(std::pow(x, 2) + std::pow(image_y, 2));
85
86         // Incidence angle is relative to the normal of the wall
87         double theta_i = std::atan(x / image_y);
88
89         std::complex<double> gamma_m = calcDielectricSlabReflection(theta_i, freq, epsilon_r, wall_thickness);
90
91         // The ray undergoes 'bounces' number of reflections, so we raise gamma_m to that power
92         std::complex<double> E_n = std::pow(gamma_m, bounces) * std::polar(1.0 / d_n, -beta * d_n);
93
94         // Multiply by 2 because the corridor is symmetric (Top->Bottom and Bottom->Top sequences)
95         E_total += 2.0 * E_n;
96     }
97
98     // Convert complex E-field sum to Received Power
99     double power_factor = std::norm(E_total);
100     double p_rx_dBm = p_tx_dBm + g_tx_dB + g_rx_dB +
101         20.0 * std::log10(lambda / (4.0 * M_PI)) +
102         10.0 * std::log10(power_factor);
103
104     corridorSeries->append(x, p_rx_dBm);

```

```

105
106 // Baseline friis
107 double p_rx_friis = p_tx_dBm + g_tx_dB + g_rx_dB - 20.0 * std::log10((4.0 * M_PI * x) / lambda);
108 friisSeries->append(x, p_rx_friis);
109 }
110
111 // 3. Build the Chart
112 QChart *chart = new QChart();
113 chart->addSeries(corridorSeries);
114 chart->addSeries(friisSeries);
115 chart->setTitle("Validation: Up to 3-Reflection Urban Canyon with Dielectric Slab modeled Walls (26 GHz)");
116 QFont titleFont = chart->titleFont();
117 titleFont.setPointSize(16);
118 titleFont.setBold(true);
119 chart->setTitleFont(titleFont);
120
121 QValueAxis *axisX = new QValueAxis();
122 axisX->setTitleText("Distance (m)");
123 axisX->setRange(1.0, max_dist);
124 axisX->setTickCount(11);
125 chart->addAxis(axisX, Qt::AlignBottom);
126 friisSeries->attachAxis(axisX);
127 corridorSeries->attachAxis(axisX);
128
129 QValueAxis *axisY = new QValueAxis();
130 axisY->setTitleText("Received Power (dBm)");
131 axisY->setRange(-110.0, -30.0);
132 chart->addAxis(axisY, Qt::AlignLeft);
133 friisSeries->attachAxis(axisY);
134 corridorSeries->attachAxis(axisY);
135
136 // 3. Create the UI Window
137 QChartView *chartView = new QChartView(chart);
138 chartView->setRenderHint(QPainter::Antialiasing);
139
140 QWidget *window = new QWidget();
141 window->setWindowTitle("Validation: Up to 3-Reflection Urban Canyon");
142 window->resize(1000, 600);
143 window->setAttribute(Qt::WA_DeleteOnClose);
144
145 QVBoxLayout *mainLayout = new QVBoxLayout(window);
146 mainLayout->addWidget(chartView, 1);
147
148 QHBoxLayout *buttonLayout = new QHBoxLayout();
149 QPushButton *btnSavePng = new QPushButton("Save Chart PNG");
150 QPushButton *btnScene = new QPushButton("View 2D Scene (d=50m)");
151 QPushButton *btnToggleAxis = new QPushButton("Toggle X-Axis (Log/Linear)");
152 buttonLayout->addWidget(btnToggleAxis);
153 buttonLayout->addWidget(btnSavePng);
154 buttonLayout->addWidget(btnScene);
155 mainLayout->addLayout(buttonLayout);
156
157 QObject::connect(btnSavePng, &QPushButton::clicked, [window, chartView]() {
158     QString fileName = QFileDialog::getSaveFileName(window, "Save Chart", "", "PNG Image (*.png)");
159     if (!fileName.isEmpty()) chartView->grab().save(fileName);
160 });
161
162 QObject::connect(btnScene, &QPushButton::clicked, []() {
163     QGraphicsScene* scene = new QGraphicsScene();
164     double scale = 10.0;
165     double W = 5.0 * scale;
166     double d = 50.0 * scale;
167
168     // Drawn manually
169

```

```

170 // Walls (Y = W and Y = -W)
171 scene->addLine(0, W, d, W, QPen(Qt::black, 3));
172 scene->addLine(0, -W, d, -W, QPen(Qt::black, 3));
173
174 // TX and RX
175 scene->addEllipse(-5, -5, 10, 10, QPen(Qt::red), QBrush(Qt::red));
176 scene->addEllipse(d-5, -5, 10, 10, QPen(Qt::blue), QBrush(Qt::blue));
177
178 // 0-Bounce (Direct)
179 scene->addLine(0, 0, d, 0, QPen(Qt::green, 1, Qt::DashLine));
180
181 // 1-Bounce (Top wall: 1 segment)
182 scene->addLine(0, 0, d/2, W, QPen(Qt::magenta, 1));
183 scene->addLine(d/2, W, d, 0, QPen(Qt::magenta, 1));
184 //scene->addLine(0, 0, d/2, -W, QPen(Qt::magenta, 1));
185 //scene->addLine(d/2, -W, d, 0, QPen(Qt::magenta, 1));
186
187 // 2-Bounce (Top then Bottom: divides X distance into 4 parts)
188 scene->addLine(0, 0, d/4, W, QPen(Qt::darkCyan, 1));
189 scene->addLine(d/4, W, 3*d/4, -W, QPen(Qt::darkCyan, 1));
190 scene->addLine(3*d/4, -W, d, 0, QPen(Qt::darkCyan, 1));
191 //scene->addLine(0, 0, d/4, -W, QPen(Qt::darkCyan, 1));
192 //scene->addLine(d/4, -W, 3*d/4, W, QPen(Qt::darkCyan, 1));
193 //scene->addLine(3*d/4, W, d, 0, QPen(Qt::darkCyan, 1));
194
195 // 3-Bounce (Top, Bottom, Top: divides X distance into 6 parts)
196 scene->addLine(0, 0, d/6, W, QPen(Qt::darkYellow, 1));
197 scene->addLine(d/6, W, 3*d/6, -W, QPen(Qt::darkYellow, 1));
198 scene->addLine(3*d/6, -W, 5*d/6, W, QPen(Qt::darkYellow, 1));
199 scene->addLine(5*d/6, W, d, 0, QPen(Qt::darkYellow, 1));
200 //scene->addLine(0, 0, d/6, -W, QPen(Qt::darkYellow, 1));
201 //scene->addLine(d/6, -W, 3*d/6, W, QPen(Qt::darkYellow, 1));
202 //scene->addLine(3*d/6, W, 5*d/6, -W, QPen(Qt::darkYellow, 1));
203 //scene->addLine(5*d/6, -W, d, 0, QPen(Qt::darkYellow, 1));
204
205 scene->setSceneRect(-20, -W - 20, d + 40, 2*W + 40);
206 showValidationScene(scene, "Urban Canyon Geometry Snapshot (d=50m), only showing half the rays (symmetric)");
207 });
208
209 QObject::connect(btnToggleAxis, &QPushButton::clicked, [max_dist, chart, corridorSeries, friisSeries]() {
210     QAbstractAxis *oldAxisX = chart->axes(Qt::Horizontal).constFirst();
211     bool isLog = (oldAxisX->type() == QAbstractAxis::AxisTypeLogValue);
212
213     corridorSeries->detachAxis(oldAxisX);
214     friisSeries->detachAxis(oldAxisX);
215
216     chart->removeAxis(oldAxisX);
217     delete oldAxisX;
218
219     if (isLog) {
220         QValueAxis *newAxisX = new QValueAxis();
221         newAxisX->setTitleText("Distance (m)");
222         newAxisX->setRange(1.0, max_dist);
223         newAxisX->setTickCount(11);
224         chart->addAxis(newAxisX, Qt::AlignBottom);
225         corridorSeries->attachAxis(newAxisX);
226         friisSeries->attachAxis(newAxisX);
227     } else {
228         QLogValueAxis *newAxisX = new QLogValueAxis();
229         newAxisX->setTitleText("Distance (m) [Log Scale]");
230         newAxisX->setBase(10.0);
231         newAxisX->setRange(1.0, max_dist);
232         newAxisX->setMinorTickCount(8);
233         newAxisX->setLabelFormat("%g");
234         chart->addAxis(newAxisX, Qt::AlignBottom);

```

```

235         corridorSeries->attachAxis(newAxisX);
236         friisSeries->attachAxis(newAxisX);
237     }
238 });
239
240     return window;
241 }
242

```

C.15 validate_common_utils

C.15.1 validate_common_utils.h

```

1  #ifndef VALIDATE_COMMON_UTILS_H
2  #define VALIDATE_COMMON_UTILS_H
3
4  #include <QGraphicsScene>
5
6  void showValidationScene(QGraphicsScene* scene, const QString& title);
7
8  #endif // VALIDATE_COMMON_UTILS_H

```

C.15.2 validate_common_utils.cpp

```

1  #include "validate_common_utils.h"
2
3  #include <QGraphicsView>
4  #include <QGraphicsScene>
5  #include <QPushButton>
6  #include <QFileDialog>
7  #include <QVBoxLayout>
8
9  void showValidationScene(QGraphicsScene* scene, const QString& title) {
10     QGraphicsView* view = new QGraphicsView(scene);
11     view->setRenderHint(QPainter::Antialiasing);
12     // invert y-axis to have it point up ?
13     view->scale(1, -1);
14
15     QWidget* window = new QWidget();
16     window->setWindowTitle(title);
17     window->resize(1000, 600);
18     window->setAttribute(Qt::WA_DeleteOnClose);
19
20     QVBoxLayout* layout = new QVBoxLayout(window);
21     layout->addWidget(view);
22
23     QPushButton* btnSave = new QPushButton("Save Scene as PNG");
24     layout->addWidget(btnSave);
25
26     QObject::connect(btnSave, &QPushButton::clicked, [window, view]() {
27         QString fileName = QFileDialog::getSaveFileName(window, "Save Scene", "", "PNG Image (*.png)");
28         if (!fileName.isEmpty()) {
29             QPixmap pixmap(view->viewport()->size());
30             view->viewport()->render(&pixmap);
31             pixmap.save(fileName);
32         }
33     });
34
35     window->show();
36 }
37

```

C.16 resources.qrc

```
1 <!DOCTYPE RCC>
2 <RCC version="1.0">
3   <qresource>
4     <file>assets/icon.png</file>
5   </qresource>
6 </RCC>
```

Bibliography

- [DD26] Philippe De Doncker. Wireless communication channels. ELEC-H415 Communication Channels Course syllabus, OPERA-WCG, École polytechnique de Bruxelles, Université libre de Bruxelles, Brussels, Belgium, 2026.
- [PH24] Lucas Placentino and Salman Houdaibi. Réalisation d'un logiciel de ray-tracing Wi-Fi IEEE 802.11ay. ELEC-H304 Rapport de Projet, Language: French, 2024.